Huawei HCIP Certification Training

HCIP-openGauss

Lab Guide

Issue: 1.0



Huawei Technologies Co., Ltd.

Copyright © Huawei Technologies Co., Ltd. 2024. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions


and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the development of ICT infrastructure that features Cloud-Pipe-Device synergy. Our certification is always evolving to reflect the latest trends in ICT development. Huawei Certification consists of three categories: ICT Infrastructure Certification, Basic Software & Hardware Certification, and Cloud Platform & Services Certification, making it the most extensive technical certification program in the industry.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

In keeping with the ICT convergence trend, Huawei Certification covers all ICT fields. Through its leading talent development system and certification standards, Huawei is committed to cultivating new ICT professionals in the digital era and building a healthy ICT talent ecosystem.

HCIP-openGauss certification is particular for senior openGauss engineers. It is intended to improve the skills of senior database development engineers, database administrators, and personnel dedicated to database development. The certification provides rich in-class exercises and industry cases to improve trainees' practical capabilities and promote the cultivation of database talent.

The Huawei certification system introduces the industry, fosters innovation, and imparts cutting-edge database knowledge.


Huawei Career Certification
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Basic Software & Hardware Domain
IT Infrastructure Domain
Datacom Domain
Optical Domain
Wireless Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
openEuler
openGauss
HarmonyOS
AI
Kunpeng
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom
WLAN
Security
Transmission
Access
5G
LTE
HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate

About This Document

Overview

This document is designed for the HCIP-openGauss V1.0 certification training. It is intended for trainees who are preparing for the HCIP-openGauss V1.0 exam or learners of the openGauss database architecture, advanced management of database objects, database system management, advanced SQL syntax, stored procedures and triggers, application development, database migration management, and database security management.

Description

This document consists of six labs, covering topics such as database object management, database object permission management, scheduled jobs, database system management, routine check and maintenance, advanced SQL syntax, functions, stored procedures and triggers, database migration management, database security management, and a comprehensive lab.

- Lab 1 is about database object management. Through the permission management of database objects and the query of system views and system catalogs, this lab aims to help readers get familiar with the management of database objects.
- Lab 2 is about database system management. Routine check and WDR reports help readers master advanced database O&M and management capabilities.
- Lab 3 covers advanced SQL topics, including advanced SQL syntax, function usage, stored procedures, and triggers. It aims to help readers master how to write complex SQL statements, stored procedures, and triggers.
- Lab 4 is about database migration. It aims to help readers master the principles of database migration by setting up and configuring migration tools and creating and running migration tasks.
- Lab 5 is about database security. It helps readers fully master database security management methods through security-related labs, such as SSL connection configuration, permission management, row-level security, and security audit.
- Lab 6 is a comprehensive lab. Through a real-word case, most knowledge points of the certification course are connected to help readers obtain the capability of operating actual database projects.

Background Knowledge Required

This course is for HCIP certification. To better understand this course, you need to:

- Have basic database knowledge and be familiar with Huawei openGauss and basic SQL syntax.
- Be able to compile common shell commands.
- Have the basic knowledge of Linux or Linux-like operating systems.
- Have knowledge of HCIA-openGauss certification courses.

Lab Environment

Networking Description

This lab environment is intended for database engineers who are preparing for the HCIP-openGauss V1.0 exam. For the lab environment, you need to purchase an Elastic Cloud Server (ECS) on Huawei Cloud and set up an openGauss database on the ECS. Each lab environment consists of one elastic IP address (EIP), one ECS server, openEuler 20.03LST, openGauss V5.0.0, and one gsql Client (for connecting to the database).

In this lab environment, standalone openGauss V5.0.0 (enterprise edition) is used.

To meet the HCIP-openGauss lab requirements, you are advised to use the following configurations in each lab environment:

The mapping between device, model, and version is as follows.

Device	Model	Software Version
ECS	8 vCPUs | 32 GiB	-
openGauss	-	V5.0.0
openEuler	-	V20.03
EIP	-	-

Contents

About This Document 3
Overview 3
Description 3
Background Knowledge Required 3
Lab Environment 4
1 Database Object Management 1
1.1 Introduction 1
1.1.1 About This Lab 1
1.1.2 Objectives 1
1.2 Creating Database Objects 1
1.2.1 Connecting to the Database 1
1.2.2 Creating Database Objects 2
1.3 System Catalogs and System Views 3
1.3.1 Viewing System Catalogs and System Views 3
1.3.2 Viewing and Stopping Running Query Statements 3
1.3.3 Querying Object Information 6
1.3.4 Querying Resource Monitoring View 7
1.4 Database Object Permission Management 10
1.4.1 Setting Object Access Permissions 10
1.4.2 Querying Access Permissions 12
1.4.3 Granting ANY Permissions 15
1.5 Scheduled Jobs 19
1.5.1 Basic Syntax of Scheduled Jobs 19
1.5.2 PKG_SERVICE Advanced Package 22
2 Database System Management 25
2.1 Introduction 25
2.1.1 About This Lab 25
2.1.2 Objectives 25
2.2 Routine Checks 25
2.2.1 Checking the Database Cluster Status 25
2.2.2 Generating Static Configuration Files 26
2.2.3 Checking the Database Status 27
2.2.4 Checking the OS 28
2.2.5 Database Parameters 29

2.2.6 Lock Waits 31
2.3 WDR 32
3 Advanced SQL Lab 34
3.1 Introduction 34
3.1.1 About This Lab 34
3.1.2 Objectives 34
3.2 Database Constraints 34
3.2.1 Procedure 34
3.3 Table Joins 38
3.3.1 Preparations 38
3.3.2 Procedure 40
3.4 UNION Clause 41
3.4.1 Preparations 41
3.4.2 Procedure 42
3.5 WHERE Clause 43
3.5.1 Preparations 43
3.5.2 Procedure 43
3.6 Grouping and Sorting 44
3.6.1 Preparations 44
3.6.2 Procedure 45
3.7 Subquery 46
3.7.1 Preparations 46
3.7.2 Derived Table 46
3.7.3 IN Operator 47
3.7.4 ALL Operator 47
3.7.5 ANY Operator 48
3.7.6 EXISTS Operator 48
3.7.7 Correlated Subquery 49
3.7.8 Using Subquery in INSERT and UPDATE Statements 49
3.8 Functions 50
3.8.1 Numeric Functions 50
3.8.2 Character Processing Functions 52
3.8.3 Time/Date Functions 53
3.8.4 Type Conversion Functions 53
3.8.5 System Information Functions 54
3.9 Stored Procedures and Triggers 54
3.9.1 Simple Stored Procedures 54
3.9.2 Stored Procedure with Parameters 55
3.9.3 Anonymous Blocks 56

3.9.4 Nested Subblocks....56
3.9.5 Conditional Statements....57
3.9.6 CASE Statements....57
3.9.7 Loop Statements....58
3.9.8 Error Trapping Statement....60
3.9.9 Triggers....60
3.10 API Development....62
3.10.1 JDBC....63
3.10.2 ODBC....68
3.10.3 Python....71
4 Database Migration Management....78
4.1 Introduction....78
4.1.1 About This Lab....78
4.1.2 Objectives....78
4.2 Environment Configuration....78
4.2.1 JDK Installation and Configuration....78
4.2.2 MySQL Installation and Configuration....79
4.2.3 Common MySQL Commands....82
4.2.4 openGauss Settings....82
4.3 DataKit Data Migration Practice....84
4.3.1 Deploying DataKit....84
4.3.2 Creating Data Sources on DataKit....86
4.3.3 Adding a Server on DataKit....87
4.3.4 Presetting Data in the MySQL Database....87
4.3.5 Creating a Migration Task....88
4.3.6 Installing Migration Plug-ins....88
4.3.7 Starting a Migration Task....89
4.4 DataKit Data Migration Summary....93
5 Database Security Management....94
5.1 Introduction....94
5.1.1 About This Lab....94
5.1.2 Objectives....94
5.2 SSL Connection Configuration....94
5.2.1 Setting Up the CA Environment....94
5.2.2 Creating a CA-Signed Certificate....96
5.2.3 SSL Connection Configuration (Creating a Server Certificate)....97
5.2.4 SSL Connection Configuration (Creating a Client Certificate)....99
5.2.5 SSL Connection Configuration (Configuring the Client Certificate)....101
5.3 Permission Management....102

5.3.1 User Permission Management 103
5.3.2 Granting Object Permissions to Users or Roles 104
5.3.3 Granting the Permissions of One User or Role to Others 105
5.4 Row-level Security 105
5.5 Database Security Audit 108
5.5.1 Database Security Audit 108
5.5.2 Unified Database Audit 110
5.6 Encrypted Equality Query 111
5.7 Ledger Database 117
5.7.1 Creating and Operating Tamper-proof User Tables 117
5.7.2 Viewing the Operation History of a Ledger Database 119
5.7.3 Checking Data Consistency of a Ledger Database 120
5.7.4 Archiving and Repairing a Ledger Database 120
6 Comprehensive Lab 123
6.1 Introduction 123
6.1.1 About This Lab 123
6.1.2 Objectives 124
6.1.3 Model Overview 124
6.2 Procedure 124
6.2.1 Creating Tables and Importing Data 124
6.2.2 Database Security Management 128
6.2.3 Advanced SQL Syntax 130
6.2.4 Application Development 132
6.2.5 Database Object Management 137
6.2.6 Health Check 139
6.2.7 Performance Check 147
6.2.8 Log Collection 150
6.2.9 Parameter Settings 152
6.2.10 Background Maintenance 153
6.2.11 WDR 155
6.2.12 Encrypted Query 156
6.2.13 Ledger Database 157

1 Database Object Management

1.1 Introduction

1.1.1 About This Lab

Database objects include database instances, tables, views, and indexes. Object creation and basic management have been introduced and practiced in the HCIA certification course. This lab describes advanced management and application of database objects, including exercises and application of system catalogs and views, exercises of system information functions and database object functions, permission management of database objects, development and management of scheduled jobs, and so on.

1.1.2 Objectives

This lab provides practical exercises on the knowledge covered in chapter 2 "Advanced Management of Database Objects" of the HCIP-openGauss V1.0 certification course. By completing this lab, you will gain a deeper understanding and the necessary skills that can be applied to real-world production.

1.2 Creating Database Objects

Create database objects such as users, database instances, schemas, tables, and views for this lab.

1.2.1 Connecting to the Database

Connect to the default database **postgres** using the gsql client. Before the connection, obtain the IP address of the server where the the primary node of the database is deployed and the port number of the primary database node.

Step 1 Log in as the OS user **omm** to a database node.

```
[root@ecs-opengauss ~]# su - omm
```

Step 2 Run the **gs_om -t status --detail** command to query openGauss instances.

```
[omm@ecs-opengauss ~]$ gs_om -t status --detail
```

Step 3 Check the port number of the database node.

Check the port number in the **postgresql.conf** file in the data directory of the primary database node obtained in step 2. The following is an example:

```
[omm@ecs-opengauss ~]$ cat /opt/huawei/install/data/dn/postgresql.conf | grep port
```

The results are as follows.



26000 is the port number of the primary database node.

Record the IP address, data path, and port number of the server where the primary database instance is located in actual operations, and replace them as required.

Step 4 Start the database service.

```
[root@ecs-opengauss ~]# gs_om -t start
```

Step 5 Connect to the database. Connect to the default database **postgres**.

```
[omm@ecs-opengauss /]$ gsql -d postgres -p 26000 -r
```

Step 6 (Optional) Exit the database.

```
openGauss=# \q
```

1.2.2 Creating Database Objects

Log in to the **postgres** database as the **omm** user and create the following objects:

Step 1 Create user **user_test**.

```
openGauss=# create user if not exists user_test password 'Test@123';
```

Step 2 Create database instance **db_test**.

```
openGauss=# create database if not exists db_test;
```

Step 3 Create schema **sche_test**.

```
openGauss=# create schema if not exists sche_test;
```

Step 4 Create table **tb_test**.

```
CREATE TABLE if not exists tb_test
```

```
(
    c_customer_sk             integer,
    c_customer_id             char(5),
    c_first_name              char(6),
    c_last_name               char(8),
    Amount                    integer
);
```

1.3 System Catalogs and System Views

1.3.1 Viewing System Catalogs and System Views

Step 1 Run the following meta-commands to view all system catalogs:

```
openGauss=# \dt pg_*
openGauss=# \dt gs_*
```

Note that system catalogs and views inherited from PostgreSQL have the prefix **PG**. PostgreSQL has 72 system catalogs while openGauss has 107 system catalogs.

Step 2 Run the following meta-commands to view all system views:

```
openGauss=# \dv pg_*
openGauss=# \dv gs_*
```

Step 3 View all tables in the public schema in the PG_TABLES system catalog.

```
openGauss=# SELECT distinct(tablename) FROM pg_tables WHERE SCHEMANAME = 'public';
```

Step 4 Use **PG_USER** to view the list of all users in the database, and view the user ID (**USESYSID**) and permissions.

```
openGauss=# SELECT * FROM pg_user;
```

1.3.2 Viewing and Stopping Running Query Statements

Step 1 Set **track_activities** to **on**. The database system collects the running information about active queries only if **track_activities** is **on**.

```
openGauss=# SET track_activities = on;
```

Step 2 View the running query statements.

View the database connected to each running query statement, user who issued each query, query status, and the corresponding PID. If the **state** column is **idle**, the connection is idle and is waiting for a user to enter a command.

```
openGauss=# SELECT datname, usename, state,pid FROM pg_stat_activity;
 datname   | usename | state   |          pid
```

```
----------+---------+--------+-----------------
 postgres | omm     | active | 281450627373424
 postgres | omm     | idle   | 281451069413744
 postgres | omm     | active | 281451105130864
 postgres | omm     | active | 281451246164336
 postgres | omm     | idle   | 281451277621616
 postgres | omm     | idle   | 281451323234672
 postgres | omm     | idle   | 281451389819248
 postgres | omm     | active | 281451495790960
(8 rows)
```

Step 3 View non-idle query statements.

```
openGauss=# SELECT datname, usename, state FROM pg_stat_activity WHERE state != 'idle';
 datname  | usename | state
----------+---------+--------
 postgres | omm     | active
 postgres | omm     | active
 postgres | omm     | active
 postgres | omm     | active
(4 rows)
```

Step 4 Terminate the session using its thread ID.

```
openGauss=# SELECT PG_TERMINATE_BACKEND(281451069413744);
 pg_terminate_backend
----------------------
 t
(1 row)
```

Note that if the PG_TERMINATE_BACKEND function is used to terminate the backend threads of the current session, the gsql client will be reconnected automatically rather than be logged out. The message "The connection to the server was lost. Attempting reset: Succeeded." is returned.

Step 5 Check the maximum number of database connections.

```
openGauss=# show max_connections;
 max_connections
-----------------
 5000
(1 row)
```

Step 6 Change the maximum number of connections to **1000** as user **user_test**.

To change the maximum number of connections, you must have the superuser permission. Log in to the **postgres** database as user **user_test** and change the number of connections.

```
openGauss=# \q
[omm@ecs-b168 ~]$

[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -U user_test -W 'Test@123' -r
```

```
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=>

openGauss=> show max_connections;
 max_connections
-----------------
 5000
(1 row)

openGauss=> alter system set max_connections= 1000;
ERROR:  Permission denied, must be sysadmin to execute ALTER SYSTEM SET command.
```

The result shows that the change fails. Grant the sysadmin permission to the user as the super administrator **omm**. Run the preceding command again. Then, restart the database and check the maximum number of connections.

```
openGauss=# GRANT ALL PRIVILEGES TO user_test;
ALTER ROLE
openGauss=# \q
[omm@ecs-b168 ~]$   gsql -d postgres -p 26000 -U user_test -W 'Test@123' -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=> alter system set max_connections= 1000;
openGauss=> \q
[omm@ecs-b168 ~]$ gs_om -t restart
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
Starting cluster.
=========================================
[SUCCESS] ecs-b168
2023-08-30 11:58:44.899 64eebe74.1 [unknown] 281459514212368 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
2023-08-30 11:58:44.899 64eebe74.1 [unknown] 281459514212368 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
=========================================
Successfully started.
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -U user_test -W 'Test@123' -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=> show max_connections;
 max_connections
-----------------
 1000
(1 row)
```

The result shows that the change is successful.

1.3.3 Querying Object Information

Query information about database objects or users through system catalogs and system views.

Step 1 Query tables and objects that contain columns or tables, including indexes and primary keys.

```
openGauss=# select * from pg_class;
```

Step 2 Query database information.

```
openGauss=#select * from pg_database;
```

Step 3 Query table information.

```
openGauss=#select * from pg_tables where tablename = 'tb_test';
```

Step 4 Query table column information.

```
openGauss=#select * from information_schema.columns where table_name = 'tb_test';
```

Step 5 Query check, primary key, unique, foreign key, and exclusion constraints.

```
openGauss=#select * from pg_constraint;
```

Step 6 Query table inheritance relationships.

```
openGauss=#select * from pg_inherits;
```

Note that **inhparent** corresponds to the parent table OID and **inhrelid** corresponds to the child table OID.

Step 7 Query the namespace, that is, schema.

```
openGauss=# select * from pg_namespace;
```

Step 8 Query database role information.

```
openGauss=# select * from pg_roles;
```

Note that **rolconnlimit** indicates the maximum number of connections.

Step 9 Query user access permission.

```
openGauss=# select * from pg_shadow;
```

Step 10 Query user information.

```
openGauss=# select * from pg_user;
```

Note that pg_user is the public readable view of pg_shadow, with the password field eliminated.

Step 11 Query View Information.

```
openGauss=# select * from pg_views;
```

1.3.4 Querying Resource Monitoring View

The real-time resource monitoring view records the resource usage (including memory, disk, CPU time, and I/O) and performance alarm information during job running.

openGauss provides resource monitoring views at the query and operator levels for you to query real-time and historical top SQL records.

To use resource load management, check whether it is enabled. If no, you need to configure users and parameters.

Step 1 Check whether resource load management is enabled.

```
openGauss=# show use_workload_manager;
 use_workload_manager
----------------------
 off
(1 row)
```

The result shows that the function is not enabled.

Step 2 Check which users have the system administrator permission.

In openGauss, you can manage system resources only as a database administrator. Run the following statement to query user permissions:

```
openGauss=# SELECT rolname FROM pg_roles WHERE rolsystemadmin = 't';
  rolname
-----------
 omm
 user_test
(2 rows)
```

Step 3 Check which users have the login permission.

Resource load management can be applied only to users with the login permission. Run the following statement to query user permissions:

```
openGauss=# SELECT rolname FROM pg_roles WHERE rolcanlogin = 't';
  rolname
------------
 omm
 user_test2
 user_test
(3 rows)
```

If a user's login permission is revoked, the user's resource pool will be changed to **default_pool**.

Step 4 Enable resource load management.

Exit the database and enable resource load management.

```
openGauss=# \q
[omm@ecs-b168 ~]$ gs_guc set -N all -I all -c "use_workload_manager=on"
The gs_guc run with the following arguments: [gs_guc -N all -I all -c use_workload_manager=on
set ].
Begin to perform the total nodes: 1.
Popen count is 1, Popen success count is 1, Popen failure count is 0.
Begin to perform gs_guc for datanodes.
Command count is 1, Command success count is 1, Command failure count is 0.

Total instances: 1. Failed instances: 0.
ALL: Success to perform gs_guc!
```

Step 5 Restart the database to make the setting take effect.

```
[omm@ecs-b168 ~]$ gs_om -t restart
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
Starting cluster.
=========================================
[SUCCESS] ecs-b168
2023-08-30 14:42:31.662 64eee4d7.1 [unknown] 281464229462032 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
2023-08-30 14:42:31.662 64eee4d7.1 [unknown] 281464229462032 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
=========================================
Successfully started.
```

Step 6 Check that resource load management is enabled.

Log in to the **postgres** database as user **omm** and check that resource load management is enabled.

```
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=#   show use_workload_manager;
 use_workload_manager
----------------------
 on
(1 row)
```

The result shows that resource load management is enabled.

Step 7 View load management information about jobs being executed by the current user on the primary database node.

```
openGauss=# select * from dbe_perf.STATEMENT_COMPLEX_RUNTIME;
```

Step 8 View operators of the jobs that are being executed by the current user.

```
openGauss=# select * from dbe_perf.OPERATOR_RUNTIME;
```

Step 9 View historical load records.

DBE_PERF.STATEMENT_COMPLEX_HISTORY displays load management information about a completed job executed on the primary database node. This step requires that **use_workload_manager** and **enable_resource_track** have been set to **on**.

Set filter parameters.

```
openGauss=# resource_track_duration = 1min;
openGauss=# resource_track_cost = 0;
openGauss=# resource_track_level = 'query';
```

- **resource_track_level**: specifies the resource monitoring level of the current session. **none** indicates that resources are not monitored. **query** indicates that resources used at the query level are monitored. **operator** indicates that resources used at query and operator levels are monitored. The default value is **query**. This parameter is valid only when **enable_resource_track** is set to **on**.
- **resource_track_cost**: specifies the minimum execution cost for resource monitoring on statements in the current session. Value range: an integer ranging from –1 to *INT_MAX*. **–1** indicates that resource monitoring is disabled. A value ranging from **0** to **9** indicates that statements whose execution cost is at least 10 will be monitored. A value greater than or equal to **10** indicates that statements whose execution cost exceeds this value will be monitored. The default value is **100000**. This parameter is valid only when **enable_resource_track** is set to **on**.
- **resource_track_duration**: specifies the minimum statement execution time that determines whether information about jobs of a statement recorded in the real-time view will be dumped to a historical view after the statement is executed. Job information will be dumped from the real-time view (with the suffix **statistics**) to a historical view (with the suffix **history**) if the statement execution time is no less than this value. Value range: an integer ranging from 0 to *INT_MAX*. The unit is second. **0** indicates that historical information about all statements recorded in the real-time resource monitoring view are archived. If the value is greater than **0**, historical information will be archived when the statement execution time exceeds this value. The default value is **1min**. This parameter is valid only when **enable_resource_track** is set to **on**.

View load management records.

```
openGauss=# select * from DBE_PERF.STATEMENT_COMPLEX_HISTORY;
```

Step 10 View archived load management records.

STATEMENT_COMPLEX_HISTORY_TABLE displays load management records after jobs executed by the primary node of the database are complete. The displayed data is dumped from the kernel to the system catalog. To continue, ensure that **use_workload_manager** and **enable_resource_track** have been set to **on**.

To view archived load management records, run the following command:

```
openGauss=# select * from DBE_PERF.STATEMENT_COMPLEX_HISTORY_TABLE;
```

1.4 Database Object Permission Management

Permissions are classified into system permissions and object permissions.

System permissions are also regarded as user attributes, which can be specified when a user is being created or modified. System permissions include SYSADMIN, MONADMIN, OPRADMIN, POLADMIN, CREATEDB, CREATEROLE, AUDITADMIN, and LOGIN.

They can be specified only by the CREATE USER or ALTER USER statement. System permissions except SYSADMIN cannot be granted or revoked by the GRANT or REVOKE statement. In addition, system permissions cannot be inherited from roles.

1.4.1 Setting Object Access Permissions

After an object is created, only the object owner or system administrator can query, modify, and delete the object, and grant permissions for the object to other users through **GRANT** by default.

Only an object owner or system administrator can use the **GRANT** or **REVOKE** statement to grant or revoke object permissions.

Step 1 Revoke user **user_test**'s **sysadmin** and **LOGIN** permission, and grant it with the usage permission on the **sche_test** schema and all permissions on the **tb_test** table.

```
openGauss=# REVOKE ALL PRIVILEGES FROM user_test;
ALTER ROLE
openGauss=# ALTER USER user_test NOLOGIN;
ALTER ROLE
openGauss=# GRANT USAGE ON SCHEMA sche_test TO user_test;
GRANT
openGauss=# GRANT ALL PRIVILEGES ON tb_test TO user_test;
GRANT
```

Step 2 Grant the query permission on the **c_customer_sk**, **c_customer_id**, and **c_first_name** columns and the update permission on the **c_last_name** column in the **tb_test** table to user **user_test**.

```
openGauss=# GRANT select (c_customer_sk,c_customer_id,c_first_name),update (c_last_name) ON
tb_test TO    user_test;
GRANT
```

After the permissions are successfully granted, user **user_test** immediately has the query permission on the **c_customer_sk**, **c_customer_id**, **c_first_name** columns in the **tb_test** table.

To allow user **user_test** to grant these permissions to other users, execute the following statement:

```
openGauss=# GRANT select (c_customer_sk,c_customer_id,c_first_name),update (c_last_name) ON
tb_test TO    user_test WITH GRANT OPTION;
GRANT
```

Step 3 Connect to the **db_test** database as user **user_test** and grant permissions to the user.

```
[omm@ecs-b168 ~]$ gsql -d db_test -p 26000 -U user_test -W 'Test@123';
gsql: FATAL:    role "user_test" is not permitted to login
[omm@ecs-b168 ~]$
```

The result shows that the connection fails.

Grant the **db_test** database connection permission and schema creation permission to user **user_test** and allow this user to be able to grant those permissions to other users.

```
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr    )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# GRANT create,connect on database db_test TO user_test WITH GRANT OPTION;
GRANT
```

Step 4 Create the **test_manager** role.

```
openGauss=# CREATE ROLE test_manager PASSWORD 'openGauss123';
CREATE ROLE
```

Step 5 Grant permissions to the **test_manager** role.

Grant the login permission of the **db_test** database and the access and object creation permissions on the **sche_test** schema to the **test_manager** role, but do not allow this role to grant these permissions to others.

```
openGauss=# GRANT connect on database db_test TO test_manager;
GRANT
openGauss=# GRANT USAGE,CREATE ON SCHEMA sche_test TO test_manager;
GRANT
```

Step 6 Log in to the **db_test** database as the **test_manager** role.

```
openGauss=# \q
[omm@ecs-b168 ~]$ gsql -d db_test -p 26000 -U test_manager -W 'openGauss123' -r
gsql: FATAL:    role "test_manager" is not permitted to login
```

The result shows that the login fails. The role does not have the login permission.

Step 7 Create user **test_manager01**, assign role **test_manager** to the user, and log in to database **db_test** as user **test_manager01**.

```
openGauss=# create user test_manager01 password 'openGauss123';
CREATE ROLE
openGauss=# grant test_manager to test_manager01;
GRANT ROLE
openGauss=# \q
[omm@ecs-b168 ~]$   gsql -d db_test -p 26000 -U test_manager01 -W 'openGauss123' -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

db_test=>
```

Step 8 Create the **tpcds_tbspc** tablespace and grant all permissions on the tablespace to user **user_test**, but do not allow user **user_test** to grant these permissions to other users. (Log in to the **postgres** database as user **omm**.)

```
openGauss=# CREATE TABLESPACE tpcds_tbspc RELATIVE LOCATION 'tablespace/tablespace_1';
CREATE TABLESPACE
openGauss=# GRANT ALL ON TABLESPACE tpcds_tbspc TO user_test;
GRANT
```

1.4.2 Querying Access Permissions

The access permission query functions return whether a user has a certain permission on an object (that is, a permission on an object recorded in the **acl** column of a system catalog is returned).

Step 1 Check whether the current user has the permission to query the **tb_test** table.

(Log in to the **postgres** database as user **test_manager01**.)

```
openGauss=> SELECT has_table_privilege('tb_test', 'select');
 has_table_privilege
---------------------
 f
(1 row)
```

The result shows that this user does not have the query permission.

Grant this permission to this user as user **omm** and then perform the query again.

```
openGauss=> \q
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# grant select on tb_test to test_manager01;
GRANT
```

```
openGauss=# \q
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -U test_manager01 -W 'openGauss123' -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=> SELECT has_table_privilege('tb_test', 'select');
 has_table_privilege
---------------------
 t
(1 row)
```

has_table_privilege(*table*, *privilege*) specifies whether the current user has permissions for tables. The return type is Boolean.

Step 2 Check whether a specified user has the permission to query and insert data into the **tb_test** table.

Check whether user **omm** has the permission to query and insert data into the **tb_test** table and test whether the permission is held with the grant option.

```
openGauss=# SELECT has_table_privilege('omm', 'tb_test', 'select,INSERT WITH GRANT OPTION ');
 has_table_privilege
---------------------
 t
(1 row)
```

The result shows that user **omm** has these permissions.

Step 3 Check whether user **test_manager01** has the permission to query the **c_customer_sk** column in the **tb_test** table.

has_column_privilege(*user*, *table*, *column*, *privilege*):

```
openGauss=# select has_column_privilege('test_manager01', 'tb_test', 'c_customer_sk', 'select');
 has_column_privilege
----------------------
 t
(1 row)
```

Revoke user **test_manager01**'s permission to query the **tb_test** table, and perform the query again.

```
openGauss=# revoke select on tb_test from test_manager01;
REVOKE
openGauss=# select has_column_privilege('test_manager01', 'tb_test', 'c_customer_sk', 'select');
 has_column_privilege
----------------------
 f
(1 row)
```

The result shows that if a user's query permission on a table is revoked, the user does not have the query permission on any column in this table.

Run the following command to grant user **test_manager01** the permission to query the **c_customer_id** column in the **tb_test** table:

```
openGauss=# grant select (c_customer_id) on tb_test to test_manager01;
GRANT
```

Check whether user **test_manager01** has the permission to query the **tb_test** table's **c_customer_sk** and **c_customer_id** columns and the **tb_test** table.

```
openGauss=# select has_column_privilege('test_manager01', 'tb_test', 'c_customer_id', 'select');
 has_column_privilege
----------------------
 t
(1 row)
openGauss=# select has_column_privilege('test_manager01', 'tb_test', 'c_customer_sk', 'select');
 has_column_privilege
----------------------
 f
(1 row)
openGauss=# SELECT has_table_privilege('test_manager01', 'tb_test', 'select');
 has_table_privilege
---------------------
 f
(1 row)
```

The result shows that the user has only the permission to query the **c_customer_id** column but does not have the permission to query other columns or the table.

Step 4 Check the PUBLIC role's permissions on the **tb_test** table.

```
openGauss=# SELECT relname,relacl FROM pg_class WHERE relname = 'tb_test' and( CAST(relacl AS
TEXT) LIKE '{%,=%}' OR CAST(relacl AS TEXT) LIKE '{=%}');
 relname | relacl
---------+--------
(0 rows)
```

No data is returned. That is, the PUBLIC role does not have any permission on the **tb_test** table.

Grant the PUBLIC role the permission to query the **tb_test** table and perform the query again.

```
openGauss=# grant select on tb_test to public;
GRANT
openGauss=# SELECT relname,relacl FROM pg_class WHERE relname = 'tb_test' and( CAST(relacl AS
TEXT) LIKE '{%,=%}' OR CAST(relacl AS TEXT) LIKE '{=%}');
 relname |                                          relacl
---------+-------------------------------------------------------------------
 tb_test | {omm=arwdDxt/omm,user_test=arwdDxt/omm,user_test=APmiv/omm,=r/omm}
(1 row)
```

The result shows that the **omm** role has granted the read permission (SELECT permission) to the PUBLIC role. That is, the PUBLIC role has the permission to query the **tb_test** table.

The result of the **relacl** field is in either of the following formats:

- rolename=xxxx/yyyy -- Permissions granted to a role.
- =xxxx/yyyy -- Permissions granted to the PUBLIC role.

Note that *xxxx* indicates the assigned permissions, and *yyyy* indicates the role who has assigned the permissions.

Create user **test_manager02** and check whether the user has the permission to query the **tb_test** table.

```
openGauss=# create user test_manager02 password 'openGauss123';
CREATE ROLE
openGauss=#   SELECT has_table_privilege('test_manager02', 'tb_test', 'select');
 has_table_privilege
---------------------
 t
(1 row)
```

The result shows that user has the permission.

Summary: The PUBLIC role belongs to all users. The keyword PUBLIC indicates that the permissions are granted to all roles, including any users created in future. By default, permissions on tables, columns, sequences, foreign data sources, foreign servers, schemas, and tablespaces are not granted to the PUBLIC role.

Permissions on the following objects are granted to the PUBLIC role: CONNECT and CREATE TEMP TABLE permissions on databases, EXECUTE permission on functions, and USAGE permission on languages and data types (including domains).

1.4.3 Granting ANY Permissions

ANY permissions are database permissions. If a user is granted with an ANY permission, the user has the corresponding permission on the public and user schemas except for the system schema. For example, SELECT ANY TABLE only allows a user to view all user table data in the current database.

These system permissions containing "ANY" are very powerful and should be granted only to database administrators, not common users.

The ANY permissions of openGauss are queried using the new system catalog gs_db_privilege.

If a user is granted with any ANY permission, the user has the USAGE permission on the public and user schemas but does not have the USAGE permission on the system schemas except public listed in Table 1-1.

Table 1-1 ANY permissions

System Permission	Description
CREATE ANY TABLE	DROP permission on tables or views in the public and user

System Permission	Description
	schemas.
SELECT ANY TABLE	SELECT permission on tables or views in the public and user schemas, which is still subject to row-level security.
UPDATE ANY TABLE	UPDATE permission on tables or views in the public and user schemas, which is still subject to row-level security.
INSERT ANY TABLE	INSERT permission on tables or views in the public and user schemas.
DELETE ANY TABLE	DELETE permission on tables or views in the public and user schemas, which is still subject to row-level security.
CREATE ANY FUNCTION	Permission to create functions or stored procedures in the user schemas.
EXECUTE ANY FUNCTION	EXECUTE permission on functions or stored procedures in the public and user schemas.
CREATE ANY PACKAGE	Permission to create packages in the public and user schemas.
EXECUTE ANY PACKAGE	EXECUTE permission on packages in the public and user schemas.
CREATE ANY TYPE	Permission to create types in the public and user schemas.
CREATE ANY SEQUENCE	Permission to create sequences in the public and user schemas.
CREATE ANY INDEX	Permission to create indexes in the public and user schemas. The users must be granted with the permission to create tablespaces to create a partitioned table index in a tablespace.
ALTER ANY TYPE	ALTER permission on types in public and user schemas, excluding modifying the owner of a type or modifying the schema of a type.
DROP ANY TYPE	DROP permission on types in the public and user schemas.
ALTER ANY SEQUENCE	ALTER permission on sequences in public and user schemas, excluding modifying the owner of a sequence.
DROP ANY SEQUENCE	DROP permission on sequences in the public and user schemas.
SELECT ANY SEQUENCE	SELECT, USAGE, and UPDATE permissions on sequences in the public and user schemas.
ALTER ANY INDEX	ALTER permission on indexes in the public and user

System Permission	Description
	schemas. To rename an index, users also need the permission to create objects in the schema where the index is located. If tablespace operations are involved, users need to have the corresponding operation permission on the tablespace. To set an index to **UNUSABLE**, users must have the DROP ANY INDEX permission.
DROP ANY INDEX	DROP permission on indexes in the public and user schemas.
CREATE ANY TRIGGER	Permission to create triggers in the public and user schemas.
ALTER ANY TRIGGER	ALTER permission on triggers in the public and user schemas.
DROP ANY TRIGGER	DROP permission on triggers in the public and user schemas.
CREATE ANY SYNONYM	Permission to create synonyms in user schema.
DROP ANY SYNONYM	DROP permission on synonyms in the public and user schemas.

The ANY permissions are database permissions and are valid only for database objects that are granted with the permissions. For example, SELECT ANY TABLE only allows a user to view all user table data in the current database, but the user does not have the permission to view user tables in other databases.

```
--The ANY permissions automatically override the public schema permissions.
--1. Log in to the postgres database as user omm.
gsql -d postgres -p 26000 -r;
--2. Create schemas sche_test2 and sche_test3, and create tables tb_test2 and tb_test3 in the two schemas respectively.
openGauss=# create schema sche_test2;
CREATE SCHEMA
openGauss=#    create schema sche_test3;
CREATE SCHEMA
openGauss=# create table sche_test2.tb_test2(a int,b int);
CREATE TABLE
openGauss=# create table sche_test3.tb_test3(a int,b int);
CREATE TABLE
--3. Grant the SELECT ANY TABLE permission on the sche_test2 schema to user user_test.
openGauss=# grant SELECT ANY TABLE to user_test;
GRANT
--4. Connect to the postgres database as user user_test.
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -U user_test -W 'Test@123' -r;
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=>
```

```
--5. Query data in tables tb_test, tb_test2, and tb_test3.
openGauss=> select * from tb_test;
 c_customer_sk | c_customer_id | c_first_name | c_last_name | amount
---------------+---------------+--------------+-------------+--------
(0 rows)

openGauss=> select * from sche_test2.tb_test2;
 a | b
---+---
(0 rows)

openGauss=> select * from sche_test3.tb_test3;
 a | b
---+---
(0 rows)
-- Conclusion: The table tb_test belongs to the default public schema, and the other two tables belong to different user schemas. With necessary permission granted, user user_test can query the three tables. Therefore, the SELECT ANY TABLE permission can be used to view tables in the public schema and all user schemas in the database.
--The ANY permission cannot be used to access objects of private users.
--1. Log in to the postgres database as user omm and create private user user_test2.
CREATE USER user_test2 WITH INDEPENDENT PASSWORD 'openGauss@123';
--2. Log in to the postgres database as user user_test2 and create table tb_test4.
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -U user_test2 -W 'openGauss@123' -r;
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=>
--3. Log in to the postgres database as user user_test and query the tb_test, tb_test2, and tb_test4 tables.
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -U user_test -W 'Test@123' -r;
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=> select * from tb_test;
 c_customer_sk | c_customer_id | c_first_name | c_last_name | amount
---------------+---------------+--------------+-------------+--------
(0 rows)

openGauss=> select * from sche_test2.tb_test2;
 a | b
---+---
(0 rows)

openGauss=> select * from user_test2.tb_test4;
ERROR:  permission denied for relation tb_test4
DETAIL:  N/A
openGauss=>
Conclusion: Users with ANY permissions cannot query tables owned by private users.
```

1.5 Scheduled Jobs

1.5.1 Basic Syntax of Scheduled Jobs

Operations related to scheduled jobs are supported only when **sql_compatibility** is set to **'B'**. This parameter specifies the type of mainstream database with which the SQL syntax and statement behavior of the database is compatible. This is an INTERNAL parameter. It can be viewed but cannot be modified. The default value is **A**.

A indicates that the database is compatible with Oracle databases. **B** indicates that the database is compatible with MySQL databases. **C** indicates that the database is compatible with Teradata databases. **PG** indicates that the database is compatible with PostgreSQL databases.

In the database, this parameter must be set to a specific value. It can be set to **A** or **B** and cannot be changed. Otherwise, the setting is not consistent with the database behavior.

This parameter can be set only when you run the **CREATE DATABASE** command to create a database.

1.5.1.1 One-off Scheduled Jobs

Step 1 Create a B-compatible database (that is, **sql_compatibility** = **'B'**) **my_database01**.

```
openGauss=# CREATE DATABASE my_database01 DBCOMPATIBILITY 'B';
CREATE DATABASE
```

Step 2 Log in to **my_database01** as user **omm** and check the **sql_compatibility** parameter.

```
[omm@ecs-b168 ~]$ gsql -d my_database01 -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

my_database01=# show sql_compatibility ;
 sql_compatibility
-------------------
 B
(1 row)
```

Step 3 Create table **event_table01**.

```
my_database01=#    create table event_table01(a int,b timestamp);
CREATE TABLE
```

Step 4 Create scheduled job **event01** that is executed once.

```
openGauss=# CREATE EVENT IF NOT EXISTS event01 ON SCHEDULE AT current_timestamp + interval
10 second + interval 3 minute DISABLE DO insert into event_table01 values(0,current_timestamp(3));
```

ON SCHEDULE AT is followed by the job execution time. **current_timestamp + interval 10 second + interval 3 minute** indicates that the job will be executed in 3 minutes and 10 seconds later than the current time. **DISABLE** indicates that the job is unavailable. This

parameter is not included by default, indicating that the job is available. **DO** is followed by the statement to be executed, which is a data insertion statement in this example.

Step 5 Query job **event01**.

You can run either of the following statements:

```
my_database01=# show events;
 job_name | schema_name | log_user | priv_user | job_status |            start_date            | interval |
end_date           | enable | failure_msg
----------+-------------+----------+-----------+------------+----------------------------+----------+-------------
-------+--------+-------------
 event01  | public      | omm      | omm       | s          | 2023-08-29 15:31:12.409041 | null
| 3999-12-31 16:00:00 | f      |
(1 row)
--Or,
my_database01=# show events in public like 'eve';
 job_name | schema_name | log_user | priv_user | job_status |            start_date            | interval |
end_date           | enable | failure_msg
----------+-------------+----------+-----------+------------+----------------------------+----------+-------------
-------+--------+-------------
 event01  | public      | omm      | omm       | s          | 2023-08-29 15:31:12.409041 | null
| 3999-12-31 16:00:00 | f      |
(1 row)
```

The current schema is public. Therefore, you can use the show events statement. Alternatively, you can use the second statement, where **in** is used to specify a mode, and **like** (optional) is used to match the job name.

Step 6 Modify the job to make it available.

```
my_database01=# alter event event01 enable;
ALTER EVENT
```

After the modification, pay attention to the current time. Query data in table **event_table01** immediately to check whether the job is executed.

```
my_database01=# select * from event_table01;
 a | b
---+---
(0 rows)
--3 minutes and 10 seconds later:
my_database01=# select * from event_table01;
 a |          b
---+------------------------
 0 | 2023-08-29 15:35:24+08
(1 row)
```

The query is performed immediately after the modification. In this case, no data exists in the table because the job is not executed. Wait for 3 minutes and 10 seconds and query again. If a record is displayed in the table, the job has been executed.

Step 7 After the job is executed, query again.

```
my_database01=# show events;
 job_name | schema_name | log_user | priv_user | job_status | start_date | interval | end_date | enable |
failure_msg
----------+-------------+----------+-----------+------------+------------+----------+----------+--------+---------
----
(0 rows)
```

If no job is displayed, the job is a one-off job. After the job is executed, the job is automatically deleted.

1.5.1.2 Cyclic Scheduled Jobs

Step 1 Create a scheduled job that is executed every minute.

```
my_database01=# CREATE EVENT IF NOT EXISTS event02 ON SCHEDULE EVERY 1 minute DO insert
into event_table01 values(1,current_timestamp(3));
CREATE EVENT
```

Step 2 Check the job execution status.

Query data in the table and check whether the job is executed every minute.

```
my_database01=# select * from event_table01;
 a |                b
---+------------------------
 0 | 2023-08-29 15:35:24+08
 1 | 2023-08-29 16:06:26+08
 1 | 2023-08-29 16:07:27+08
 1 | 2023-08-29 16:08:27+08
(4 rows)
```

The result shows that the job is running properly.

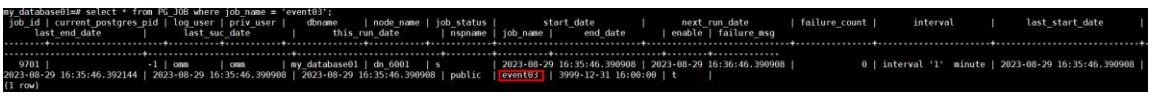
Step 3 Change the job name to **event03**.

```
my_database01=# alter event event02 rename to event03;
ALTER EVENT
```

Step 4 View the job in the PG_JOB system catalog.

```
my_database01=# select * from PG_JOB where job_name = 'event03';
```

The results are as follows.



Step 5 Delete scheduled job **event03**.

```
my_database01=# drop event event03;
DROP EVENT
```

1.5.2 PKG_SERVICE Advanced Package

This section describes how to manage jobs by using the APIs of the PKG_SERVICE advanced package.

Different from the lab in 1.5.1 Basic Syntax of Scheduled Jobs, this lab uses an A-compatible database (**sql_compatibility** set to **'A'**). This means that you need to log in to the **postgres** database as user **omm**.

Step 1 Log in to the **postgres** database as user **omm**.

```
[omm@ecs-b168 ~]$ gsql -d postgres -p 26000 -r
```

Check the **sql_compatibility** parameter:

```
openGauss=# show sql_compatibility;
 sql_compatibility
-------------------
 A
(1 row)
```

Step 2 Create table **event_table02**.

```
openGauss =# create table event_table02(id int);
CREATE TABLE
```

Step 3 Create sequence **seq01**.

```
openGauss =# create sequence seq01 start with 1 increment 1 nomaxvalue;
CREATE SEQUENCE
```

start with 1: indicates that the value starts from 1.

increment 1: indicates that the step is 1.

nomaxvalue: If **maxvalue** is not declared or **NO MAXVALUE** is declared, the default value of the ascending sequence is 2^{63}**-1**, and that of the descending sequence is **-1**. **NOMAXVALUE** is equivalent to **NO MAXVALUE**.

Step 4 Create stored procedure **job_test01**.

```
create or replace procedure job_test01
as
begin
insert into event_table02 values(seq01.nextval);
end;
/
```

Step 5 Query the stored procedure.

Check the OID of the stored procedure in the pg_proc system catalog.

```
openGauss =# select oid from pg_proc where proname = 'job_test01';
  oid
```

```
-------
 18975
(1 row)
```

Use the pg_get_functiondef(oid) function to view the definition of the stored procedure.

```
openGauss =# select * from pg_get_functiondef(18975);
 headerlines |                          definition
-------------+-----------------------------------------------------
           1 | CREATE DEFINER = omm PROCEDURE public.job_test01()+
             | AS    DECLARE                                      +
             | begin                                              +
             | insert into event_table02 values(seq01.nextval);   +
             | end;                                               +
             | /                                                  +
             |
(1 row)
```

Step 6 Create a scheduled job.

```
openGauss=# select pkg_service.job_submit(1,'select job_test01()',sysdate,'sysdate+1/(24*60)');
 job_submit
------------
          1
(1 row)
```

- **1**: job ID. If the input ID is **NULL**, a job ID is generated internally.
- **'select job_test01()'**: SQL statement to be executed. One or multiple DMLs, anonymous blocks, and statements for calling stored procedures, or all three combined are supported.
- **Sysdate**: specifies the next time the job will be executed. The default value is the current system time (**sysdate**). If the specified time has past, the job is executed immediately on submission.
- **'sysdate+1/(24*60)'**: calculates the next time to execute the job. It can be an interval expression, or sysdate followed by a numeric value, for example, **sysdate+1.0/24**. If this parameter is left empty or set to **null**, the job will be executed only once, and the job status will change to **'d'** afterward.

In addition, the job ID is omitted, because pkg_service.job_submit is called using SELECT.

sysdate+1/(24*60) indicates that the scheduled job runs every minute.

Step 7 Check the pg_job system catalog for scheduled jobs.

```
openGauss=# select * from pg_job;
```

Step 8 Check whether the scheduled job is successful.

View data in the **event_table02** table.

```
openGauss=# select * from event_table02;
 id
```

```
----
  1
  2
  3
```

The result shows that data is inserted into the table on time and the job is executed successfully.

Step 9 Invalidate the scheduled job.

```
openGauss=# select pkg_service.job_finish(1,true);
 job_finish
------------

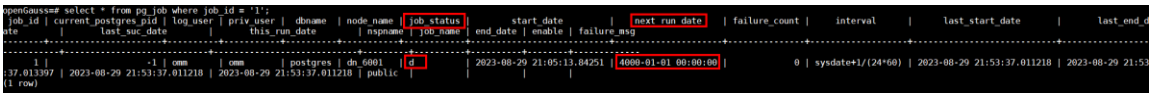
(1 row)
```

Step 10 Check the job status.

Because the scheduled job has been invalidated, its **job_status** is changed to **d**, and **next_run_date** is **4000-01-01 00:00:00**.

```
openGauss=# select * from pg_job where job_id = '1';
```

The results are as follows.



Step 11 Schedule the scheduled job to run once a day.

Note that before the modification, start the scheduled job **pkg_service.job_finish**.

```
openGauss=# select pkg_service.job_update(1,sysdate,'sysdate+1',null);
 job_update
------------

(1 row)
```

Step 12 Delete the scheduled job.

```
openGauss=# select pkg_service.job_cancel(1);
 job_cancel
------------

(1 row)
```

2 Database System Management

2.1 Introduction

2.1.1 About This Lab

In this lab, practice the use of openGauss tools such as gs_checkos, gs_guc, and gs_om for routinely checking and managing the database and OS. The following topics are covered: checking the cluster status, generating configuration files, checking the OS, modifying database parameters, and generating a WDR.

2.1.2 Objectives

By completing this lab, you will:

- Learn how to use openGauss tools, such as gs_checkos, gs_guc, and gs_om.
- Become proficient in performing routine checks and modifying parameters in the database system.
- Familiarize yourself with the necessary checks of the OS.
- Acquire the skills to generate a WDR, modify related parameters, and view a WDR.

2.2 Routine Checks

2.2.1 Checking the Database Cluster Status

Step 1 Check the database cluster status.

CMServer State and **Datanode State** information is displayed in the check result.

P: indicates that the initial role of the node is **Primary**. After the database is installed, the role does not change and is read from the system static file.

S: indicates that the initial role of the node is **Standby**. After the database is installed, the role does not change and is read from the system static file.

Primary indicates a primary instance, while **Standby** indicates a standby instance.

```
 [omm@Run861 ~]$ gs_om -t status --detail
[   CMServer State    ]

node         node_ip             instance                                    state
------------------------------------------------------------------------------
1   Run861 7.213.197.105    1       /opt/huawei/data/cmserver/cm_server Primary
```

```
2   Run862 7.213.202.227    2      /opt/huawei/data/cmserver/cm_server Standby
3   Run863 7.213.208.176    3      /opt/huawei/data/cmserver/cm_server Standby

[    Cluster State    ]

cluster_state    : Normal
redistributing   : No
balanced          : Yes
current_az        : AZ_ALL

[   Datanode State    ]

node         node_ip            instance                                state
-------------------------------------------------------------------------------
1   Run861 7.213.197.105    6001 /opt/huawei/install/data/dn P Primary Normal
2   Run862 7.213.202.227    6002 /opt/huawei/install/data/dn S Standby Normal
3   Run863 7.213.208.176    6003 /opt/huawei/install/data/dn S Standby Normal
```

2.2.2 Generating Static Configuration Files

Step 1 Use the gs_om tool to generate static configuration files.

The static configuration files of openGauss are essential for its proper functioning, because they define the cluster topology structure and primary/standby relationship.

```
[omm@Run861 openGauss]$ gs_om -t generateconf -X /opt/software/openGauss/cluster_config.xml --distribute
Generating static configuration files for all nodes.
Creating temp directory to store static configuration files.
Successfully created the temp directory.
Generating static configuration files.
Successfully generated static configuration files.
Static configuration files for all nodes are saved in /opt/huawei/install/om/script/static_config_files.
Distributing static configuration files to all nodes.
Successfully distributed static configuration files.
```

After the command is executed, the new configuration file directory is displayed in the log information. Take a one-primary dual-standby environment as an example. Three new configuration files, each named after their respective hosts, will be generated to this new directory. You need to replace the existing configuration files of their corresponding hosts with these new files.

If **--distribute** is specified, the static configuration files are automatically distributed to their corresponding hosts. If **--distribute** is not specified, you need to manually distribute static configuration files to their corresponding hosts as follows:

```
[omm@Run861 openGauss]$ cd /opt/huawei/install/om/script/static_config_files
total 276
-rwxr-x--- 1 omm dbgrp 106496 Oct 24 10:44 cluster_static_config_Run861
-rwxr-x--- 1 omm dbgrp 106496 Oct 24 10:44 cluster_static_config_Run862
-rwxr-x--- 1 omm dbgrp 106496 Oct 24 10:44 cluster_static_config_Run863
```

2.2.3 Checking the Database Status

The parameter **-i** or **-e** are mandatory. **-i** is used to check a specific item and **-e** is used to check multiple items in a specific scenario.

If no root item is specified in **-i** or**-e**, you will not be prompted to enter the name and password of user **root**.

You can also use **--skip-root-items** to skip the check of root items and hence avoid entering the name and password of user **root**.

If the nodes have different MTU values, the check may be slow down or even fail to respond. In this case, change the nodes' MTU values to be the same and then perform the check again.

result.rst: specifies the check result. This parameter is optional. Its value can be:

- **OK**: indicates that the check is passed.
- **NA**: indicates that the check is not applicable to the node.
- **NG**: indicates that the check fails.
- **WARNING**: indicates that the check is complete and that a warning is reported.
- **ERROR**: indicates that the check is interrupted due to an internal error.

Step 1 Perform a single-item check.

-i: specifies the check item, in the format of **-i Check***Specific check item*.

```
[omm@Run861 ~]$ gs_check -i CheckCPU
Parsing the check items config file successfully
Distribute the context file to remote hosts successfully
Start to health check for the cluster. Total Items:1 Nodes:3

Checking...                          [==========================] 1/1
Start to analysis the check result
CheckCPU.....................................OK
The item run on 3 nodes.    success: 3

Analysis the check result successfully
Success. All check items run completed. Total:1     Success:1
For more information please refer to
/opt/huawei/install/om/script/gspylib/inspection/output/CheckReport_20231025342222512309.tar.gz
```

Step 2 Perform a local check.

-L:indicates that the check is executed locally.

```
[omm@Run861 ~]$ gs_check -i CheckCPU -L
2023-10-25 09:39:07 [NAM] CheckCPU
2023-10-25 09:39:07 [STD] Checks the CPU usage of the host. If the value of idle is greater than 30% or the value of iowait is smaller than 30%, this check is passed. Otherwise, this check fails.
2023-10-25 09:39:07 [RST] OK
2023-10-25 09:39:07 [RAW]
Linux 5.10.0-136.12.0.86.oe2203sp1.x86_64 (Run861)    10/25/23    _x86_64_ (4 CPU)

09:39:02          CPU       %user       %nice     %system     %iowait      %steal       %idle
```

```
09:39:03        all      2.30      0.00      4.86      0.51      0.00     92.33
09:39:04        all      1.53      0.00      5.85      0.00      0.00     92.62
09:39:05        all      2.32      0.00      5.67      0.26      0.00     91.75
09:39:06        all      2.02      0.00      5.79      0.50      0.00     91.69
09:39:07        all      2.28      0.00      5.08      0.00      0.00     92.64
Average:        all      2.09      0.00      5.45      0.25      0.00     92.21
```

Step 3 Perform a scenario-specific check.

-e supports several preset scenarios, including **inspect** (routine check), **upgrade** (pre-upgrade check), **binary_upgrade** (local pre-upgrade check), **health** (health check), and **install** (installation check). You can also implement personalized check scenarios as required.

```
[omm@Run861 ~]$ gs_check -e inspect
Parsing the check items config file successfully
The below items require root privileges to execute:[CheckBlockdev CheckIOrequestqueue
CheckIOConfigure CheckMTU CheckRXTX CheckMultiQueue CheckFirewall CheckSshdService
CheckSshdConfig CheckCrondService CheckMaxProcMemory CheckBootItems CheckFilehandle
CheckNICModel CheckDropCache]
Please enter root privileges user[root]:root
Please enter password for user[root]:
Check root password connection successfully
Distribute the context file to remote hosts successfully
Start to health check for the cluster. Total Items:57 Nodes:3

Checking...                      [=========================] 57/57
Start to analysis the check result
```

2.2.4 Checking the OS

You can use the gs_checkos tool to check the OS, control parameters, and disk configurations, and configure control parameters, I/O parameters, network parameters, and THP services.

Before checking the OS, ensure that the hardware and network are working properly, and that the trust relationship of user **root** has been established between the hosts.

Only user **root** is authorized to run the **gs_checkos** command.

Step 1 Check the OS.

```
[root@Run861 ~]# gs_checkos -i A -X /opt/software/openGauss/cluster_config.xml --detail -o
/var/log/checkos
Performing operation system check/set. Output the result to the file /var/log/checkos.
BondMode Null
Operation system check/set is completed.
Total numbers:13. Abnormal numbers:0. Warning numbers:2.
```

Step 2 View the output.

```
[root@Run861 log]#cd /var/log
[root@Run861 log]# more checkos
Checking items:
```

```
      A1. [ OS version status ]                                   : Normal
          [Run861]
          openEuler_22.03_64bit
      A2. [ Kernel version status ]                               : Normal
          The names about all kernel versions are same. The value is "5.10.0-
136.12.0.86.oe2203sp1.x86_64".
      A3. [ Unicode status ]                                      : Normal
          The values of all unicode are same. The value is "LANG=en_US.UTF-8".
      A4. [ Time zone status ]                                    : Normal
          The informations about all timezones are same. The value is "+0800".
      A5. [ Swap memory status ]                                  : Normal
          The value about swap memory is correct.
      A6. [ System control parameters status ]                    : Normal
          All values about system control   parameters are correct.
      A7. [ File system configuration status ]                    : Normal
          Both soft nofile and hard nofile are correct.
      A8. [ Disk configuration status ]                           : Normal
          The value about XFS mount parameters is correct.
      A9. [ Pre-read block size status ]                          : Normal
          The value about Logical block size is correct.
      A11.[ Network card configuration status ]                   : Warning
          [Run861]
BondMode Null
          Warning reason: Failed to obtain the network card speed value. Maybe the network card
"eth0" is not working.
      A12.[ Time consistency status ]                             : Warning
          [Run861]
          The NTPD not detected on machine and local time is "2023-09-25 10:12:52".
      A13.[ Firewall service status ]                             : Normal
          The firewall service is stopped.
      A14.[ THP service status ]                                  : Normal
          The THP service is stopped.
```

2.2.5 Database Parameters

Change the maximum number of database connections.

Step 1 Check the value of **max_connections**.

```
[omm@Run861 ~]$ gs_guc check -D /opt/huawei/install/data/dn -c "max_connections"
The gs_guc run with the following arguments: [gs_guc -D /opt/huawei/install/data/dn -c
max_connections check ].
expected guc information: Run861: max_connections=NULL:
[/opt/huawei/install/data/dn/postgresql.conf]
gs_guc check: Run861: max_connections=5000: [/opt/huawei/install/data/dn/postgresql.conf]

Total GUC values: 1. Failed GUC values: 0.
```

Step 2 Change the maximum number of connections allowed by the database to **800**.

reload: modifies parameters in the configuration file and sends semaphores to the database process for reloading the configuration file.

-N: indicates the name of the host to be set. **ALL** indicates all the hosts in openGauss.

-I *INSTANCE-NAME*: indicates the name of the instance to be set. **ALL** indicates all instances on the host.

-c *parameter=value*: specifies the name and value of the openGauss configuration parameter to be set.

```
[omm@Run861 dn]$ gs_guc reload -N all -I all    -c "max_connections=800"
The gs_guc run with the following arguments: [gs_guc -N all -I all -c max_connections=800 reload ].
Begin to perform the total nodes: 3.
Popen count is 3, Popen success count is 3, Popen failure count is 0.
Begin to perform gs_guc for datanodes.
Command count is 3, Command success count is 3, Command failure count is 0.

Total instances: 3. Failed instances: 0.
ALL: Success to perform gs_guc!
```

Step 3 Check the maximum number of database connections.

```
openGauss=# show max_connections;
 max_connections
-----------------
 5000
```

Step 4 Restart the database and check the maximum number of database connections again.

```
[omm@Run861 dn]$ gs_om -t restart
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
Starting cluster.
=========================================================================
Successfully started primary instance. Wait for standby instance.
=========================================================================
Successfully started cluster.
=========================================================================
cluster_state         : Normal
redistributing        : No
node_count            : 3
Datanode State
      primary               : 1
      standby               : 2
      secondary             : 0
      cascade_standby       : 0
      building              : 0
      abnormal              : 0
      down                  : 0
Successfully started cluster.
[omm@Run861 dn]$ gsql -d postgres -p 15400 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:37:13 commit 0 last mr )
NOTICE : The password has been expired, please change the password.
Non-SSL connection (SSL connection is recommended when requiring high-security)
```

```
Type "help" for help.
openGauss=# show max_connections;
 max_connections
-----------------
 800
(1 row)
```

2.2.6 Lock Waits

Step 1 Create a table and insert test data into it.

```
openGauss=# CREATE TABLE graderecord
                (
                number INTEGER,
                name CHAR(20),
                class CHAR(20),
                grade INTEGER
                );
openGauss => insert into graderecord values('210101','Alan','21.01',92);
insert into graderecord values('210102','Ben','21.01',62);
insert into graderecord values('210103','Brain','21.01',26);
insert into graderecord values('210204','Carl','21.02',77);
insert into graderecord values('210205','David','21.02',47);
insert into graderecord values('210206','Eric','21.02',97);
insert into graderecord values('210307','Frank','21.03',90);
insert into graderecord values('210308','Gavin','21.03',100);
insert into graderecord values('210309','Henry','21.03',67);
insert into graderecord values('210410','Jack','21.04',75);
insert into graderecord values('210311','Jerry','21.04',60);
```

Step 2 In session 1, start a transaction and lock the **graderecord** table.

```
openGauss=#START TRANSACTION;
START TRANSACTION
openGauss=# LOCK TABLE graderecord IN ACCESS EXCLUSIVE MODE;
LOCK TABLE
```

Step 3 In session 2, modify a record in the **graderecord** table.

```
openGauss=# update graderecord set name='James' where number='210102';
```

Step 4 In session 3, query lock waits.

granted: indicates whether the transaction holds the lock (**TRUE**) or is waiting for the lock to be granted (**FALSE**).

locktag: indicates information of a lock that a session waits for.

query_start: indicates the query start time of a session.

query: indicates content to be queried.

pid: indicates the session ID.

```
openGauss=# select usename,granted,locktag,query_start,query,l.pid from pg_locks l,pg_stat_activity a
where l.pid=a.pid and locktag in(select locktag from pg_locks where granted='f');
 usename | granted |         locktag         |          query_start          |
query                                    |          pid
---------+---------+-------------------------+-------------------------------+-----------------------------------------
----------------+-----------------
 test    | f       | 494e:695b:0:0:0:0 | 2023-09-24 17:33:03.215094+08 | update graderecord set
name='James' where number='210102'; | 139625771169472
 test    | t       | 494e:695b:0:0:0:0 | 2023-09-24 17:32:45.630966+08 | LOCK TABLE graderecord IN
ACCESS EXCLUSIVE MODE;                | 139625135662784
```

2.3 WDR

Step 1 View related parameters.

Before generating a WDR, ensure that the **enable_wdr _snapshot** parameter has been set to **on**. Specifically, log in to openGauss and check whether the parameter is set to **on**. If the value of **enable_wdr _snapshot** is **on**, the WDR snapshot function is enabled. If the value is **off**, the function is disabled.

```
[omm@ecs-5a5b ~]$ gs_guc set -N all -I all   -c " enable_wdr_snapshot=on"
[omm@ecs-5a5b ~]$ gs_om -t restart
openGauss=# select name ,setting from pg_settings where name like '%wdr%';
                name                | setting
------------------------------------------------+------------------+------
 enable_wdr_snapshot                | on
 wdr_snapshot_interval              | 60
 wdr_snapshot_query_timeout         | 100
 wdr_snapshot_retention_days        | 8
```

Step 2 View WDR snapshot information.

```
openGauss=# select * from snapshot.snapshot;
 snapshot_id |              start_ts                 |                end_ts
-------------+---------------------------------+-------------------------------
        2643 | 2023-10-21 10:30:42.338121+08 | 2023-10-21 10:30:48.598254+08
        2644 | 2023-10-21 11:30:43.202352+08 | 2023-10-21 11:30:49.733492+08
        2645 | 2023-10-21 12:30:43.374222+08 | 2023-10-21 12:30:49.639466+08
        2595 | 2023-10-19 10:30:14.150986+08 | 2023-10-19 10:30:20.493279+08
        2646 | 2023-10-21 13:30:44.184957+08 | 2023-10-21 13:30:50.447083+08
        2596 | 2023-10-19 11:30:15.03748+08   | 2023-10-19 11:30:21.370156+08
```

Step 3 Query the value of **pgxc_node_name**.

```
openGauss=# show pgxc_node_name;
  pgxc_node_name
------------------
 dn_6001_6002_6003
(1 row)
```

Step 4 Run the **\a \t \o** ***Server file path*** command to generate a formatted WDR.

\a: changes to the unaligned mode.

\t: changes the information and row count footer of the output column name.

\o: sends all the query results to a server file. *Server file path*: specifies the path for storing the generated WDR file. The user must have the read and write permissions on the path.

```
openGauss=# \a \t \o    /home/omm/wdrTest.html
Output format is unaligned.
Showing only tuples.
```

Step 5 Write data to the **wdrTest.html** file and select the time points for generating the WDR from the **snapshot.snapshot** view.

begin_snap_id: specifies the ID of a snapshot when a query starts, which is specified by **snapshot_id** in the **snapshot.snaoshot** table.

end_snap_id: specifies the ID of a snapshot when a query ends. By default, the value of **end_snap_id** is greater than that of **begin_snap_id table** (**snapshot_id** in the **snapshot.snaoshot** table).

report_type: specifies the type of the generated report. The value can be **summary**, **detail**, and **all**. **summary** indicates summary data, **detail** indicates data details, and **all** includes information of both **summary** and **detail** data.

report_scope: specifies the range of the generated report. The value can be **cluster** or **node**. **cluster** indicates database-level information, and **node** indicates node-level information.

node_name: When **report_scope** is set to **node**, set this parameter to the name of the corresponding node. When **report_scope** is set to **cluster**, this parameter can be omitted, left blank, or set to **NULL**.

```
select generate_wdr_report(2643 ,2645,'all','node','dn_6001_6002_6003');
```

Step 6 View the generated WDR.

```
[omm@Run861 ~]$ pwd
/home/omm
 [omm@Run861 ~]$ ls -l
-rw------- 1 omm    dbgrp     1547366 Oct 27 08:49 wdrTest.html
```

3 Advanced SQL Lab

3.1 Introduction

3.1.1 About This Lab

This lab describes how to use openGauss database constraints, multi-table joins, subqueries, transaction management, and function calls.

3.1.2 Objectives

By completing this lab, you will:

- Master how to use constraints in openGauss.
- Learn how to use multi-table connections in openGauss.
- Understand the usage of subqueries in openGauss.
- Have a good command of common where clauses, data grouping, and sorting in openGauss.
- Familiarize yourself with transaction management in openGauss.
- Acquire the skills to call general functions and system functions in openGauss.
- Be familiar with the stored procedure of openGauss.

3.2 Database Constraints

A constraint clause specifies the conditions that new rows or rows to be updated must satisfy for an INSERT or UPDATE operation to succeed. If there is any data behavior that violates the constraint, the behavior is terminated by the constraint.

Constraints can be specified when a table is created (by using the CREATE TABLE statement) or after a table is created (by using the ALTER TABLE statement).

Constraints can be column-level or table-level. Column-level constraints apply only to columns, and table-level constraints apply to the entire table.

3.2.1 Procedure

Step 1 Use the gsql client to connect to the openGauss database as user **omm**. The port number is **26000**. Replace it with the actual port number of the openGauss node..

```
[omm@Run861 ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:37:13 commit 0 last mr   )
```

```
NOTICE : The password has been expired, please change the password.
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.
openGauss=# SELECT current_user;
 current_user
--------------
 omm
(1 row)
```

Step 2 Use the NOT NULL constraint to ensure that a column is not empty.

```
openGauss=# CREATE TABLE t_nn(
id INT,
c1 VARCHAR(10)
);
CREATE TABLE
### ALTER TABLE: adds a NOT NULL constraint to the c1 column by using the ALTER TABLE statement.
openGauss=# ALTER TABLE t_nn ALTER COLUMN c1 SET NOT NULL;
```

Step 3 Insert data to verify the NOT NULL constraint.

In databases, null value (**NULL**) is a special value that indicates a missing value or an inapplicable value. NULL is not the same as the number **0** or an empty string (''').

```
openGauss=# INSERT INTO t_nn(id, c1) values (1, 'apple');
INSERT 0 1
openGauss=# INSERT INTO t_nn(id, c1) values (1, null);
ERROR:  null value in column "c1" violates not-null constraint
DETAIL:  Failing row contains (1, null).
```

Step 4 Use the UNIQUE constraint to ensure that the values in a column are unique.

```
openGauss=# CREATE TABLE t_unique(
 id INT UNIQUE,
 c1 INT,
 c2 INT
);
NOTICE:  CREATE TABLE / UNIQUE will create implicit index "t_unique_id_key" for table "t_unique"
CREATE TABLE
### Add UNIQUE constraints to columns c1 and c2.
openGauss=#  ALTER TABLE t_unique ADD CONSTRAINT uk_t_unique UNIQUE (c1, c2);
openGauss=# \d+  t_unique;
                        Table "public.t_unique"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 id     | integer |           | plain   |              |
 c1     | integer |           | plain   |              |
 c2     | integer |           | plain   |              |
Indexes:
    "t_unique_id_key" UNIQUE CONSTRAINT, btree (id) TABLESPACE pg_default
    "uk_t_unique" UNIQUE, btree (c1, c2) TABLESPACE pg_default
```

```
Has OIDs: no
Options: orientation=row, compression=no
```

Step 5 Insert data to verify the UNIQUE constraint.

```
INSERT INTO t_unique(id, c1, c2) VALUES (1, 2, 3);
INSERT INTO t_unique(id, c1, c2) VALUES (1, 2, 3);
ERROR:  duplicate key value violates unique constraint "t_unique_id_key"
DETAIL:  Key (id)=(1) already exists.
ERROR: syntax error at or near "(1"
LINE 1: INSERT INTO t_unique(id, c1, c2) VALUES (1, 2, 3) ;
```

Step 6 Use the PRIMARY KEY constraint.

PRIMARY KEY uniquely identifies each record in a data table. A table consists of not null and unique columns.

```
CREATE TABLE t_primary1(
id INT NOT NULL PRIMARY KEY, -- A primary key consists of one field.
c1 INT
);

CREATE TABLE t_primary2(
id INT NOT NULL,
c1 INT NOT NULL,
CONSTRAINT pk2 PRIMARY KEY(id, c1) -- A primary key consists of multiple fields.
);
##### Run the ALTER TABLE command to add a PRIMARY KEY constraint.
CREATE TABLE t_primary3(
id INT NOT NULL,
c1 INT
);
ALTER TABLE t_primary3 ADD CONSTRAINT pk3 PRIMARY KEY (id);
```

Step 7 Insert data to verify the PRIMARY KEY constraint.

```
openGauss=# INSERT INTO t_primary3(id, c1) values (1, 100);
INSERT INTO t_primary3(id, c1) values (1, 200);
INSERT 0 1
openGauss=# INSERT INTO t_primary3(id, c1) values (1, 100);
ERROR:  duplicate key value violates unique constraint "pk3"
DETAIL:  Key (id)=(1) already exists.
```

Step 8 Use the FOREIGN KEY constraint.

The FOREIGN KEY constraint establishes a referencial relationship between two tables. Generally, a foreign key is applied to a column in one table which references the primary key of a column in another table.

The referenced table of a FOREIGN KEY constraint is called the parent table (table **dept** in the following example), and the table where the FOREIGN KEY resides is called the child table (table **emp** in the following example).

```
-- Create a department table.
CREATE TABLE dept (
department_id INTEGER NOT NULL PRIMARY KEY ,
department_name CHARACTER VARYING(30) NOT NULL
) ;

## Create an employee table.
CREATE TABLE emp (
employee_id INTEGER NOT NULL PRIMARY KEY ,
first_name CHARACTER VARYING(20) ,
last_name CHARACTER VARYING(25) NOT NULL ,
salary NUMERIC(8,2) ,
manager_id INTEGER ,
department_id INTEGER
) ;
```

Step 9 Establish a referencial relationship between the **department_id** column in table **emp** and the **department_id** column in table **dept**.

```
openGauss=# ALTER TABLE emp ADD CONSTRAINT fk_emp_dept FOREIGN KEY (department_id)
REFERENCES
dept(department_id);
ALTER TABLE
openGauss=# \d+ emp
                                        Table "public.emp"
     Column      |            Type             | Modifiers | Storage  | Stats target | Description
-----------------+-----------------------------+-----------+----------+--------------+-------------
 employee_id     | integer                     | not null  | plain    |              |
 first_name      | character varying(20)       |           | extended |              |
 last_name       | character varying(25)       | not null  | extended |              |
 salary          | numeric(8,2)                |           | main     |              |
 manager_id      | integer                     |           | plain    |              |
 department_id | integer                       |           | plain    |              |
Indexes:
    "emp_pkey" PRIMARY KEY, btree (employee_id) TABLESPACE pg_default
Foreign-key constraints:
    "fk_emp_dept" FOREIGN KEY (department_id) REFERENCES dept(department_id)
Has OIDs: no
Options: orientation=row, compression=no
```

Step 10 Insert data to verify the FOREIGN KEY constraint.

FOREIGN KEY constraints maintain referential integrity of data. Therefore, employees do not belong to a department that does not exist.

```
openGauss=# insert into dept values(1,'IT');
INSERT 0 1
####### Insert data of two employees. The department of the second employee does not exist
(department_id = 2), violating the FOREIGN KEY constraint. The insertion fails.
openGauss=# INSERT INTO emp VALUES (101, 'Steven', 'King', 30000, 1, 1);
INSERT 0 1
openGauss=# INSERT INTO emp VALUES (101, 'Adam', 'Fripp', 50000, 1, 2);
ERROR:  duplicate key value violates unique constraint "emp_pkey"
DETAIL:  Key (employee_id)=(101) already exists.
```

Step 11 Use the CHECK constraint.

The CHECK constraint specifies an expression producing a Boolean result where the INSERT or UPDATE operation on new rows or rows to be updated can succeed only when the expression result is **TRUE** or **UNKNOWN**; otherwise, an error is thrown and the database is not altered.

```
CREATE TABLE t_check(
 id INT PRIMARY KEY,
 c1 VARCHAR(10),
 c2 INT,
 c3 INT
);
##### Add a CHECK constraint. The value in column c2 must be greater than that in column c3.
ALTER TABLE t_check ADD CONSTRAINT check_c2c3 CHECK ( c2 > c3);

INSERT INTO t_check(id, c1, c2, c3) VALUES (1,   'START', 20, 19);
INSERT INTO t_check(id, c1, c2, c3) VALUES (2,   'START', 20, 19);
INSERT INTO t_check(id, c1, c2, c3) VALUES (2,   'PROC', 17, 18);
###### In the third INSERT INTO statement, the value 17 of column c2 is less than the value 18 of column c3, which violates the CHECK constraint. The insertion fails.
ERROR:   new row for relation "t_check" violates check constraint "check_c2c3"
DETAIL:   N/A
```

3.3 Table Joins

3.3.1 Preparations

```
-- Create a department table.
 CREATE TABLE departments (
      department_id integer NOT NULL, /*Department ID, the primary key.*/
      department_name varchar(30) NOT NULL, /*Department name.*/
      manager_id integer, /*Department manager ID.*/
      location_id integer /*Department location ID.*/
);
-- Create an employee table.
CREATE TABLE employees (
      employee_id integer NOT NULL, /*Employee ID, the primary key*/
      first_name VARCHAR(20), /*First name*/
      last_name VARCHAR(25) NOT NULL, /*Last name*/
      hire_date timestamp(0) without time zone NOT NULL, /*Hiring date*/
      salary numeric(8,2), /*Salary*/
      department_id integer, /*Department ID*/
      CONSTRAINT emp_salary_min CHECK (salary > (0))
);
-- Insert data into the departments table.
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (10,'Administration',200,1700);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (20,'Marketing',201,1800);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
```

```
 VALUES (30,'Purchasing',114,1700);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (40,'Human Resources',203,2400);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (50,'Shipping',121,1500);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (60,'IT',103,1400);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (70,'Public Relations',204,2700);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (80,'Sales',145,2500);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (90,'Executive',100,1700);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (100,'Finance',108,1700);
-- Insert data into the employees table.
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (101,'Neena','Kochhar','2005-09-21 00:00:00',17000.00,90);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (111,'Ismael','Sciarra','2005-09-30 00:00:00',7700.00,100);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (113,'Luis','Popp','2007-12-07 00:00:00',6900.00,100);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (114,'Den','Raphaely','2002-12-07 00:00:00',11000.00,30);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (105,'David','Austin','2005-06-25 00:00:00',4800.00,60);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (106,'Valli','Pataballa','2006-02-05 00:00:00',4800.00,60);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (117,'Sigal','Tobias','2005-07-24 00:00:00',2800.00,30);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (185,'Alexis','Bull','2005-02-20 00:00:00',4100.00,50);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (186,'Julia','Dellinger','2006-06-24 00:00:00',3400.00,50);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (145,'John','Russell','2004-10-01 00:00:00',14000.00,80);

-- Query data in the departments and employees tables.
openGauss=# select * from departments;
 department_id | department_name   | manager_id | location_id
---------------+-----------------+------------+-------------
            10 | Administration    |        200 |        1700
            20 | Marketing         |        201 |        1800
            30 | Purchasing        |        114 |        1700
            40 | Human Resources   |        203 |        2400
            50 | Shipping          |        121 |        1500
            60 | IT                |        103 |        1400
            70 | Public Relations  |        204 |        2700
            80 | Sales             |        145 |        2500
            90 | Executive         |        100 |        1700
           100 | Finance           |        108 |        1700
openGauss=# select * from employees;
 employee_id | first_name | last_name       |        hire_date         |   salary       | department_id
-------------+------------+-----------------+--------------------------+------------------+-
-------------
```

```
 101 | Neena     | Kochhar     | 2005-09-21 00:00:00 | 17000.00 |  90
 111 | Ismael    | Sciarra     | 2005-09-30 00:00:00 |  7700.00 | 100
 113 | Luis      | Popp        | 2007-12-07 00:00:00 |  6900.00 | 100
 114 | Den       | Raphaely    | 2002-12-07 00:00:00 | 11000.00 |  30
 105 | David     | Austin      | 2005-06-25 00:00:00 |  4800.00 |  60
 106 | Valli     | Pataballa   | 2006-02-05 00:00:00 |  4800.00 |  60
 117 | Sigal     | Tobias      | 2005-07-24 00:00:00 |  2800.00 |  30
 185 | Alexis    | Bull        | 2005-02-20 00:00:00 |  4100.00 |  50
 186 | Julia     | Dellinger   | 2006-06-24 00:00:00 |  3400.00 |  50
 145 | John      | Russell     | 2004-10-01 00:00:00 | 14000.00 |  80
```

3.3.2 Procedure

Step 1 Query the names, salaries, department IDs, and department names of employees.

```
openGauss=# SELECT
  e.first_name,
  e.last_name,
  e.salary,
  d.department_id,
  d.department_name
FROM employees e join departments d
On e.department_id = d.department_id;

first_name | last_name   |   salary   | department_id   | department_name
-----------------+----------------+--------------+----------------------+-----------------
  Sigal         |Tobias       |   2800.00 |                30 | Purchasing
  Den          | Raphaely    | 11000.00 |                30 | Purchasing
  Julia         | Dellinger    |   3400.00 |                50 | Shipping
  Alexis        | Bull         |   4100.00 |                50 | Shipping
  Valli         | Pataballa    |   4800.00 |                60 | IT
  David         | Austin       |   4800.00 |                60 | IT
  John          | Russell      | 14000.00 |                80 | Sales
  Neena         | Kochhar     | 17000.00 |                90 | Executive
  Luis          | Popp         |   6900.00 |               100 | Finance
  Ismael        | Sciarra      |   7700.00 |               100 | Finance
(10 rows)
```

Step 2 Query the names, salaries, department names, and department IDs of employees in all departments.

Some departments may not have employees yet. Therefore, information about these departments was not displayed in the previous query using inner join. If you want information about these departments to be returned in a join query, use left outer join.

```
openGauss =# SELECT
          e.first_name,
          e.last_name,
          e.salary,
          d.department_name,
          d.department_id
FROM departments d LEFT JOIN employees e
ON e.department_id = d.department_id;
```

```
first_name | last_name |   salary    | department_name   | department_id
---------------+-----------------+-------------+---------------------------+--------------------
               |                 |             | Administration      |        10
               |                 |             | Marketing           |        20
 Sigal         | Tobias          |  2800.00    | Purchasing          |        30
 Den           | Raphaely        | 11000.00    | Purchasing          |        30
               |                 |             | Human Resources     |        40
 Julia         | Dellinger       |  3400.00    | Shipping            |        50
 Alexis        | Bull            |  4100.00    | Shipping            |        50
 Valli         | Pataballa       |  4800.00    | IT                  |        60
 David         | Austin          |  4800.00    | IT                  |        60
               |                 |             | Public Relations    |        70
 John          | Russell         | 14000.00    | Sales               |        80
 Neena         | Kochhar         | 17000.00    | Executive           |        90
 Luis          | Popp            |  6900.00    | Finance             |       100
 Ismael        | Sciarra         |  7700.00    | Finance             |       100
(14 rows)
```

Step 3 Query the names and IDs of departments that do not have employees yet.

NULL indicates unknown data. **NULL** is not equivalent to **0**.

```
SELECT      d.department_id,
            d.department_name
FROM departments d LEFT JOIN employees e
ON e.department_id = d.department_id where employee_id is null;

department_id   | department_name
----------------------+------------------
                10 | Administration
                20 | Marketing
                40 | Human Resource
     70 | Public Relations
```

3.4 UNION Clause

3.4.1 Preparations

```
-- Create the t_demo table.
openGauss =# CREATE TABLE t_demo(
  id int NOT NULL,
  name varchar(255) DEFAULT NULL,
  age int DEFAULT NULL,
  num int DEFAULT NULL,
  PRIMARY KEY (id)
);

-- Create the t_demo_copy table.
CREATE TABLE t_demo_copy(
  id int NOT NULL,
```

```
  name varchar(255) DEFAULT NULL,
  age int DEFAULT NULL,
  num int DEFAULT NULL,
  PRIMARY KEY (id)
);

-- Inserts data into the t_demo table.
INSERT INTO t_demo VALUES('1','Zhang San', '21', '69');
INSERT INTO t_demo VALUES('2','Li Si', '22', '98');
INSERT INTO t_demo VALUES('3','Wang Wu', '20', '54');
INSERT INTO t_demo VALUES('4','Zhao Tian', '22', '80');

-- Inserts data into the t_demo_copy table.
INSERT INTO t_demo_copy VALUES ('1','Zhang San', '21', '69');
INSERT INTO t_demo_copy VALUES('2', 'Zhu Bajie', '22', '98');
INSERT INTO t_demo_copy VALUES ('3','Wang Wu', '20', '54');
INSERT INTO t_demo_copy VALUES('4','Zhao Tian', '22', '80');
INSERT INTO t_demo_copy VALUES('5','Sun Wukong', '22', '100');
INSERT INTO t_demo_copy VALUES ('6','Li Si', '24', '99');

openGauss=# select * from t_demo;
id | name | age | num
----+------+-----+-----
  1 | Zhang San |  21 |  69
  2 | Li Si |  22 |  98
  3 | Wang Wu |  20 |  54
  4 | Zhao Tian |  22 |  80
(4 rows)

openGauss =# select * from t_demo_copy;
id |   name   | age | num
----+--------+-----+-----
  1 | Zhang San    |  21 |  69
  2 | Zhu Bajie |  22 |  98
  3 | Wang Wu    |  20 |  54
  4 | Zhao Tian    |  22 |  80
  5 | Sun Wukong |  22 | 100
  6 | Li Si    |  24 |  99
(6 rows)
```

3.4.2 Procedure

Step 1 Combine the **t_demo** and **t_demo_copy** tables, and delete duplicate data.

```
openGauss=# SELECT * FROM t_demo
UNION
SELECT * FROM t_demo_copy;
 id |   name   | age | num
----+--------+-----+-----
  2 | Li Si    |  22 |  98
  5 | Sun Wukong |  22 | 100
  6 | Li Si    |  24 |  99
  3 | Wang Wu    |  20 |  54
  4 | Zhao Tian    |  22 |  80
  2 | Zhu Bajie |  22 |  98
```

```
   1 | Zhang San     |   21 |   69
(7 rows)
```

Step 2 Combine the **t_demo** and **t_demo_copy** tables, and retain duplicate data.

```
openGauss=# SELECT * FROM t_demo
UNION ALL
SELECT * FROM t_demo_copy;
 id |    name    | age | num
----+--------+-----+-----
   1 | Zhang San     |   21 |   69
   2 | Li Si     |   22 |   98
   3 | Wang Wu     |   20 |   54
   4 | Zhao Tian     |   22 |   80
   1 | Zhang San     |   21 |   69
   2 | Zhu Bajie |   22 |   98
   3 | Wang Wu     |   20 |   54
   4 | Zhao Tian     |   22 |   80
   5 | Sun Wukong |   22 | 100
   6 | Li Si     |   24 |   99
(10 rows)
```

3.5 WHERE Clause

3.5.1 Preparations

```
openGauss=# select * from employees;
 employee_id | first_name | last_name   |         hire_date          |   salary     | department_id
-------------------+----------------+---------------+---------------------------------- +-----------------+-------
--------
            101 | Neena      | Kochhar     | 2005-09-21 00:00:00      | 17000.00     |     90
            111 | Ismael     | Sciarra     | 2005-09-30 00:00:00      |   7700.00    |   100
            113 | Luis       | Popp        | 2007-12-07 00:00:00      |   6900.00    |   100
            114 | Den        | Raphaely    | 2002-12-07 00:00:00      | 11000.00     |     30
            105 | David        | Austin      | 2005-06-25 00:00:00    |   4800.00    |     60
            106 | Valli        | Pataballa   | 2006-02-05 00:00:00    |   4800.00    |     60
            117 | Sigal        | Tobias      | 2005-07-24 00:00:00    |   2800.00    |     30
            185 | Alexis       | Bull        | 2005-02-20 00:00:00    |   4100.00    |     50
            186 | Julia        | Dellinger   | 2006-06-24 00:00:00    |   3400.00    |     50
            145 | John         | Russell     | 2004-10-01 00:00:00    | 14000.00     |     80
(10 rows)
```

3.5.2 Procedure

Step 1 Query the information of employees who were hired between June 24, 2005 and December 31, 2007, and whose salary is less than 5000.

```
openGauss =# select * from employees where hire_date between '2005-06-24' and '2007-12-31' and
salary < 5000;
 employee_id | first_name   | last_name   |         hire_date          | salary     | department_id
```

```
-------------------+-----------------+----------------+------------------------------+-------------+---------
------
          105 | David       | Austin      | 2005-06-25 00:00:00    | 4800.00  |     60
          106 | Valli       | Pataballa   | 2006-02-05 00:00:00    | 4800.00  |     60
          117 | Sigal       | Tobias      | 2005-07-24 00:00:00    | 2800.00  |     30
          186 | Julia       | Dellinger   | 2006-06-24 00:00:00    | 3400.00  |     50
(4 rows)
```

Step 2 Query the information of employees whose department ID is either in (30, 60) or whose salary is greater than 4000, and whose name does not start with "D".

```
openGauss=# select   *   from employees where (department_id in(30,60) or salary > 4000) and
first_name not like 'D%' ;

 employee_id | first_name | last_name |         hire_date          |   salary   | department_id
-------------+------------+-----------+----------------------------+------------+---------------
          101 | Neena      | Kochhar   | 2005-09-21 00:00:00 | 17000.00 |            90
          111 | Ismael     | Sciarra   | 2005-09-30 00:00:00 |  7700.00 |           100
          113 | Luis       | Popp      | 2007-12-07 00:00:00 |  6900.00 |           100
          106 | Valli      | Pataballa | 2006-02-05 00:00:00 |  4800.00 |            60
          117 | Sigal      | Tobias    | 2005-07-24 00:00:00 |  2800.00 |            30
          185 | Alexis     | Bull      | 2005-02-20 00:00:00 |  4100.00 |            50
          145 | John       | Russell   | 2004-10-01 00:00:00 | 14000.00 |            80
```

3.6 Grouping and Sorting

The aggregate function performs calculation on multiple values and returns a single value.

GROUP BY is one of the most frequently-used database query clauses. It collects data across multiple records and groups the results by one or more columns.

3.6.1 Preparations

```
openGauss=# select * from employees;
 employee_id | first_name | last_name  |         hire_date          |   salary    | department_id
-------------+------------+------------+----------------------------+-------------+---------------
          101 | Neena      | Kochhar    | 2005-09-21 00:00:00    | 17000.00    |     90
          111 | Ismael     | Sciarra    | 2005-09-30 00:00:00    |  7700.00    |    100
          113 | Luis       | Popp       | 2007-12-07 00:00:00    |  6900.00    |    100
          114 | Den        | Raphaely   | 2002-12-07 00:00:00    | 11000.00    |     30
          105 | David      | Austin     | 2005-06-25 00:00:00    |  4800.00    |     60
          106 | Valli      | Pataballa  | 2006-02-05 00:00:00    |  4800.00    |     60
          117 | Sigal      | Tobias     | 2005-07-24 00:00:00    |  2800.00    |     30
          185 | Alexis     | Bull       | 2005-02-20 00:00:00    |  4100.00    |     50
          186 | Julia      | Dellinger  | 2006-06-24 00:00:00    |  3400.00    |     50
          145 | John       | Russell    | 2004-10-01 00:00:00    | 14000.00    |     80
(10 rows)
```

3.6.2 Procedure

Step 1 Query the average salary, maximum salary, minimum salary, and total salary of all employees.

```
openGauss=# SELECT
 AVG(salary),
 MAX(salary),
 MIN(salary),
 SUM(salary)
 FROM employees;
   Avg   | max   | min   |    sum
---------+---------+---------+---------
 7650.00| 17000.00 | 2800.00 | 76500.00
```

Step 2 Query the average salary, total number of employees, maximum salary, minimum salary, and total salary of all employees in each department. Group them by department, and sort them by the highest salary in each department in descending order.

```
openGauss=# SELECT department_id,
 AVG(salary),
 COUNT(*),
 MAX(salary),
 MIN(salary),
 SUM(salary)
 FROM employees
GROUP BY department_id
order by max desc;

 department_id | avg                  | count   |    max      |    min      |    sum
---------------------+----------------------+---------+-----------------+----------------+----------
                90 | 17000.00         |       1 | 17000.00    | 17000.00 | 17000.00
                80 | 14000.00         |       1 | 14000.00    | 14000.00 | 14000.00
                30 |   6900.00        |       2 | 11000.00   |   2800.00 | 13800.00
                100|   7300.00       |       2 |   7700.00    |   6900.00 | 14600.00
                60 |   4800.00        |       2 |   4800.00   |   4800.00 |   9600.00
                50 |   3750.00        |       2 |   4100.00   |   3400.00 |   7500.00
```

Step 3 Query the average salary, total number of employees, maximum salary, minimum salary, and total salary of all employees in each department. Group them by department. Return only the departments whose average salary is less than 7000, and sort them by the highest salary in each department in descending order.

```
SELECT department_id,
 AVG(salary),
 COUNT(*),
 MAX(salary),
 MIN(salary),
 SUM(salary)
 FROM employees
GROUP BY department_id
```

```
Having avg < 7000
order by max desc;

department_id   | avg                 | count   |    max      |    min   |    sum
----------------------+---------------------+-----------+---------------+------------+----------
                30 | 6900.00          |       2 | 11000.00   | 2800.00 | 13800.00
                60 | 4800.00          |       2 |   4800.00   | 4800.00 | 9600.00
                50 | 3750.00          |       2 |   4100.00   | 3400.00 | 7500.00
```

3.7 Subquery

A subquery is a query statement nested in other statements such as SELECT, INSERT, UPDATE, and DELETE.

Similar to a multi-table join query, a subquery returns or filters data from multiple associated tables.

3.7.1 Preparations

```
openGauss=# select * from departments;
 department_id | department_name   | manager_id | location_id
----------------------+----------------------------+------------+-------------
                10 | Administration      |           200 |    1700
                20 | Marketing           |           201 |    1800
                30 | Purchasing          |           114 |    1700
                40 | Human Resources   |           203 |     2400
                50 | Shipping              |           121 |     1500
                60 | IT                      |           103 |     1400
                70 | Public Relations     |           204 |     2700
                80 | Sales                  |           145 |     2500
                90 | Executive            |           100 |      1700
               100 | Finance               |           108 |      1700
(10 rows)

openGauss=# select * from employees;
employee_id | first_name | last_name   |        hire_date          |   salary    | department_id
-------------------+---------------+---------------+-------------------------------- +-----------------+-------
--------
            101 | Neena      | Kochhar     | 2005-09-21 00:00:00      | 17000.00    |    90
            111 | Ismael     | Sciarra     | 2005-09-30 00:00:00      |   7700.00    |   100
            113 | Luis         | Popp        | 2007-12-07 00:00:00      |   6900.00    |   100
            114 | Den         | Raphaely    | 2002-12-07 00:00:00      | 11000.00    |    30
            105 | David        | Austin      | 2005-06-25 00:00:00    |   4800.00    |    60
            106 | Valli         | Pataballa   | 2006-02-05 00:00:00    |   4800.00    |    60
            117 | Sigal         | Tobias      | 2005-07-24 00:00:00    |   2800.00    |    30
            185 | Alexis        | Bull        | 2005-02-20 00:00:00    |   4100.00    |    50
            186 | Julia         | Dellinger   | 2006-06-24 00:00:00    |   3400.00    |    50
            145 | John          | Russell     | 2004-10-01 00:00:00    | 14000.00    |    80
 (10 rows)
```

3.7.2 Derived Table

A subquery in the FROM clause is called a derived table.

Such a subquery is equivalent to creating a temporary table **table_alias**.

Step 1 Query the names and average salaries of the departments.

The subquery returns the department IDs and average salaries of the departments, which are then joined with the **departments** table.

```
openGauss=# SELECT d.department_name, ds.sum_salary
  FROM departments d
  JOIN (SELECT department_id, ROUND(AVG(salary), 2) AS sum_salary
             FROM employees
           group by department_id ) ds ON (d.department_id = ds.department_id);
department_name | sum_salary
------------------------+------------
 Purchasing         |    6900.00
 Shipping           |    3750.00
 IT                 |    4800.00
 Sales              |   14000.00
 Executive          |   17000.00
 Finance            |    7300.00
```

3.7.3 IN Operator

When the WHERE subquery returns multiple records, you can use the IN operator to filter the records.

The records returned by the WHERE subquery are values in the IN list.

Step 1 Query the IDs and names of the departments whose employees' salaries are greater than 5000.

```
openGauss=# SELECT d.department_id,
 d.department_name
 FROM departments d
WHERE d.department_id in (SELECT department_id FROM employees WHERE
salary > 5000);
 department_id | department_name
---------------+-----------------
            30 | Purchasing
            80 | Sales
            90 | Executive
           100 | Finance
```

3.7.4 ALL Operator

The multiple records returned by the WHERE subquery are compared with all values in the ALL operator list and filtered.

Step 1 Query the names and salaries of the employees whose salaries are higher than those of all the employees whose department ID is 30.

```
openGauss=# SELECT first_name, last_name, salary
  FROM employees
 WHERE salary > ALL(SELECT salary
```

```
                              FROM employees
                             WHERE department_id =30);
first_name | last_name |    salary
---------------+----------------+----------
 Neena     | Kochhar    | 17000.00
 John      | Russell    | 14000.00
```

3.7.5 ANY Operator

You can compare the multiple records returned by the WHERE subquery with all values in the ANY operator list.

Step 1 Query the names and salaries of the employees whose salaries are higher than those of any employee whose department ID is 30.

```
SELECT first_name, last_name, salary
  FROM employees
 WHERE salary > ANY(SELECT salary
                              FROM employees
                             WHERE department_id =30);
 first_name | last_name |    salary
-----------------+-----------+----------
 Neena      | Kochhar    | 17000.00
 Ismael     | Sciarra    |   7700.00
 Luis       | Popp       |   6900.00
 Den        | Raphaely   | 11000.00
 David      | Austin     |   4800.00
 Valli      | Pataballa  |   4800.00
 Alexis     | Bull       |   4100.00
 Julia      | Dellinger  |   3400.00
 John       | Russell    | 14000.00
```

3.7.6 EXISTS Operator

The EXISTS operator is used to check whether the subquery result exists. If the subquery returns any result, the result of EXISTS is **True**. Otherwise, the result of EXISTS is **False**.

Step 1 Query the IDs and names of the departments whose employees' salaries are greater than 5000.

```
openGauss=# SELECT d.department_id,
 d.department_name
 FROM departments d
WHERE exists (SELECT * FROM employees WHERE department_id = d.department_id
and salary > 5000);
 department_id | department_name
---------------+-----------------
            30 | Purchasing
            80 | Sales
            90 | Executive
           100 | Finance
```

3.7.7 Correlated Subquery

A correlated subquery references columns in an outer query and therefore is correlated with the outer query.

Step 1 Query the names and salaries of employees whose salaries are greater than the average salary of their departments.

The subquery references **o.department_id** in the outer query.

```
openGauss=# SELECT first_name, last_name, salary
  FROM employees o
 WHERE o.salary >(SELECT avg(salary)
                    FROM employees i
                   WHERE i.department_id = o.department_id);
 first_name | last_name |   salary
------------+-----------+----------
 Ismael     | Sciarra   |  7700.00
 Den        | Raphaely  | 11000.00
 Alexis     | Bull      |  4100.00
```

Step 2 Query the IDs, names, and total salaries of each department.

The subquery references the **department_id** column in the **departments** table.

```
openGauss=# SELECT department_id,
           d.department_name,
 (SELECT SUM(salary)
 FROM employees e
 WHERE e.department_id = d.department_id) AS sum_salary
 FROM departments d;
 department_id | department_name   | sum_salary
---------------+-------------------+------------
            10 | Administration    |
            20 | Marketing         |
            30 | Purchasing        |   13800.00
            40 | Human Resources   |
            50 | Shipping          |    7500.00
            60 | IT                |    9600.00
            70 | Public Relations  |
            80 | Sales             |   14000.00
            90 | Executive         |   17000.00
           100 | Finance           |   14600.00
(10 rows)
```

3.7.8 Using Subquery in INSERT and UPDATE Statements

Step 1 Create the **employees_new** table.

```
openGauss=# CREATE TABLE employees_new (
    employee_id integer NOT NULL,
    first_name character varying(20),
    last_name character varying(25) NOT NULL,
    hire_date timestamp(0) without time zone NOT NULL,
```

```
    salary numeric(8,2),
    department_id integer,
    CONSTRAINT emp_salary_min CHECK ((salary > (0)::numeric))
);
CREATE TABLE
```

Step 2 Insert all information about employees whose salaries are greater than 5000 in the **employees** table into the **employees_new** table.

```
openGauss=# insert into employees_new
select *
  from employees e
 where e.employee_id in(select e.employee_id
                          from employees
                         where e.salary > 5000);
INSERT 0 5

openGauss=# select * from employees_new;
employee_id  | first_name | last_name      |         hire_date          |   salary   | department_id
 ------------------+--------------+--------------------+--------------------------------+-------------+--------
-------
           101 | Neena      | Kochhar         | 2005-09-21 00:00:00    | 17000.00 |   90
           111 | Ismael     | Sciarra         | 2005-09-30 00:00:00    |   7700.00 |   100
           113 | Luis       | Popp            | 2007-12-07 00:00:00    |   6900.00 |   100
           114 | Den        | Raphaely         | 2002-12-07 00:00:00    | 11000.00 |    30
           145 | John       | Russell         | 2004-10-01 00:00:00    | 14000.00 |   80
```

Step 3 Change the salary of an employee whose salary is greater than 10000 in the **employees_new** table to 50% of the original salary.

```
openGauss=# update employees_new
    set salary = salary * 0.5
 where employee_id in(select employee_id
                        from employees_new
                       where salary > 10000);

openGauss=# select * from employees_new;
 employee_id | first_name     | last_name |         hire_date          | salary   | department_id
------------------+------------------+-----------+--------------------+---------+---------------
           111 | Ismael       | Sciarra     | 2005-09-30 00:00:00 | 7700.00 |                100
           113 | Luis         | Popp        | 2007-12-07 00:00:00 | 6900.00 |                 100
           101 | Neena        | Kochhar     | 2005-09-21 00:00:00 | 8500.00 |                  90
           114 | Den          | Raphaely    | 2002-12-07 00:00:00 | 5500.00 |                  30
           145 | John         | Russell     | 2004-10-01 00:00:00 | 7000.00 |                 80
```

3.8 Functions

3.8.1 Numeric Functions

- abs(x) returns the absolute value of *X*.

```
openGauss=# select abs(-15.9);
 abs
------
 15.9
(1 row)
```

- ceil(exp) returns the smallest integer greater than or equal to the specified expression *n*.

```
openGauss=# select   ceil(15.3);
 ceil
------
   16
(1 row)
```

- floor (exp) calculates the largest integer equal to or less than the value of an expression.

```
openGauss=# select floor(15.3);
 floor
-------
    15
(1 row)
```

- power(base, expn) returns the result of multiplying a number by a power. *base* specifies the base number and *expn* specifies the exponent. The return type is NUMBER.

```
openGauss=# select power(5,3);
 power
-------
   125
(1 row)
```

- div(y, x) calculates the integer quotient of *y* divided by *x*. mod(y, x) calculates the remainder of *y* divided by *x*.

```
openGauss=# SELECT div(11,3), mod(11,3);
 div | mod
-----+-----
   3 |   2
(1 row)
```

- random()returns a random number.

```
openGauss=# SELECT random();
       random
------------------
 .781858928501606
(1 row)

openGauss=# SELECT random();
       random
```

```
------------------
 .545018189121038
(1 row)
```

3.8.2 Character Processing Functions

- concat(str[, ...]) or concat_ws(separator, str1, str2, ...) concatenates one or more character strings. There is no delimiter in the former function. Delimiters can be specified in the latter.

```
openGauss=# select concat('123','xyz','456');
   concat
-----------
 123xyz456
(1 row)

openGauss=# select concat_ws('-','123','xyz','456');
   concat_ws
-------------
 123-xyz-456
(1 row)
```

- replace(str,src,dst) inserts and replaces strings.

```
openGauss=# select replace('12345623','23','xyz');
   replace
------------
 1xyz456xyz
(1 row)
```

- instr(str1,str2[,pos[,n]])) searches for character string. Returns the position of the string to be searched for in the source string. Returns the position matching string 2 in string 1 for the *int2*th time from the position specified by *int1*. *int1* specifies the start position for matching, and *int2* specifies the number of matching times.

```
openGauss=# SELECT INSTR('123456abcd123456abcd', '45'), INSTR('123456abcd123456abcd', 'bcd', 1,
2 );
 instr | instr
-------+-------
     4 |    18
(1 row)
```

- lower(string) function converts a string to lowercase letters, and upper(string) function converts a string to uppercase letters.

```
openGauss=# SELECT lower('JAMES'), upper('James');
 lower | upper
-------+-------
 james | JAMES
(1 row)
```

- substr(str, start[, len]) truncates a substring. It truncates and returns the string of **len** bytes starting from **start** in **str**.

```
openGauss=# select substr('abcdefg',3,4);
 substr
--------
 cdef
(1 row)
```

3.8.3 Time/Date Functions

- age(timestamp, timestamp) calculates the interval between two timestamps.

```
openGauss=# SELECT age(timestamp '2023-09-30', timestamp '2023-09-05');
    age
---------
 25 days
(1 row)
```

- current_timestamp obtains the current system time and time zone, and current_date obtains the current date.

```
openGauss=# select current_timestamp,current_date;
            pg_systimestamp             |      date
------------------------------------------------+-----------------
 2023-09-14 08:41:24.920154+08     | 2023-09-14
(1 row)
```

3.8.4 Type Conversion Functions

- cast(expr as datatype) converts a value to a specified data type, which is equivalent to the conversion operator (::).

```
openGauss=# SELECT CAST ('201' AS INTEGER), '2023-09-15'::DATE;
 int4 |         timestamp
------+---------------------
   201 | 2023-09-15 00:00:00
(1 row)
```

- to_date(string, format) converts a string *string* into a date based on the *format*.

```
openGauss=# select    to_date('2023/09/15','YYYY/MM/DD');
 to_date
---------------------
 2023-09-15 00:00:00
```

- to_char (datetime/interval [, fmt]) converts the input into a character string.

```
openGauss=# SELECT to_char(current_timestamp, 'HH24:MI:SS');
 to_char
----------
```

```
 09:02:36
```

3.8.5 System Information Functions

Step 1 Query the database instance startup time, current database name (or catalog in standard SQL), and host name.

```
openGauss=# select pg_postmaster_start_time(),current_catalog,current_database(),get_hostname();
    pg_postmaster_start_time       | current_database | current_database | get_hostname
-----------------------------------+------------------+------------------+--------------
 2023-09-27 10:25:27.886419+08 | postgres             | postgres         | Run861
(1 row)
```

Step 2 Query the current schema and schemas in the search path.

```
openGauss=# select current_schema;
 current_schema
----------------
 public
(1 row)
```

Step 3 Query the username and user ID in the current environment.

```
openGauss=# select user,current_user,current_role,session_user,pg_current_userid();
 current_user      | current_user      | current_user      | session_user | pg_current_userid
-------------------+-------------------+-------------------+--------------+-------------------
-
 omm               | omm               | omm               | omm          |        10


```

3.9 Stored Procedures and Triggers

A stored procedure is a set of SQL statements that can implement specific functions in a database for external programs to call. It can be regarded as a function or subprogram of a database.

SQL statements are compiled once and called repeatedly. This reduces the number of repeated SQL statements and improves work efficiency. More SQL statements are sent during one interaction, reducing the number of interactions between the client and server.

For complex SQL statements that are run for multiple times, the stored procedure can cache SQL plans, reducing the time for generating plans.

3.9.1 Simple Stored Procedures

Step 1 Create table **t1**.

```
openGauss=# CREATE TABLE t1(
ID INT,
```

```
NAME VARCHAR2(20)
);
```

Step 2 Create stored procedure **p1**.

```
openGauss=# CREATE OR REPLACE PROCEDURE p1 IS
BEGIN
INSERT INTO t1 VALUES (1,'Li Ming');
COMMIT;
END;
/
```

Step 3 Call stored procedure **p1**.

```
openGauss=# call p1();
 p1
----
(1 row)
```

Step 4 Query data in the **t1** table.

```
openGauss=# select * from t1;
 id | name
----+------
  1 | Li Ming
```

Step 5 Delete the stored procedure.

```
openGauss=# DROP PROCEDURE p1;
DROP PROCEDURE
```

3.9.2 Stored Procedure with Parameters

Step 1 Create a stored procedure with parameters. The stored procedure is used to multiply two input parameters and output the calculation result.

```
openGauss=# CREATE OR REPLACE PROCEDURE p2 ( param1 IN INTEGER, param2 IN OUT
INTEGER )
AS
BEGIN
      param2:= param1 * param2;
      RAISE INFO 'param2 is: %', param2;
END;
/
CREATE PROCEDURE
```

Step 2 Call stored procedure **p2**.

```
openGauss=# call p2(6,7);
INFO:   param2 is: 42
 param2
--------
```

```
      42
(1 row)
```

3.9.3 Anonymous Blocks

An anonymous block is generally used for scripts that are not frequently executed or activities that are not repeatedly executed. An anonymous block is executed in a session and is not stored.

The following is an anonymous block, as opposed to a named block (that is, stored procedures and functions). The DO statement is used to execute an anonymous block.

Define a string variable *name*, assign a value to it and output a message. RAISE NOTICE is used to output a message.

```
openGauss=# DO $$
DECLARE
 name text;
BEGIN
 name := 'PL/SQL';
 RAISE NOTICE 'Hello %!', name;
END $$;
NOTICE:  Hello PL/SQL!
ANONYMOUS BLOCK EXECUTE
```

3.9.4 Nested Subblocks

PL/SQL supports code block nesting, that is, embedding a code block into the body of another code block.

A nested code block is called a subblock, and a code block including the subblock is called an outer block.

A subblock can logically split code. A variable with the same name as the outer block can be defined in a subblock, and it has a higher priority within the subblock.

```
openGauss=# DO $$
DECLARE
 name text;
BEGIN
 name := 'outer_block';
 RAISE NOTICE 'This is %', name;
 DECLARE
 name text := 'sub_block';
 BEGIN
 RAISE NOTICE 'This is %', name;
 RAISE NOTICE 'The name FROM the outer block is %',name;
 END;
 RAISE NOTICE 'This is %', name;
END $$;

NOTICE:  This is outer_block
NOTICE:  This is sub_block
NOTICE:  The name FROM the outer block is sub_block
NOTICE:  This is outer_block
```

```
ANONYMOUS BLOCK EXECUTE
```

3.9.5 Conditional Statements

Conditional statements are used to decide whether given conditions are met. Operations are executed based on the decisions made.

IF_THEN is the simplest form of IF statements. If the condition is true, statements are executed. If the condition is false, the statement is skipped and IF_THEN ends.

IF_THEN_ELSE has an ELSE branch and can be executed if the condition is false.

The IF_THEN_ELSE statement can be nested.

IF_THEN_ELSE has an ELSE branch and can be executed if the condition is false.

```
Use IF_THEN_ELSIF_ELSE to compare two numbers.
openGauss=# DO $$
DECLARE
  i integer := 4;
  j integer := 5;
BEGIN
  IF i > j THEN
    RAISE NOTICE 'i is greater than j';
  ELSIF i < j THEN
    RAISE NOTICE 'i is less than j';
  ELSE
    RAISE NOTICE 'i is equal to j';
  END IF;
END $$;
ENDNOTICE: i is less than j.
ANONYMOUS BLOCK EXECUTE
```

3.9.6 CASE Statements

3.9.6.1 Simple CASE Statements

Simple CASE statements can only be used for simple equivalent comparison.

It evaluates each expression following the WHEN keyword. If an expression evaluates to true, it executes the corresponding statement, and subsequent branches are not evaluated.

```
openGauss=#CREATE OR REPLACE PROCEDURE proc_case_branch(pi_result in integer, pi_return out
integer)
AS
    BEGIN
        CASE pi_result
            WHEN 1 THEN
                pi_return := 111;
            WHEN 2 THEN
                pi_return := 222;
            WHEN 3 THEN
                pi_return := 333;
            WHEN 6 THEN
                pi_return := 444;
```

```
            END CASE;
            raise info 'pi_return : %',pi_return ;
END;
/
test=> call proc_case_branch(3,0);
INFO:   pi_return : 333
 pi_return
-----------
          333
(1 row)
```

3.9.6.2 Searched CASE Statements

A searched CASE statement can implement more complex control logic.

It evaluates each expression following the WHEN keyword. If an expression evaluates to true, it executes the corresponding statement, and subsequent branches are not evaluated. If no matching value is found, it executes the ELSE statement.

```
openGauss=# DO $$
DECLARE
 i integer := 3;
BEGIN
 CASE
 WHEN i BETWEEN 1 AND 5 THEN
RAISE NOTICE 'The value falls within the range of 1 to 5.';
 WHEN i BETWEEN 6 AND 10 THEN
RAISE NOTICE 'The value falls within the range of 6 to 10.';
 ELSE
 RAISE NOTICE 'other value';
 END CASE;
END $$;

NOTICE: The value falls within the range of 1 to 5.
ANONYMOUS BLOCK EXECUTE
```

3.9.7 Loop Statements

3.9.7.1 LOOP Statement

The loop must be exploited together with EXIT; otherwise, an infinite loop occurs.

In the following stored procedure, the initial value of **i** is **0**. The loop does not end until the value of **i** reaches **8**.

```
openGauss=# DO $$
DECLARE
   i integer := 0;
BEGIN LOOP
      EXIT WHEN i = 8;
      i := i + 1;
      RAISE NOTICE 'Loop: %', i;
   END LOOP;
END $$;
```

```
NOTICE:  Loop: 1
NOTICE:  Loop: 2
NOTICE:  Loop: 3
NOTICE:  Loop: 4
NOTICE:  Loop: 5
NOTICE:  Loop: 6
NOTICE:  Loop: 7
NOTICE:  Loop: 8
ANONYMOUS BLOCK EXECUTE
```

Using the Loop statement in conjunction with CONTINUE directly starts the next loop.

The following stored procedure indicates that when the remainder of **i** to 2 is **1**, the next loop is skipped. If the remainder is not **1**, the result is output.

```
openGauss=# DO $$
DECLARE
  i integer := 0;
BEGIN LOOP
     EXIT WHEN i = 8;
     i := i + 1;
     CONTINUE WHEN mod(i, 2) = 1;
     RAISE NOTICE 'Loop: %', i;
  END LOOP;
END $$;

NOTICE:  Loop: 2
NOTICE:  Loop: 4
NOTICE:  Loop: 6
NOTICE:  Loop: 8
NOTICE:  Loop: 10
ANONYMOUS BLOCK EXECUTE
```

3.9.7.2 FOR Statement

Use the FOR statement to iterate the value of **i** over the range of 1 to 5, until the loop ends.

```
openGauss=# DO $$
BEGIN
  FOR i IN 1..6  LOOP
     RAISE NOTICE 'Output: %', i;
  END LOOP;
END $$;

NOTICE: Output: 1
NOTICE: Output: 2
NOTICE: Output: 3
NOTICE: Output: 4
NOTICE: Output: 5
NOTICE: Output: 6
ANONYMOUS BLOCK EXECUTE
```

Use the FOR statement to iterate over the query results of a data table.

```
openGauss=# DO $$
DECLARE
 e record;
BEGIN
 FOR e IN (SELECT * FROM employees LIMIT 5) LOOP
 RAISE NOTICE 'Loop: %,%', e.first_name, e.last_name;
 END LOOP;
END $$;

NOTICE:  Loop: Neena,Kochhar
NOTICE:  Loop: Ismael,Sciarra
NOTICE:  Loop: Luis,Popp
NOTICE:  Loop: Den,Raphaely
NOTICE:  Loop: David,Austin
ANONYMOUS BLOCK EXECUTE
```

3.9.8 Error Trapping Statement

By default, any error occurring in a stored procedure aborts execution, and the transaction is rolled back. You can trap errors and continue the transaction by using an EXCEPTION clause in the code block.

If an error occurs during code execution, the program enters the EXCEPTION module.

```
openGauss=# CREATE OR REPLACE PROCEDURE proc_exp()
AS
DECLARE
      x INT :=0;
      y INT;
BEGIN
      UPDATE employees SET first_name = 'James' WHERE first_name = 'John';
      x := x + 1;
      y := x / 0;
EXCEPTION
      WHEN division_by_zero THEN
            RAISE NOTICE 'caught division_by_zero';
            RAISE NOTICE 'x: %',x;
            RETURN ;
END;
/
CREATE PROCEDURE

openGauss=# call proc_exp();
NOTICE:  caught division_by_zero
NOTICE:  x: 1
```

3.9.9 Triggers

A trigger automatically executes functions when an event occurs in a specified database.

A trigger function is different from a regular function because it does not have parameters and its return value is a trigger type.

FOR EACH ROW specifies a row-level trigger, and FOR EACH STATEMENT specifies a statement-level trigger.

The data type of NEW is RECORD, indicating the new data rows after a row-level trigger performs the INSERT and UPDATE operations. The data type of OLD is RECORD, indicating the old data rows before a row-level trigger performs the UPDATE and DELETE operations.

An event can be INSERT, UPDATE, or DELETE. UPDATE supports the update operation of specific fields (for example, UPDATE OF col1, clo2). A trigger can be triggered before (specified by **BEFORE**) or after (specified by **AFTER**) an event.

Step 1 Create table **employees_history**.

```
CREATE TABLE employees_history (
      employee_id integer NOT NULL, /*Employee ID, the primary key*/
      first_name VARCHAR(20), /*First name*/
      last_name VARCHAR(25) NOT NULL, /*Last name*/
      hire_date timestamp(0) without time zone NOT NULL, /*Hiring date*/
      salary numeric(8,2), /*Salary*/
      department_id integer, /*Department ID*/
  action_type varchar(10) not null, /*Operation type*/
  change_dt timestamp not null,/*Operation time*/
      CONSTRAINT emp_salary_min CHECK (salary > (0))
);
```

Step 2 Create a trigger function to insert old data into the **employees_history** table before the data in the **employees** table is modified.

```
openGauss=# create or replace function employees_update_change()
returns trigger
as $$
begin
      INSERT INTO employees_history
         (employee_id, first_name, last_name, hire_date,salary,department_id, action_type, change_dt)
      values
         (old.employee_id, old.first_name, old.last_name, old.hire_date, old.salary,old.department_id,
'UPDATE', current_timestamp);
          RETURN new;
end;
$$ language plpgsql;
```

Step 3 Create a trigger and call the trigger function employees_update_change().

```
openGauss=# create trigger trg_employees_change
  before UPDATE on employees
  for each row
  execute PROCEDURE    employees_update_change();
```

Step 4 Change the **salary** of the employee whose **employee_id** is **145** to **10** in the **employees** table.

```
openGauss=# select * from employees;
 employee_id | first_name | last_name   |         hire_date          |   salary   | department_id
-------------+------------+-------------+----------------------------+------------+---------------
         101 | Neena      | Kochhar     | 2005-09-21 00:00:00        | 17000.00 |   90
         111 | Ismael     | Sciarra     | 2005-09-30 00:00:00        |   7700.00 |   100
         113 | Luis       | Popp        | 2007-12-07 00:00:00        |   6900.00 |   100
         114 | Den        | Raphaely    | 2002-12-07 00:00:00        | 11000.00 |    30
         105 | David      | Austin      | 2005-06-25 00:00:00        |   4800.00 |    60
         106 | Valli      | Pataballa   | 2006-02-05 00:00:00        |   4800.00 |    60
         117 | Sigal      | Tobias      | 2005-07-24 00:00:00        |   2800.00 |    30
         185 | Alexis     | Bull        | 2005-02-20 00:00:00        |   4100.00 |    50
         186 | Julia      | Dellinger   | 2006-06-24 00:00:00        |   3400.00 |    50
         145 | John       | Russell     | 2004-10-01 00:00:00        | 14000.00 |    80

openGauss=# update employees set salary=10 where employee_id=145;
UPDATE 1
```

Step 5 Check the **employees** table. The **salary** has been changed from **14000** to **10**.

```
openGauss=# select * from employees;
 employee_id | first_name | last_name   |         hire_date          |   salary   | department_id
-------------+------------+-------------+----------------------------+------------+---------------
         101 | Neena      | Kochhar     | 2005-09-21 00:00:00        | 17000.00 |    90
         111 | Ismael     | Sciarra     | 2005-09-30 00:00:00        |   7700.00 |   100
         113 | Luis       | Popp        | 2007-12-07 00:00:00        |   6900.00 |   100
         114 | Den        | Raphaely    | 2002-12-07 00:00:00        | 11000.00 |    30
         105 | David      | Austin      | 2005-06-25 00:00:00        |   4800.00 |    60
         106 | Valli      | Pataballa   | 2006-02-05 00:00:00        |   4800.00 |    60
         117 | Sigal      | Tobias      | 2005-07-24 00:00:00        |   2800.00 |    30
         185 | Alexis     | Bull        | 2005-02-20 00:00:00        |   4100.00 |    50
         186 | Julia      | Dellinger   | 2006-06-24 00:00:00        |   3400.00 |    50
         145 | John       | Russell     | 2004-10-01 00:00:00        |     10.00 |    80
```

The result shows that the **employees_history** table retains the values before modification.

```
openGauss=# select * from employees_history;
 employee_id | first_name| last_name | hire_date |   salary   | department_id | action_type | change_dt
-------------+-----------+-----------+---------------------+------------+---------------+-------------+----------------------------
         145 | John      | Russell | 2004-10-01 00:00:00 | 14000.00 |    80 | UPDATE | 2023-10-16
11:36:33.22553
```

3.10 API Development

This lab practices the use of JDBC, ODBC, and Python APIs in openGauss.

3.10.1 JDBC

Java Database Connectivity (JDBC) complies with Java standards. It provides an API for Java programs to connect to relational databases.

The JDBC standard is defined and implemented through the standard java.sql interfaces. This enables individual providers to implement and extend the standard with their own JDBC drivers.

Step 1 Check the data directory.

Search for the **pg_hba.conf** file in the data directory. This lab assumes that the directory is **/opt/huawei/install/data/dn**.

```
[omm@Run861 ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:37:13 commit 0 last mr  )
NOTICE : The password has been expired, please change the password.
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# show  data_directory;
          data_directory
----------------------------
 /opt/huawei/install/data/dn
(1 row)

openGauss=# \q
```

Step 2 Append **host all all 0.0.0.0/0 sha256** to the **pg_hba.conf** file.

If the message "FATAL: Forbid remote connection with trust method!" is displayed, change client access authentication mode of the target IP address **trust** to **sha256**.

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD
# "local" is for Unix domain socket connections only
local   all             all                                     trust
# IPv4 local connections:
host    all             all             127.0.0.1/32            trust
host    all             all             0.0.0.0/0               sha256
```

Step 3 Log in as user **omm** and restart the database for the policy to take effect.

```
[omm@Run861 dn]$ gs_om -t restart
Successfully started cluster.
```

Step 4 Create user **dbuser**, set the password to **openGauss@123456**, and grant the Sysadmin permission to the user.

```
[omm@Run861 dn]$ gsql -d postgres -p 26000 -r
openGauss=# CREATE USER dbuser IDENTIFIED BY 'openGauss@123456';
CREATE ROLE

openGauss=# alter user dbuser Sysadmin;
```

```
ALTER ROLE
openGauss=# \du

Role name | Attributes                                                                                                 | Member of
--------------+------------------------------------------------------------------------------------------------------------ ------+-
----------
 dbuser     | Sysadmin                                                                                                   | {}
 omm        | Sysadmin, Create role, Create DB, Replication, Administer audit, Monitoradmin,
Operatoradmin, Policyadmin, UseFT | {}
```

Step 5 Change the database listening address.

In this lab, the data directory is set to **/opt/huawei/install/data/dn**.

```
cd /opt/huawei/install/data/dn
vi postgresql.conf
```

Change the value of **listen_addresses** to *****.

```
listen_addresses = '*'                  # what IP address(es) to listen on;
                                        # comma-separated list of addresses;
                                        # defaults to 'localhost'; use '*' for all
                                        # (change requires restart)
```

Restart the database for the modification to take effect.

```
[omm@Run861 dn]$ gs_om -t restart
Successfully started cluster.
```

Step 6 Create test database **demo**.

Use the **gsql** tool to connect to the database. Log in as user **dbuser**, and the password is **openGauss@123456**.

```
gsql -d postgres -p 26000 -U dbuser -r
```

Create database **demo**.

```
openGauss=> create database demo ENCODING 'UTF8' template = template0;
CREATE DATABASE
```

Switch to database **demo**.

```
openGauss=> \connect   demo
Password for user dbuser:
Non-SSL connection (SSL connection is recommended when requiring high-security)
You are now connected to database "demo" as user "dbuser".
demo=>
```

Step 7 Create schema **demo**.

```
demo=> CREATE SCHEMA demo;
```

```
CREATE SCHEMA
```

Set the default search path to **demo**.

```
demo=> SET search_path TO demo;
SET
```

Step 8 Create test table **websites**.

```
demo=> CREATE TABLE websites (
id int NOT NULL,
name char(20) NOT NULL DEFAULT '',
url varchar(255) NOT NULL DEFAULT '',
PRIMARY KEY (id)
);
COMMENT ON COLUMN websites.name IS' Site Name';
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "websites_pkey" for table
"websites"
CREATE TABLE
COMMENT
```

Step 9 Insert data.

```
demo=> INSERT INTO websites VALUES
('1', 'openGauss', 'https://opengauss.org/zh/'),
('2','Huawei Cloud', 'https://www.huaweicloud.com/'),
('3', 'openEuler', 'https://openeuler.org/zh/'),
('4','Huawei Support Center', 'https://support.huaweicloud.com/')
INSERT 0 4
```

Step 10 Install Java runtime environment (JRE). Run the **java –version** command as user **root**. If the Java environment does not exist, perform this step to install the JRE.

Install Arm openJDK.

```
[root@ecs-3ddf ~]#cd /home
[root@ecs-3ddf home]# wget https://github.com/AdoptOpenJDK/openjdk8-
binaries/releases/download/jdk8u252-b09/OpenJDK8U-jdk_aarch64_linux_hotspot_8u252b09.tar.gz
[root@ecs-3ddf home]# tar -zxf OpenJDK8U-jdk_aarch64_linux_hotspot_8u252b09.tar.gz -C /usr/local
```

Add environment variables. Append the following two export statements to the end of the line:

```
[root@ecs-3ddf home]# vim /etc/profile
export JAVA_HOME=/usr/local/jdk8u252-b09
export PATH=$JAVA_HOME/bin:$PATH
```

Make the environment variables take effect.

```
[root@ecs-3ddf home]#source /etc/profile
```

Check that openJDK has been installed successfully.

```
[root@ecs-3ddf home]#java -version
```

Step 11 Download the JDBC driver package corresponding to the database version, upload the package to the **/opt** directory, and decompress it. (The database version in this lab is openGauss 5.0.0.)

Download the corresponding JDBC package from the openGauss community website (https://opengauss.org/en/download/), and upload it to the **/opt** directory.

```
[root@Run861 opt]# ls -l
total 3396
drwxr-xr-x   6 omm    dbgrp      4096 Jun 12 10:00 huawei
-rw-r--r--   1 root root   1706097 Nov   9 10:45 openGauss-5.0.0-JDBC.tar.gz
drwxr-xr-x. 4 root root       4096 Mar 10   2023 patch_workspace
drwxr-xr-x   3 root root        4096 Jun   9 17:51 software
```

Decompress the JDBC driver package.

```
root@Run861 opt]# tar -xzvf openGauss-5.0.0-JDBC.tar.gz
total 3464
drwxr-xr-x   6 omm    dbgrp      4096 Jun 12 10:00 huawei
-rw-r--r--   1 omm    dbgrp 1706097 Nov   9 10:45 openGauss-5.0.0-JDBC.tar.gz
-rw-r--r--   1 root root     900999 Mar 29   2023 opengauss-jdbc-5.0.0.jar
drwxr-xr-x. 4 root root       4096 Mar 10   2023 patch_workspace
-rw-r--r--   1 root root     902147 Mar 29   2023 postgresql.jar
-rw-r--r--   1 root root       6925 Mar 29   2023 README_cn.md
-rw-r--r--   1 root root       8998 Mar 29   2023 README_en.md
drwxr-xr-x   3 root root        4096 Jun   9 17:51 software
```

Step 12 Create the **openGaussDemo.java** file in the **/opt** directory.

```
[root@Run861 opt]# vim openGaussDemo.java
import java.sql.*;
public class openGaussDemo {
static final String JDBC_DRIVER = "org.postgresql.Driver";
static final String DB_URL = "jdbc:postgresql://7.213.197.105:26000/demo";
// The username and password of the database need to be set as required.
static final String USER = "dbuser";
static final String PASS = "openGauss@123456";
public static void main(String[] args) {
Connection conn = null;
Statement stmt = null;
try{
// Register the JDBC driver.
Class.forName(JDBC_DRIVER);
// Open the link.
System.out.println ("Connecting to the database...");
conn = DriverManager.getConnection(DB_URL,USER,PASS);
// Execute the query.
System.out.println("Instantiating the Statement object...");
stmt = conn.createStatement();
String sql;
sql = "SELECT id, name, url FROM demo.websites";
ResultSet rs = stmt.executeQuery(sql);
```

```
// Open the database result set.
while(rs.next()){
// Search for fields.
int id = rs.getInt("id");
String name = rs.getString("name");
String url = rs.getString("url");
// Output data.
System.out.print("ID: " + id);
System.out.print(", Site name: "+ name);
System.out.print(", Site URL: "+ url);
System.out.print("\n");
}
// Close after completion.
rs.close();
stmt.close();
conn.close();
}catch(SQLException se){
// Fix the JDBC error.
se.printStackTrace();
}catch(Exception e){
//Fix the Class.forName error.
e.printStackTrace();
}finally{
//Disable the resource.
try{
if(stmt!=null) stmt.close();
}catch(SQLException se2){
}
// Do nothing.
try{
if(conn!=null) conn.close();
}catch(SQLException se){
se.printStackTrace();
}
}
System.out.println("Goodbye!");
}
}
```

Step 13 Log in to the host where the Java is installed as user **root**, and compile the Java program (access the directory of the Java program).

```
[root@Run861 opt]# javac -encoding utf-8 -cp    /opt/postgresql.jar    /opt/openGaussDemo.java
```

Step 14 Run the following command (pay attention to ".:"):

```
[root@Run861 opt]# java -cp .:/opt/postgresql.jar    openGaussDemo
```

Step 15 View the execution result.

```
--Connect to the database...
Nov 09, 2023 11:11:02 AM org.postgresql.core.v3.ConnectionFactoryImpl openConnectionImpl
INFO: [c4d286f9-63de-47a9-b53e-b56780225cf3] Try to connect. IP: 7.213.197.105:15400
Nov 09, 2023 11:11:02 AM org.postgresql.core.v3.ConnectionFactoryImpl openConnectionImpl
```

```
INFO: [7.213.197.105:33672/7.213.197.105:15400] Connection is established. ID: c4d286f9-63de-47a9-b53e-b56780225cf3
Nov 09, 2023 11:11:02 AM org.postgresql.core.v3.ConnectionFactoryImpl openConnectionImpl
INFO: Connect complete. ID: c4d286f9-63de-47a9-b53e-b56780225cf3
--Instantiate a Statement object...
ID: 1, Site name: openGauss, Site URL: https://openGauss.org/zh/
ID: 2, Site name: Huawei Cloud, Site URL: https://www.huaweicloud.com/
ID: 3, Site name: openEuler, Site URL: https://openeuler.org/zh/
ID: 4, Site name: Huawei Support Center, Site URL: https://support.huaweicloud.com/
Goodbye!
```

3.10.2 ODBC

Step 1 Download and decompress the unixODBC installation package.

Download the unixODB **unixODBC-2.3.0.tar.gz** from https://sourceforge.net/projects/unixodbc/files/unixODBC, and upload it to the **/opt/odbc** directory.

```
[root@Run861 odbc]# pwd
/opt/odbc
[root@Run861 odbc]# ls -l
total 1764
-rw-r--r-- 1 root root 1804749 Nov  9 16:17 unixODBC-2.3.0.tar.gz
[root@Run86 odbc]# tar -zxvf unixODBC-2.3.0.tar.gz
```

Step 2 Compile the ODBC.

Modify the **configure** file. (If it does not exist, modify the **configure.ac** file.) Find **LIB_VERSION**.

Change the value to **1:0:0** to compile a *.so.1 dynamic library with the same dependency on **psqlodbcw.so**.

```
cd unixODBC-2.3.0
[root@Run861 unixODBC-2.3.0]# pwd
/opt/odbc/unixODBC-2.3.0
[root@Run861 unixODBC-2.3.0]# vim configure
[root@Run861 unixODBC-2.3.0]# cat  configure | grep LIB_VERSION
LIB_VERSION
LIB_VERSION="1:0:0"
[root@Run861 unixODBC-2.3.0]# ./configure --enable-gui=no
```

To perform compilation on an Arm server, add the **configure** parameter **--build=aarch64-unknown-linux-gnu**.

```
[root@Run861 unixODBC-2.3.0]# make
[root@Run861 unixODBC-2.3.0]# make install
```

Step 3 Upload the **openGauss-5.0.0-ODBC.tar.gz** file and decompress it.

```
[root@Run861 odbc]# pwd
/opt/odbc
[root@Run861 odbc]# tar -zvxf openGauss-5.0.0-ODBC.tar.gz #Upload openGauss-5.0.0-ODBC.tar.gz to /opt/odbc and decompress it.
/opt/odbc
[root@Run861 odbc]# ls -l
total 10888
drwxr-xr-x   2 root root       4096 Mar 29   2023 lib
drwxr-xr-x   3 root root       4096 Mar 29   2023 odbc
-rw-r--r--   1 root root    9327411 Nov   9 17:00 openGauss-5.0.0-ODBC.tar.gz
drwxr-xr-x 19 omm   dbgrp      4096 Nov   9 16:51 unixODBC-2.3.0
-rw-r--r--   1 root root    1804749 Nov   9 16:17 unixODBC-2.3.0.tar.gz
```

Step 4 Copy files in the odbc directory to **/usr/local/lib**.

Decompress the **openGauss-*x.x.x*-ODBC.tar.gz** package. After the decompression, the **lib** and **odbc** folders are generated. The **odbc** folder contains another **lib** folder. Copy the **psqlodbca.la**, **psqlodbca.so**, **psqlodbcw.la**, and **psqlodbcw.so** files from **/odbc/lib** to **/usr/local/lib**.

```
[root@Run861 odbc]# cd lib/
[root@Run861 lib]# ls -l
total 1508
-rwxr-xr-x 1 root root     1271 Mar 29   2023 psqlodbca.la
-rwxr-xr-x 1 root root 740552 Mar 29   2023 psqlodbca.so
-rwxr-xr-x 1 root root     1271 Mar 29   2023 psqlodbcw.la
-rwxr-xr-x 1 root root 791376 Mar 29   2023 psqlodbcw.so
[root@Run861 lib]# pwd
/opt/odbc/odbc/lib
[root@Run861 lib]# pw
-bash: pw: command not found
[root@Run861 lib]# pwd
/opt/odbc/odbc/lib
[root@Run861 lib]# cp /opt/odbc/odbc/lib/*   /usr/local/lib/
```

Copy the library in the **lib** directory obtained after decompressing **openGauss-*x.x.x*-ODBC.tar.gz** package to the **/usr/local/lib** directory.

```
[root@Run861 lib]# cp /opt/odbc/lib/*   /usr/local/lib/
```

Step 5 Configure the ODBC driver file.

Append the following content to the **/usr/local/etc/odbcinst.ini** file:

```
[GaussMPP]
Driver64=/usr/local/lib/psqlodbcw.so
setup=/usr/local/lib/psqlodbcw.so
```

[DriverName]: driver name, corresponding to **Driver** in DSN.

Driver64: path of the dynamic driver library.

Setup: driver installation path, which is the same as the dynamic library path in Driver64.

Step 6 Configure the data source file.

Append the following content to the **/usr/local/etc/odbc.ini** file:

```
[MPPODBC]
Driver=GaussMPP
Servername=7.213.197.105
Database=demo
Username=dbuser
Password=openGauss@123456
Port=15400
Sslmode=allow
```

Servername: database server IP address.

Driver: driver name, corresponding to **DriverName** in **odbcinst.ini**.

Database: name of the database to be connected.

Username: database username.

Password: password of the database user.

Port: database listening port.

Step 7 Configure the database server.

Log in as the OS user **omm** to the primary node of the database.

Add NIC IP addresses or host names separated by commas (,). The NICs and hosts are used to provide external services. In the following command, *NodeName* specifies the name of the current node.

```
gs_guc reload -N NodeName -I all -c "listen_addresses='*'"
```

Add an authentication rule to the configuration file of the primary database node.

```
gs_guc reload -N all -I all -h "host all    all    0.0.0.0/0 sha256"
```

Restart openGauss.

```
gs_om -t stop
gs_om -t start
```

Step 8 Configure the environment variables on the client.

```
[root@Run861 ~]# vim ~/.bashrc
```

Append the following content to the end of the configuration file:

```
export LD_LIBRARY_PATH=/usr/local/lib/:$LD_LIBRARY_PATH
export ODBCSYSINI=/usr/local/etc
export ODBCINI=/usr/local/etc/odbc.ini
```

Run the following command to validate the setting:

```
source ~/.bashrc
```

Step 9 Verify the data source configuration.

After the installation, the generated binary file is stored in the **/usr/bin** directory. You can run the **isql -v** *MPPODBC* command (*MPPODBC* is the data source name).

If the following information is displayed, the configuration is correct and the connection succeeds.

```
[root@Run861 bin]#   isql -v   MPPODBC
+---------------------------------------+
| Connected!                            |
|                                       |
| sql-statement                         |
| help [tablename]                      |
| quit                                  |
|                                       |
+---------------------------------------+
SQL>
```

3.10.3 Python

3.10.3.1 Using Python to connect to openGauss

Step 1 Download the compiled psycopg2 package from the openGauss website.

In this lab, the OS is openEuler 22.03(LTS-SP1) x86_64 ,Python version is 3.9.9, and the openGauss database version is 5.0.0.

Download the software package of the corresponding database version from openGauss website (https://opengauss.org/en/download/). This lab uses **openGauss-5.0.0-openEuler-x86_64-Python.tar.gz**.

```
[root@Run861 dn]# python
Python 3.9.9 (main, Dec 27 2022, 17:54:38)
[GCC 10.3.1] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()

[root@Run861 dn]# cat /etc/os-release
NAME="openEuler"
VERSION="22.03 (LTS-SP1)"
ID="openEuler"
VERSION_ID="22.03"
PRETTY_NAME="openEuler 22.03 (LTS-SP1)"
ANSI_COLOR="0;31"

[root@Run861 ~]# uname -a
Linux Run861 5.10.0-136.12.0.86.oe2203sp1.x86_64 #1 SMP Tue Dec 27 17:50:15 CST 2022 x86_64
x86_64 x86_64 GNU/Linux
[root@Run861 python]# ls -l
total 3200
-rw-r--r-- 1 root root 3276346 Nov 15 17:40 openGauss-5.0.0-openEuler-x86_64-Python.tar.gz
```

Step 2 Decompress the **openGauss-5.0.0-openEuler-x86_64-Python.tar.gz** file.

```
[root@Run861 python]# tar -zvxf openGauss-5.0.0-openEuler-x86_64-Python.tar.gz
./lib/
./lib/libcrypto.so.1.1
./lib/libkrb5support_gauss.so.0
./lib/libpq.so.5
./lib/libssl.so.1.1
./lib/libgcc_s.so.1
./lib/libgssrpc_gauss.so.4
./lib/libstdc++.so.6
./lib/libcom_err_gauss.so.3
./lib/libgssapi_krb5_gauss.so.2
./lib/libkrb5_gauss.so.3
./lib/libk5crypto_gauss.so.3
./psycopg2/
./psycopg2/tz.py
./psycopg2/extensions.py
./psycopg2/errorcodes.py
./psycopg2/_json.py
./psycopg2/errors.py
./psycopg2/extras.py
./psycopg2/__init__.py
./psycopg2/pool.py
./psycopg2/_range.py
./psycopg2/sql.py
./psycopg2/_ipaddress.py
./psycopg2/_psycopg.so

 [root@Run861 python]# ls -l
total 2996
-rw-r--r-- 1 root root       147 Nov 16 02:08 connDatabase.py
drwxr-x--- 2 root root      4096 Mar 29   2023 lib
-rw-r--r-- 1 root root 3052351 Nov 16 03:32 openGauss-5.0.0-openEuler-x86_64-Python.tar.gz
drwxr-x--- 2 root root      4096 Mar 29   2023 psycopg2
```

Step 3 Locate the **site-packages** directory, and copy the **psycopg2** directory to the **site-packages** directory of the corresponding version as user **root**.

```
[root@Run861 python]# cp -r psycopg2/ /usr/lib/python3.9/site-packages/
```

Step 4 Change the permission on the **psycopg2** directory in **/usr/lib/python3.9/site-packages/** to **755**.

```
[root@Run861 site-packages]# pwd
/usr/lib/python3.9/site-packages
[root@Run861 site-packages]# chmod -R 775 psycopg2
```

Step 5 Test the connection.

Connect configures the database to be connected (**test**), username (**test**), password, host name, and port number.

Use the psycopg2.connect function to obtain a connection object. Use the connection object to create a cursor object.

Construct an operation statement and use %s as a placeholder. During execution, psycopg2 will replace the placeholder with the parameter value. You can add the RETURNING clause to obtain the automatically generated column values.

The cursor.execute method is used to perform operations on one row, and the cursor.executemany method is used to perform operations on multiple rows.

```
[omm@Run861 python]$ python3
Python 3.9.9 (main, Dec 27 2022, 17:54:38)
[GCC 10.3.1] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import psycopg2
>>> conn = psycopg2.connect(database="test", user="test",
password="test@123456",host="7.213.197.105", port="15400")
>>> cur=conn.cursor()
>>> cur.execute("CREATE TABLE student(id integer,name varchar,sex varchar);")
>>> cur.execute("INSERT INTO student(id,name,sex) VALUES(%s,%s,%s)",(1,'Aspirin','M'))
>>> cur.execute("INSERT INTO student(id,name,sex) VALUES(%s,%s,%s)",(2,'Taxol','F'))
>>> cur.execute('SELECT * FROM student')
>>> results=cur.fetchall()
>>> print (results)
[(1, 'Aspirin', 'M'), (2, 'Taxol', 'F')]
>>> conn.commit()
>>> cur.close()
>>> conn.close()
```

Step 6 Verify the data in the database.

```
test=> select * from student;
 id |   name    | sex
----+---------+-----
  1 | Aspirin | M
  2 | Taxol    | F
(2 rows)
```

3.10.3.2 Creating a Data Table Using a Python Script

Step 1 Edit the **creatTable.py** script.

```
[root@Run861 python]# more creatTable.py
#!/usr/bin/python

import psycopg2

conn = psycopg2.connect(database="test", user="test",
password="test@123456",host="7.213.197.105", port="15400")

cur = conn.cursor()
cur.execute('''CREATE TABLE COMPANY
(ID INT PRIMARY KEY NOT NULL,
NAME TEXT NOT NULL,
```

```
AGE INT NOT NULL,
ADDRESS CHAR(50),
SALARY REAL);''')
conn.commit()
conn.close()
```

Step 2 Use Python to execute the **creatTable.py** script.

```
[root@Run861 python]# python3 creatTable.py
```

Step 3 Check whether the table is created in the database.

```
test=> \d company
            Table "test.company"
 Column  |       Type         | Modifiers
---------+---------------+-----------
 id          | integer           | not null
 name       | text               | not null
 age        | integer           | not null
 address | character(50) |
 salary    | real                |
Indexes:
"company_pkey" PRIMARY KEY, ubtree (id) WITH (storage_type=USTORE) TABLESPACE pg_default
```

3.10.3.3 Inserting Data Into a Table Using a Python Script

Step 1 Edit the **insertData.py** script.

```
[root@Run861 python]# cat insertData.py
#!/usr/bin/python

import psycopg2

conn = psycopg2.connect(database="test", user="test",
password="test@123456",host="7.213.197.105", port="15400")

cur = conn.cursor()
cur.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
 VALUES (1, 'Paul', 32, 'California', 20000.00 )");
cur.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
 VALUES (2, 'Allen', 25, 'Texas', 15000.00 )");
cur.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
 VALUES (3, 'Teddy', 23, 'Norway', 20000.00 )");
cur.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
 VALUES (4, 'Mark', 25, 'Rich-Mond ', 65000.00 )");
conn.commit()
conn.close()
```

Step 2 Use Python to execute the **insertData.py** script.

```
[root@Run861 python]# vim insertData.py
```

Step 3 Check whether data is correctly inserted into the **company** table in the database.

```
test=> select * from company   ;
 id | name   | age |                                address                                | salary
----+-------+-----+-----------------------------------------------------------+--------
  1 | Paul   |  32 | California                                                |  20000
  2 | Allen |  25 | Texas                                                     |  15000
  3 | Teddy |  23 | Norway                                                    |  20000
  4 | Mark   |  25 | Rich-Mond                                                 |  65000
```

3.10.3.4 Viewing Data Using a Python Script

Step 1 Edit the **selectData.py** script.

```
#!/usr/bin/python

import psycopg2

conn = psycopg2.connect(database="test", user="test",
password="test@123456",host="7.213.197.105", port="15400")

cur = conn.cursor()
cur.execute("SELECT id, name, address, salary from COMPANY")
rows = cur.fetchall()
for row in rows:
      print("ID = ", row[0])
      print("NAME = ", row[1])
      print("ADDRESS = ", row[2])
      print("SALARY = ", row[3])
conn.close()
```

Step 2 Use Python to execute the **selectData.py** script.

```
root@Run861 python]# python3 selectData.py
ID =  1
NAME =  Paul
ADDRESS =  California
SALARY =  20000.0
ID =  2
NAME =  Allen
ADDRESS =  Texas
SALARY =  15000.0
ID =  3
NAME =  Teddy
ADDRESS =  Norway
SALARY =  20000.0
ID =  4
NAME =  Mark
ADDRESS =  Rich-Mond
SALARY =  65000.0
```

3.10.3.5 Updating Data Using a Python Script

Step 1 Edit the **updateData.py** script.

```
[root@Run861 python]# cat updateData.py
#!/usr/bin/python

import psycopg2

conn = psycopg2.connect(database="test", user="test",
password="test@123456",host="7.213.197.105", port="15400")

cur = conn.cursor()
cur.execute("UPDATE COMPANY set SALARY = 25000.00 where ID=1")
conn.commit
cur.execute("SELECT id, name, address, salary from COMPANY")
rows = cur.fetchall()
for row in rows:
  print("ID = ", row[0])
  print("NAME = ", row[1])
  print("ADDRESS = ", row[2])
  print("SALARY = ", row[3])
conn.close()
```

Step 2 Use Python to execute the **updateData.py** script.

```
[root@Run861 python]# python3 updateData.py
ID =  1
NAME =  Paul
ADDRESS =  California
SALARY =  25000.0
ID =  2
NAME =  Allen
ADDRESS =  Texas
SALARY =  15000.0
ID =  3
NAME =  Teddy
ADDRESS =  Norway
SALARY =  20000.0
ID =  4
NAME =  Mark
ADDRESS =  Rich-Mond
SALARY =  65000.0
```

3.10.3.6 Deleting Data Using a Python Script

Step 1 Edit the **deleteData.py** script.

```
[root@Run861 python]# cat deleteData.py
#!/usr/bin/python

import psycopg2
```

```
conn = psycopg2.connect(database="test", user="test",
password="test@123456",host="7.213.197.105", port="15400")

cur = conn.cursor()
cur.execute("DELETE from COMPANY where ID=2;")
conn.commit
cur.execute("SELECT id, name, address, salary from COMPANY")
rows = cur.fetchall()
for row in rows:
  print("ID = ", row[0])
  print("NAME = ", row[1])
  print("ADDRESS = ", row[2])
  print("SALARY = ", row[3])
conn.close()
```

Step 2 Use Python to execute the **deleteData.py** script.

```
[root@Run861 python]# python3 deleteData.py
ID =  1
NAME =  Paul
ADDRESS =  California
SALARY =  25000.0
ID =  3
NAME =  Teddy
ADDRESS =  Norway
SALARY =  20000.0
ID =  4
NAME =  Mark
ADDRESS =  Rich-Mond
SALARY =  65000.0
```

4 Database Migration Management

4.1 Introduction

4.1.1 About This Lab

The MySQL migration process goes through five phases: full migration, full verification, incremental migration, incremental verification, and reverse migration. The implementation principles have been described in detail in the certification course. openGauss offers DataKit, a toolkit that manages data throughout its lifecycle. DataKit uses a browser/server (B/S) architecture to manage software and hardware resources, including server and database instances. It can be expanded through on-demand deployment of function plug-ins, which currently include installation and deployment, data development, intelligent O&M, and data migration. The data migration plug-in provides the complete capabilities required for migrating data from MySQL. This chapter describes how to set up the DataKit platform and provides hands-on exercises of MySQL migration.

4.1.2 Objectives

In this lab, you will find practical exercises for the data migration process covered in chapter 5 "Database Migration Management" of the HCIP-openGauss V1.0 certification course. By completing this lab, you will have the skills to perform data migration from start to finish.

4.2 Environment Configuration

To install and operate the openGauss database, a non-root user is required. In this manual, the data migration process using DataKit is carried out by individual user **omm**. Unless specified otherwise, all operations are performed as user **omm**.

Previous courses have already covered the installation and deployment process for openGauss. Therefore, in this lab, openGauss has already been deployed and installed as user **omm**. The following will detail how to install and deploy MySQL and DataKit on the same node as user **omm** and complete the entire migration process.

4.2.1 JDK Installation and Configuration

DataKit is a migration platform written in Java. Therefore, before setting up and using DataKit, it is necessary to install and configure the JRE properly.

On an Arm server, DataKit 5.1.0 works with JDK 11 or later.

Run the **java -version** command to check the JDK version.

```
[test@openEuler2003SP1 ~]$ java -version
-bash: java: command not found
```

If the environment does not have the JDK (as shown in the previous figure) or the JDK version is earlier than 11, install or reinstall the JDK as follows.

Step 1 Download the JDK.

This lab assumes that the JDK is installed in the **/opt/DataKit/tools/jdk** directory. Log in to the database node as user **root**, create a directory, change the owner of the **/opt** directory to user **omm**, switch to user **omm**, and download the necessary installation package.

```
[root@ecs-dc2b ~]# mkdir -p /opt/datakit/tools/jdk
[root@ecs-dc2b ~]# chown -R omm:dbgrp /opt
[root@ecs-dc2b ~]# su - omm
[omm@ecs-dc2b ~]$ cd /opt/datakit/tools/jdk
[omm@ecs-dc2b jdk]$ wget
https://mirrors.huaweicloud.com/kunpeng/archive/compiler/bisheng_jdk/bisheng-jdk-11.0.20-linux-aarch64.tar.gz
[omm@ecs-dc2b jdk]$ tar -zxvf bisheng-jdk-11.0.20-linux-aarch64.tar.gz
```

Step 2 Set environment variables.

```
[omm@ecs-dc2b jdk]$ vim ~/.bashrc
```

Append the following lines to the file:

```
export JAVA_HOME=/opt/datakit/tools/jdk/bisheng-jdk-11.0.20
export PATH=$JAVA_HOME/bin:$PATH
```

Run the **:wq** command to save the file and exit. Then, run the following command:

```
[omm@ecs-dc2b jdk]$ source ~/.bashrc
```

Check that JDK has been successfully installed in your environment.

```
[omm@ecs-dc2b jdk]$ java -version
```

```
[test@openEuler2003SP1 jdk]$ java -version
openjdk version "11.0.20" 2023-07-18
OpenJDK Runtime Environment BiSheng (build 11.0.20+11)
OpenJDK 64-Bit Server VM BiSheng (build 11.0.20+11, mixed mode)
```

4.2.2 MySQL Installation and Configuration

Step 1 Download MySQL.

Assume that MySQL is installed in the **/opt/DataKit/tools/mysql5.7** directory.

```
[omm@ecs-dc2b jdk]$ mkdir -p /opt/datakit/tools/mysql5.7
[omm@ecs-dc2b jdk]$ cd /opt/datakit/tools/mysql5.7
```

```
[omm@ecs-dc2b mysql5.7]$ wget https://obs.cn-north-4.myhuaweicloud.com/obs-mirror-ftp4/database/mysql-5.7.27-aarch64.tar.gz
[omm@ecs-dc2b mysql5.7]$ tar -xvf mysql-5.7.27-aarch64.tar.gz
```

Step 2 Create a configuration file.

Create the configuration file **/opt/DataKit/tools/mysql5.7/my.cnf**.

```
[omm@ecs-dc2b mysql5.7]$ vim /opt/datakit/tools/mysql5.7/my.cnf
```

Fill in the following configuration items:

```
[mysqld]
port=3306
basedir=/opt/datakit/tools/mysql5.7/mysql-5.7.27-aarch64
datadir=/opt/datakit/tools/mysql5.7/datadir
pid-file=/opt/datakit/tools/mysql5.7/mysql.pid
socket=/opt/datakit/tools/mysql5.7/mysql.sock
log_error=/opt/datakit/tools/mysql5.7/error.log
server-id=1
enforce_gtid_consistency=on
gtid_mode=on
binlog_format= ROW
log_bin=mysql-bin
binlog_row_image=FULL
bind_address = xx.xx.xx.xx
```

Replace **bind_address = *xx.xx.xx.xx*** with the actual IP address. You can run the **ifconfig** command to view your IP address. If a Huawei Cloud ECS is used, your IP address is a private address.

Run **:wq** to save the file and exit.

Step 3 Set environment variables.

Set environment variables.

```
[omm@ecs-dc2b mysql5.7]$ vim ~/.bashrc
```

Append the following lines to the file:

```
export MYSQL_PATH=/opt/datakit/tools/mysql5.7/mysql-5.7.27-aarch64
export PATH=$MYSQL_PATH/bin:$PATH
```

Run the **:wq** command to save the file and exit. Then, run the following command:

```
[omm@ecs-dc2b mysql5.7]$ source ~/.bashrc
```

Check the MySQL version and verify that the environment variables have taken effect.

```
[[omm@ecs-dc2b mysql5.7]$ mysql --version
```

```
[test@openEuler2003SP1 bin]$ mysql --version
mysql  Ver 14.14 Distrib 5.7.27, for Linux (aarch64) using  EditLine wrapper
```

Step 4 Install MySQL.

```
[omm@ecs-840f mysql5.7]$ mysqld --defaults-file=/opt/datakit/tools/mysql5.7/my.cnf --initialize --user=omm --basedir=/opt/datakit/tools/mysql5.7/mysql-5.7.27-aarch64 --datadir=/opt/datakit/tools/mysql5.7/datadir
```

If you receive the following error message during the installation, "./mysqld: error while loading shared libraries: libatomic.so.1: cannot open shared object file: No such file or directory," fix the error as follows:

Create a soft link as the **root** user.

```
[root@openEuler2003SP1 ~]$ ln -s /opt/datakit/tools/mysql5.7/mysql-5.7.27-aarch64/extra/libatomic.so.1 /usr/lib64/libatomic.so.1
```

Step 5 Start MySQL.

```
[omm@ecs-840f mysql5.7]$ mysqld_safe --defaults-file=/opt/datakit/tools/mysql5.7/my.cnf --user=omm &
```

Step 6 Check the temporary password of the **root** user.

```
[omm@ecs-840f mysql5.7]$ cat /opt/datakit/tools/mysql5.7/error.log | grep root@localhost
```

```
[test@openEuler2003SP1 bin]$ cat /opt/datakit/tools/mysql5.7/error.log | grep root@localhost
2023-08-26T09:37:53.688962Z 1 [Note] A temporary password is generated for root@localhost: kr%rYP#wa9mf
```

Step 7 Log in to MySQL.

Use the temporary password to log in to MySQL.

```
[omm@ecs-840f mysql5.7]$ mysql -u root -p -S /opt/datakit/tools/mysql5.7/mysql.sock
```

Step 8 Set a new password for the **root** user

This example assumes that the new password of the **root** user is set to **123456**.

```
mysql> SET PASSWORD FOR 'root'@'localhost' = PASSWORD('123456');
```

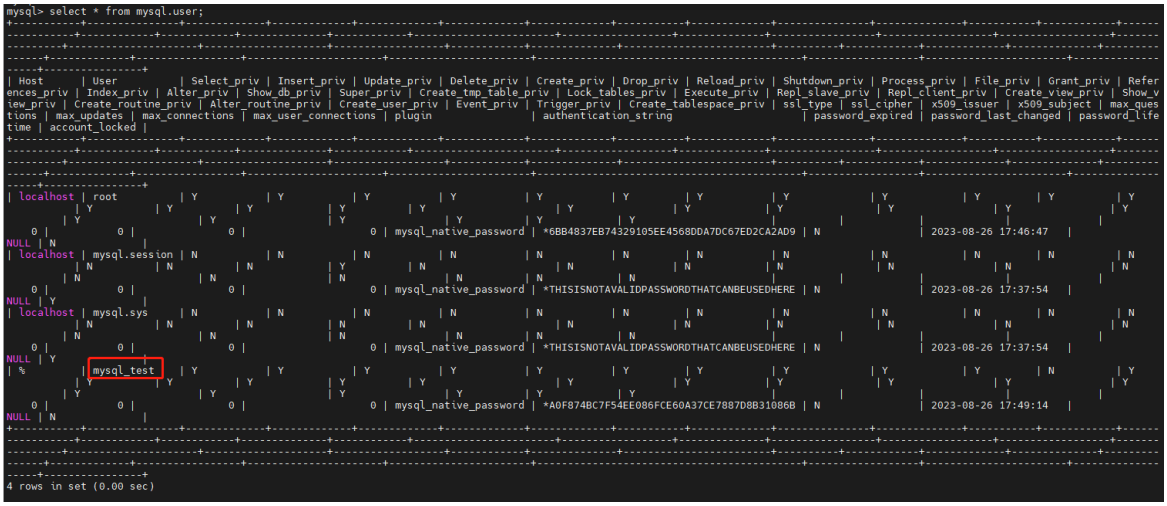
Step 9 Create a user.

Create user **mysql_test**, set the password to **password123**, and grant necessary permissions to the user for data migration.

```
mysql> SET PASSWORD FOR 'root'@'localhost' = PASSWORD('123456');
mysql> CREATE USER mysql_test;
mysql> SET PASSWORD FOR mysql_test=PASSWORD('password123');
mysql> GRANT ALL ON *.* TO 'mysql_test';
mysql> GRANT RELOAD ON *.* to 'mysql_test';
mysql> GRANT REPLICATION CLIENT ON *.* to 'mysql_test';
mysql> GRANT REPLICATION SLAVE ON *.* to 'mysql_test';
mysql> FLUSH PRIVILEGES;
```

Check that the new MySQL user has been successfully created.

```
mysql> select * from mysql.user;
```



4.2.3 Common MySQL Commands

To manually start, stop, or connect to MySQL, use the following commands.

Exit MySQL:

```
mysql> \q
Bye
[omm@ecs-840f mysql5.7]$
```

Start MySQL:

```
[omm@ecs-840f mysql5.7]$ mysqld_safe --defaults-file=/opt/datakit/tools/mysql5.7/my.cnf --
user=omm &
```

Remotely connect to MySQL:

```
 [omm@ecs-840f mysql5.7]$ mysql -h xx.xx.xx.xx -P 3306 -u mysql_test -ppassword123 -S
/opt/datakit/tools/mysql5.7/mysql.sock
```

Replace *xx.xx.xx.xx* specified by parameter **-h** with the actual IP address. The password is **password123**.

Stop MySQL:

```
[omm@ecs-840f mysql5.7]$ mysqladmin shutdown -u mysql_test -p -S
/opt/datakit/tools/mysql5.7/mysql.sock
```

4.2.4 openGauss Settings

Step 1 Set parameters in the **pg_hba.conf** configuration file.

```
[omm@ecs-840f dn]$ gs_guc set -D /opt/huawei/install/data/dn -h "host all all 0.0.0.0/0 sha256"
```

```
[omm@ecs-840f dn]$ gs_guc set -D /opt/huawei/install/data/dn -h "host replication all 0.0.0.0/0
sha256"
```

The **replication** line is the permission configuration for reverse migration.

Step 2 Set parameters in the **postgresql.conf** configuration file.

```
[omm@ecs-840f dn]$ gs_guc set -D /opt/huawei/install/data/dn -c "listen_addresses = '*'"
[omm@ecs-840f dn]$ gs_guc set -D /opt/huawei/install/data/dn -c "wal_level = logical"
```

The **wal_level=logic** line is the parameter configuration for reverse migration.

Step 3 Restart the database for the preceding parameters to take effect.

```
[omm@ecs-840f ~]$ gs_om -t restart
```

Step 4 Connect to the database.

```
[omm@ecs-840f ~]$ gsql -d postgres -p 26000 -r
```

Step 5 Create a migration user.

```
openGauss=# create user opengauss_test with password 'Sample@123';
```

Step 6 Grant permissions to the user.

```
openGauss=# grant all privileges to opengauss_test;
```

Step 7 Create a database.

```
openGauss=# create database datakit;
openGauss=# create database target_db with dbcompatibility = 'b';
```

The **datakit** database functions as the low-level database of DataKit, and the **target_db** database functions as the target database for data migration.

Step 8 Exit the database.

```
openGauss=# \q
```

Step 9 Connect to database **b** as the initial user.

```
[omm@ecs-840f ~]$ gsql -d target_db -p 26000 -r
```

Connect to database **b** as the initial user to load the dolphin plug-in for MySQL compatibility purposes during migration.

4.3 DataKit Data Migration Practice

4.3.1 Deploying DataKit

Step 1 Download DataKit.

Assume that DataKit is installed in the **/opt/DataKit/datakit5.1** directory.

```
[omm@ecs-840f ~]$ mkdir -p /opt/datakit/datakit5.1
[omm@ecs-840f ~]$ cd /opt/datakit/datakit5.1
[omm@ecs-840f datakit5.1]$ wget https://opengauss.obs.cn-south-
1.myhuaweicloud.com/5.1.0/tools/Datakit/Datakit-5.1.0.tar.gz
```

Step 2 Decompress the installation package.

```
[omm@ecs-840f datakit5.1]$ tar -zxvf Datakit-5.1.0.tar.gz
```

Step 3 Create a working directory.

```
[omm@ecs-840f datakit5.1]$ mkdir -p logs config ssl files
```

Step 4 Copy the configuration file.

Copy the configuration file to the **config** directory.

```
[omm@ecs-840f datakit5.1]$ cp ./application-temp.yml ./config
```

Step 5 Modify the configuration file.

Open the default configuration file **application-temp.yml** in the **config** directory, set the **ip**, **port**, **database**, **username**, and **password** parameters, change the default working directory **/ops** to **/opt/DataKit/datakit5.1**, and set **server-ssl-enabled** to **false**.

The datakit database functions as the low-level database of DataKit. Its database user is **opengauss_test** and the user password is **Sample@123**. Therefore, set the IP address to the actual IP address (private IP address of the ECS). Set **port** to the actual port number (26000, as specified by the lab and configured during openGauss database installation). Set **database** to **DataKit**, **username** to **opengauss_test**, and **password** to **Sample@123**.

```
[omm@ecs-840f datakit5.1]$ vi /opt/datakit/datakit5.1/config/application-temp.yml
```

The following gives an example of the modified configuration file.

```
system:
  # File storage path
  defaultStoragePath: /opt/datakit/datakit5.1/files
  # Whitelist control switch
  whitelist:
    enabled: false
server:
  port: 9494
```

```
  ssl:
    key-store: /opt/datakit/datakit5.1/ssl/keystore.p12
    key-store-password: 123456
    key-store-type: PKCS12
    enabled: false
  servlet:
    context-path: /
logging:
  file:
    path: /opt/datakit/datakit5.1/logs/
spring:
  datasource:
    type: com.alibaba.druid.pool.DruidDataSource
    driver-class-name: org.opengauss.Driver
    url: jdbc:opengauss://192.168.0.20:26000/datakit?currentSchema=public&batchMode=off
    username: opengauss_test
    password: Sample@123
    druid:
      test-while-idle: false
      test-on-borrow: true
      validation-query: "select 1"
      validation-query-timeout: 10000
      connection-error-retry-attempts: 0
      break-after-acquire-failure: true
      max-wait: 3000
management:
  server:
    port: 9494
```

Change the IP address (*xx.xx.xx.xx*) to the actual IP address and port number to the actual port number.

Step 6 Create an SSL file.

```
[omm@ecs-840f datakit5.1]$ keytool -genkey -noprompt -dname "CN=opengauss, OU=opengauss, O=opengauss, L=Beijing, S=Beijing, C=CN" -alias opengauss -storetype PKCS12 -keyalg RSA -keysize 2048 -keystore /opt/datakit/datakit5.1/ssl/keystore.p12 -validity 3650 -storepass 123456
```

Note that the above is a complete command.

storepass is equivalent to **key-store-password** in the configuration file, and the keystore path is equivalent to the key-store path in the configuration file.

Step 7 Start DataKit.

Go to the DataKit installation directory and start DataKit.

```
[omm@ecs-840f datakit5.1]$ cd /opt/datakit/datakit5.1
[omm@ecs-840f datakit5.1]$ nohup java -Xms2048m -Xmx4096m -jar openGauss-datakit-5.1.0.jar --spring.profiles.active=temp > ./logs/datakit.out 2>&1 &
```

Verify that DataKit has been started successfully and port 9494 is available.

```
[omm@ecs-840f datakit5.1]$ ss -lpnat | grep 9494
```

Return example:

```
[omm@ecs-840f datakit5.1]$ ss -lpnat | grep 9494
LISTEN 0      100                        *:9494                     *:*      users:(("java",pid=1983196,fd=48))
```

This example shows that the port is available and the startup is successful.

Step 8 Log in to the DataKit platform.

Log in to DataKit through a browser.

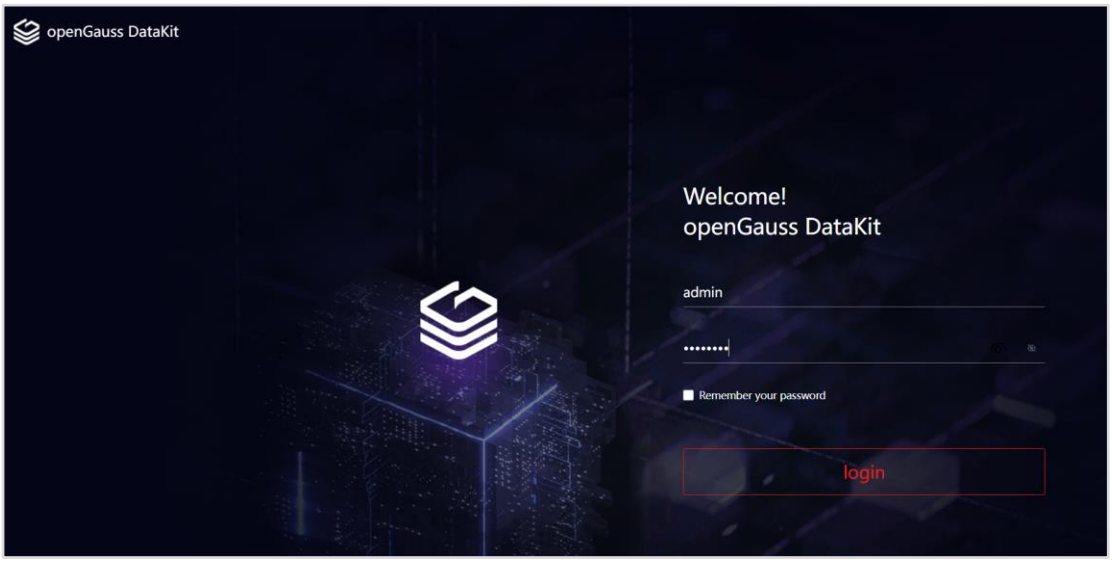
Login address: http://*xx.xx.xx.xx*:9494/, where *xx.xx.xx.xx* indicates the actual IP address. In this lab, the tool is set up on a public cloud ECS. Therefore, the IP address is an EIP.

Initial user: **admin**

Initial password: **admin123**

Change the initial password upon the first login.

The following figure shows the login page.



4.3.2 Creating Data Sources on DataKit

Before creating a migration task, you need to create the MySQL and openGauss data sources on the **Instance management** page of DataKit.

After the MySQL and openGauss databases are started, you can choose **Resource Center** > **Instance management** on DataKit to create data sources.

Step 1 Create a MySQL data source.

Specify the username and password of the MySQL user created in 4.2.2 MySQL Installation and Configuration.

IP address: Enter the actual IP address.

port: The default port number is 3306.

Username: Enter **mysql_test**.

password: Enter **password123**.

Note: In this lab, openGauss and MySQL are deployed on the same ECS, and the IP addresses are the same private IP address starting with 192.168.

Step 2 Create an openGauss data source.

Specify the name and password of the openGauss user created in 4.2.4 openGauss Settings.

IP address: Enter the actual IP address.

port: Use port 26000 specified by this lab.

Username: Enter **opengauss_test**.

password: Enter **Sample@123**.

4.3.3 Adding a Server on DataKit

Before creating a migration task, you need to add a server on DataKit for installing the migration plug-in.

On DataKit, choose **Resource Center** > **Server management** to add a server. Migration plug-ins can be installed only on managed servers.

Step 1 Add a server.

Note that the password must be that of user **root** of the server and make sure **Remember your password** is selected. Otherwise, the server cannot be used as the executor of the migration plug-in.

Because this lab is set up on a public cloud ECS, set **Private IP address** to the private IP address starting with 192.168 and **Public IP address** to the EIP.

Step 2 Add an individual user to the server.

On the **User management** page, add individual user **omm**, which is used to set up the database and install plug-ins.

4.3.4 Presetting Data in the MySQL Database

Preset some data in the MySQL database before you start migration.

This lab will preset tables, data, indexes, views, functions, stored procedures, and triggers in MySQL to prepare for the migration.

Step 1 Connect to the MySQL database.

```
[[omm@ecs-840f ~]$ mysql -P 3306 -u mysql_test -ppassword123 -S
/opt/datakit/tools/mysql5.7/mysql.sock
```

Step 2 Preset data.

Run the following SQL statements:

```
drop database if exists source_db;
create database source_db;
use source_db;
```

```
create table table1(id int, name varchar(10), col varchar(20), primary key(id));
insert into table1 values(1,'data', 'data1');
insert into table1 values(2,'data', 'data2');
insert into table1 values(3,'data', 'data3');
insert into table1 values(4,'data', 'data4');
insert into table1 values(5,'data', 'data5');
insert into table1 values(6,'data', 'data6');

create view view1 as select * from table1;

create function mysql_func1(s char(20)) returns char(50) deterministic return concat('mysql_func1,
',s,'!');

create trigger trigger1 before insert on table1 for each row set new.col = concat(new.name, new.id);

create procedure procedure1() select * from table1;
```

4.3.5 Creating a Migration Task

The MySQL migration process involves five phases: full migration, full verification, incremental migration, incremental verification, and reverse migration. The following describes the migration process in detail.

In the migration task center of DataKit, create a migration task, select the source database (**source_db**) and target database (**target_db**), create a subtask, and set the migration mode to **Online mode**.

There are two migration modes: offline and online. The offline mode includes two phases: full migration and full verification. The online mode includes five phases: full migration, full verification, incremental migration, incremental verification, and reverse migration. For this lab, we will be using the online mode to practice the complete service migration process.

The details are as follows.

Step 1 Create a migration task.

In the migration task center of DataKit, create a migration task.

Step 2 Select the source and target databases.

Select the source database (**source_db**) and target database (**target_db**), add a subtask, and set the migration process mode to **Online mode**.

Step 3 Edit configuration parameters.

You can retain the default settings or modify them as required.

4.3.6 Installing Migration Plug-ins

When creating a migration task for the first time, you need to install the migration plug-ins. For a server added on the **Server management** page, plug-ins can be installed with one-click.

If no physical machine is available, add the machine where the plug-ins are to be installed to the **Server management** page.

Plug-ins can be installed in offline, online, or import mode. In the recommended offline installation mode, you need to download the plug-in installation package in advance and upload it.

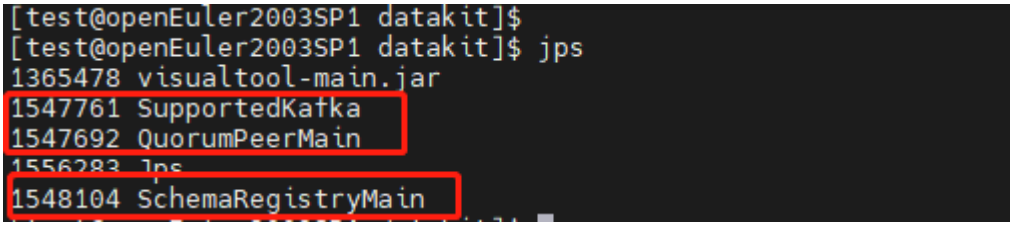
Download the installation package of the migration plug-in that works with your OS from:

https://opengauss.obs.cn-south-1.myhuaweicloud.com/5.1.0/tools/openEuler20.03/PortalControl-5.1.0-aarch64.tar.gz

- For online installation, specify the installation user (individual user added to the server managed under **Server management**) and installation directory.
- For offline installation, specify the installation user (individual user added to the server managed under **Server management**) and installation directory, and upload the installation package.
- For import installation, specify the installation user (individual user added to the server managed under **Server management**) and installation directory.

After the migration plug-in is successfully installed, a message is displayed indicating that the plug-ins have been installed.

Once the migration plug-ins have been installed successfully, you can use the **jps** command to view three processes that are essential for incremental migration and data verification: zookeeper, kafka, and schema-registry. The three processes start automatically upon successful plug-in installation.



4.3.7 Starting a Migration Task

For online migration, a migration task starts with full migration where the preset MySQL data is migrated to the target database. Then, in the full verification phase, the verification result of each table is generated. Finally, incremental migration and incremental verification are performed at the same time. On the details page, you can view the migration progress.

The details are as follows.

Step 1 Start a migration task.

On the details page, you can view the details about the migration process.

Step 2 Start full migration.

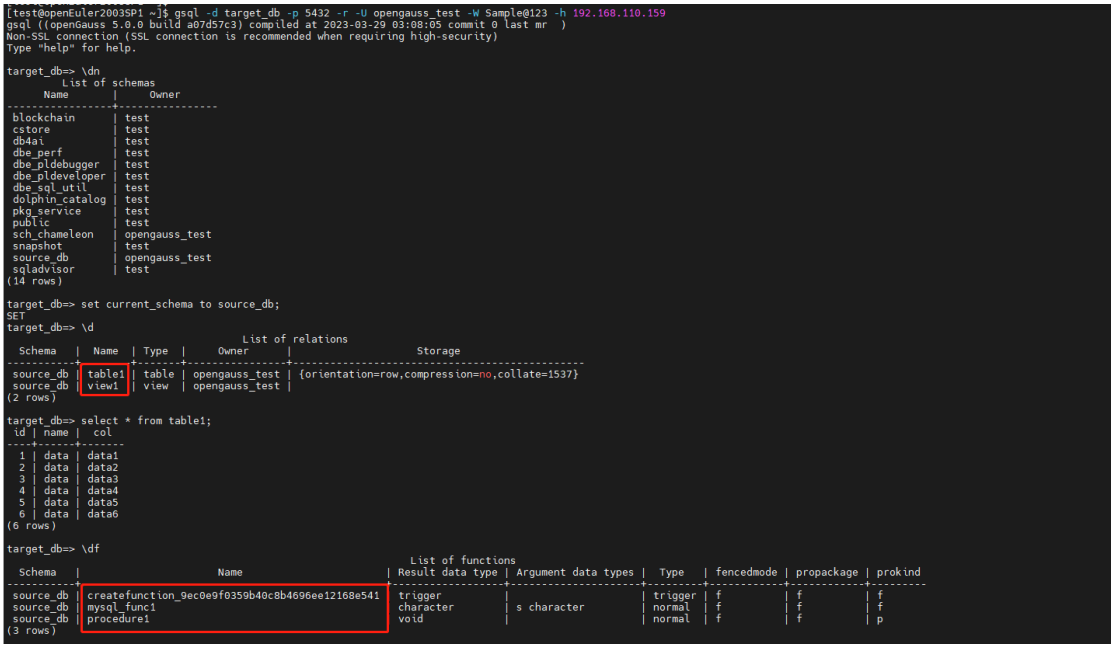
A migration task starts with full migration, where tables, data, views, functions, triggers, and stored procedures are migrated to the openGauss database. After the full migration is complete, the migration status of each object is displayed on the DataKit page. In addition, on openGauss, you can check whether data and objects are properly migrated.

Check the openGauss verification result.

Connect to the database, switch to the schema with the same name as MySQL, and check whether tables, data, views, functions, stored procedures, and triggers are migrated properly.

```
[omm@ecs-840f ~]$ gsql -d target_db -p 26000 -r -U opengauss_test -W Sample@123
```

The following figures show that the full migration is successful.




```
target_db=> \d+ table1
                                Table "source_db.table1"
 Column |         Type          |         Modifiers          | Storage  | Stats target | Description
--------+-----------------------+----------------------------+----------+--------------+-------------
 id     | integer               | not null                   | plain    |              |
 name   | character varying(10) | collate utf8mb4_general_ci | extended |              |
 col    | character varying(20) | collate utf8mb4_general_ci | extended |              |
Indexes:
    "pk_table1_1693277797_0" PRIMARY KEY, btree (id) TABLESPACE pg_default
Triggers:
    trigger1 BEFORE INSERT ON table1 FOR EACH ROW EXECUTE PROCEDURE createfunction_9ec0e9f0359b40c8b4696ee12168e541()
Replica Identity: FULL
Has OIDs: no
Options: orientation=row, compression=no, collate=1537
```

Step 3 Start incremental migration.

Start incremental migration after full verification is complete.

During incremental migration, both incremental data and DDL statements will be migrated to the openGauss database. At the same time, incremental verification is performed alongside the incremental migration process.

Add, delete, or modify data during incremental migration.

- Adding data

Add a data record during MySQL incremental migration.

```
[omm@ecs-840f ~]$ mysql -P 3306 -u mysql_test -ppassword123 -S
/opt/datakit/tools/mysql5.7/mysql.sock
mysql> use source_db;
mysql> select * from table1;
```

```
mysql> insert into table1(id,name) values(7,'data');
```

```
mysql> select * from table1;
+----+------+-------+
| id | name | col   |
+----+------+-------+
|  1 | data | data1 |
|  2 | data | data2 |
|  3 | data | data3 |
|  4 | data | data4 |
|  5 | data | data5 |
|  6 | data | data6 |
+----+------+-------+
6 rows in set (0.01 sec)

mysql> insert into table1(id,name) values(7,'data');
Query OK, 1 row affected (0.00 sec)
```

Data is properly synchronized to openGauss.

```
[omm@ecs-840f ~]$ gsql -d target_db -p 26000 -r -U opengauss_test -W Sample@123
target_db=> set current_schema to source_db;
SET
target_db=> select * from table1;
```

```
target_db=>  set current_schema to source_db;
SET
target_db=> select * from table1;
 id | name |  col
----+------+-------
  1 | data | data1
  2 | data | data2
  3 | data | data3
  4 | data | data4
  5 | data | data5
  6 | data | data6
  7 | data | data7
(7 rows)
```

- Modifying data

Modify a data record during MySQL incremental migration.

```
update table1 set name = 'new_data' where id = 6;
```

```
mysql> update table1 set name = 'new_data' where id = 6;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Data is properly synchronized to openGauss.

```
target_db=> select * from table1;
 id |   name    |  col
----+-----------+-------
  1 | data      | data1
  2 | data      | data2
  3 | data      | data3
  4 | data      | data4
  5 | data      | data5
  7 | data      | data7
  6 | new_data  | data6
(7 rows)
```

- Deleting data

Delete a data record during MySQL incremental migration.

```
delete from table1 where id = 5;
```

```
mysql> delete from table1 where id = 5;
Query OK, 1 row affected (0.01 sec)
```

Data is properly synchronized to openGauss.

```
target_db=> select * from table1;
 id |   name    |  col
----+-----------+-------
  1 | data      | data1
  2 | data      | data2
  3 | data      | data3
  4 | data      | data4
  7 | data      | data7
  6 | new_data  | data6
(6 rows)
```

- DDL operation

Execute a DDL statement during MySQL incremental migration.

```
create table table2(id int, name varchar(10));
```

```
mysql> create table table2(id int, name varchar(10));
Query OK, 0 rows affected (0.03 sec)
```

Data is properly synchronized to openGauss.

```
target_db=> \d
                        List of relations
  Schema   |  Name  | Type  |     Owner      |                    Storage
-----------+--------+-------+----------------+-----------------------------------------------
 source_db | table1 | table | opengauss_test | {orientation=row,compression=no,collate=1537}
 source_db | table2 | table | opengauss_test | {orientation=row,compression=no}
 source_db | view1  | view  | opengauss_test |
(3 rows)

target_db=> \d+ table2
                        Table "source_db.table2"
 Column |         Type          | Modifiers | Storage  | Stats target | Description
--------+-----------------------+-----------+----------+--------------+-------------
 id     | integer               |           | plain    |              |
 name   | character varying(10) |           | extended |              |
Has OIDs: no
Options: orientation=row, compression=no
```

The preceding provides some incremental migration examples to simulate the migration of incremental service data.

4.4 DataKit Data Migration Summary

The MySQL migration process consists of five phases, each requiring a different tool: full migration, full verification, incremental migration, incremental verification, and reverse migration. DataKit provides a user-friendly interface to visualize the entire migration process and enable simple, complete, and efficient migration in all scenarios. The migration portal is integrated with a migration and verification toolkit, making it easy to install, start, stop, and maintain tools, and to orchestrate migration tasks. Table 4-1 lists the available migration tools.

Table 4-1 Migration tools

Tool Usage	Tool Name	Code Repository	Remarks
Full migration	gs_mysync	https://gitee.com/opengauss/openGauss-tools-chameleon	Standalone tool
Incremental migration	gs_replicate	https://gitee.com/opengauss/debezium	Standalone tool
Reverse migration	gs_replicate	https://gitee.com/opengauss/debezium	Standalone tool
Data verification	gs_datacheck	https://gitee.com/opengauss/openGauss-tools-datachecker-performance	Standalone tool
Migration portal	gs_rep_portal	https://gitee.com/opengauss/openGauss-migration-portal	Migration and verification toolkit
Migration platform	DataKit	https://gitee.com/opengauss/openGauss-workbench	UI-based integrated migration portal

Upon completing this DataKit data migration lab, you should have:

- Learned how to deploy DataKit.
- Become familiar with the entire data migration process.
- Learned how to use DataKit to migrate services.

This lab aims to increase your interest in data migration and improve your understanding of the process. Additionally, DataKit offers capabilities beyond data migration, including installation and deployment, data development, and intelligent O&M. We encourage you to continue exploring and becoming familiar with these other capabilities.

5 Database Security Management

5.1 Introduction

5.1.1 About This Lab

This lab focuses on database security operations, including SSL connection configuration, user and role permission management, row-level security, security audit, encrypted equality query, and ledger database.

5.1.2 Objectives

By completing this lab, you will:

- Learn how to configure SSL connections, including generating client and server certificates.
- Learn how to grant and revoke user and role permissions, and properly configure permissions.
- Learn how to configure row-level security policies and related parameters.
- Learn how to view and delete database audit logs and manage unified audit.
- Learn how to perform encrypted equality queries.
- Gain familiarity with ledger database operations, including tamper-proof tables, ledger data consistency verification, and archiving and repairing the ledger database.

5.2 SSL Connection Configuration

openGauss supports the standard SSL (TLS 1.2) protocol, which ensures secure data transmission by authenticating bidirectional identification between the server and client using digital signatures and certificates.

To use SSL connections, you must apply for a formal server certificate, client certificate, and key from the CA in advance.

When a user remotely accesses the primary node of the database, they are required to use the SHA-256 authentication method.

For internal inter-server connections, the trust authentication mode is used, and IP address whitelist authentication is supported.

5.2.1 Setting Up the CA Environment

Step 1 Assume that user **omm** exists and the CA path is **/root/test**.

```
[root@ecs-5a5b ~]# mkdir /root/test
[root@ecs-5a5b ~]# cd /etc/pki/tls
[root@ecs-5a5b ~]# cp openssl.cnf ~/test
[root@ecs-5a5b ~]# cd ~/test
[root@ecs-5a5b test]# mkdir ./demoCA ./demoCA/newcerts ./demoCA/private
[root@ecs-5a5b test]# chmod 777 ./demoCA/private
```

Step 2 Create file **serial** and write **01** to it.

```
[root@ecs-5a5b test]# echo '01'>./demoCA/serial
```

Step 3 Create file **index.txt**.

```
[root@ecs-5a5b test]# touch ./demoCA/index.txt
```

Step 4 Modify parameters in the **openssl.cnf** configuration file.

```
[root@ecs-5a5b test]# vi ~/test/openssl.cnf
----------------------------------------
dir = ./demoCA
default_md = sha256
```

Step 5 Modify environment variables.

```
[root@ecs-5a5b test]# vi /etc/profile
```

Enter **i** to enter the INSERT mode and append the following line to the file. Press **Esc** to exit the INSERT mode. Then, enter **:wq** and press **Enter** to save the settings and exit.

```
export LD_LIBRARY_PATH=/usr/lib64
```

Run the following command to make the settings take effect:

```
[root@ecs-98c8 test]#    source /etc/profile
```

Remarks: OpenSSL cannot find **libcrypto.so.1.1** under the default path **/usr/lib64** because the bottom-line environment variable in the **/etc/profile** file has been modified during database installation. Because **libcrypto.so.1.1** is not located in **/usr/lib64**, the following error message is displayed: "openssl: relocation error: openssl: symbol EVP_md2 version OPENSSL_1_1_0 not defined in file libcrypto.so.1.1 with link time reference."

To view the actual **libcrypto.so.1.1** path, perform the following steps:

a. Run the **type openssl** command to check the path of the OpenSSL execution file.

```
[root@ecs-5a5b ~]# type openssl
openssl is /usr/bin/openssl
```

b. Run the **ls -l /usr/bin/openssl** command and view the result.

```
[root@ecs-5a5b ~]# ls -l /usr/bin/openssl
-rwxr-xr-x 1 root root 753104 Apr  2  2021 /usr/bin/openssl
```

Note that **/usr/bin/openssl** retained in this step is that returned in step a.

c. Run the **ldd /usr/bin/openssl** command. (If this indicates a soft link, use the original file instead.)

```
[root@ecs-5a5b ~]# ldd /usr/bin/openssl
        linux-vdso.so.1 (0x0000fffd35f60000)
        libssl.so.1.1 => /opt/huawei/install/om/script/gspylib/clib/libssl.so.1.1 (0x0000fffd35de0000)
        libcrypto.so.1.1 => /opt/huawei/install/om/script/gspylib/clib/libcrypto.so.1.1 (0x0000fffd35b80000)
        libz.so.1 => /lib64/libz.so.1 (0x0000fffd35b30000)
        libdl.so.2 => /lib64/libdl.so.2 (0x0000fffd35b00000)
        libpthread.so.0 => /lib64/libpthread.so.0 (0x0000fffd35ac0000)
        libc.so.6 => /lib64/libc.so.6 (0x0000fffd35940000)
        /lib/ld-linux-aarch64.so.1 (0x0000fffd35f70000)
```

Check the file that **libcrypto.so.1.1** points to. Because the default path **/usr/lib64** has been changed, the environment variable must be changed accordingly. If you run only the **export** command, the change is temporary and becomes invalid once you close the window or restart the server. Therefore, you need to add the change to the end of the **/etc/profile** file.

5.2.2 Creating a CA-Signed Certificate

Step 1 Generate a CA private key. **openssl genrsa** generates an RSA private key, and **-out flename** specifies the output file.

```
[root@ecs-5a5b test]# openssl genrsa -aes256 -out demoCA/private/cakey.pem 2048

Generating RSA private key, 2048 bit long modulus (2 primes)
.............+++++
.....................................................+++++
e is 65537 (0x010001)
Enter pass phrase for demoCA/private/cakey.pem: # Enter a password, for example, Test@123.
Verifying - Enter pass phrase for demoCA/private/cakey.pem: # Enter the password again, for example, Test@123.
```

Step 2 Generate a root CA certificate request file.

```
[root@ecs-5a5b test]# openssl req -config openssl.cnf -new -key demoCA/private/cakey.pem -out demoCA/careq.pem
Remarks: When specifying CA information, set Common Name to the host name.
Enter pass phrase for demoCA/private/cakey.pem: # Enter the password of the root private key, Test@123.
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:CN
State or Province Name (full name) [Some-State]:shanxi
Locality Name (eg, city) []:xian
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Abc
Organizational Unit Name (eg, section) []:hello
Common Name (e.g. server FQDN or YOUR name) []:ecs-5a5b   # Enter the host name.
Email Address []:swea425@a.com

Please enter the following 'extra' attributes
to be sent with your certificate request
```

```
A challenge password []:Test@123
An optional company name []:xian
```

Step 3 Generate a CA-signed root certificate.

```
[root@ecs-5a5b test]# vi openssl.cnf
---------------------------
basicConstraints=CA:TRUE ##Multiple constraints need to be changed.
---------------------------
[root@ecs-5a5b test]# openssl ca -config openssl.cnf -out demoCA/cacert.pem -keyfile
demoCA/private/cakey.pem -selfsign -infiles demoCA/careq.pem

Using configuration from openssl.cnf
Enter pass phrase for demoCA/private/cakey.pem: # Enter the password of the root private key,
Test@123.
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 1 (0x1)
        Validity
            Not Before: Nov  6 02:20:13 2023 GMT
            Not After : Nov  5 02:20:13 2024 GMT
        Subject:
            countryName               = CN
            stateOrProvinceName       = shanxi
            organizationName          = Abc
            organizationalUnitName    = hello
            commonName                = ecs-5a5b
            emailAddress              = swea425@a.com
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:TRUE
            Netscape Comment:
                OpenSSL Generated Certificate
            X509v3 Subject Key Identifier:
                69:D8:80:CC:6E:18:5F:97:50:ED:03:9C:2B:70:44:A1:4E:1D:62:72
            X509v3 Authority Key Identifier:
                keyid:69:D8:80:CC:6E:18:5F:97:50:ED:03:9C:2B:70:44:A1:4E:1D:62:72

Certificate is to be certified until Nov  5 02:20:13 2024 GMT (365 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
-- A root CA certificate named demoCA/cacert.pem has been signed.
```

5.2.3 SSL Connection Configuration (Creating a Server Certificate)

Step 1 Generate the server private key file **server.key**.

```
[root@ecs-5a5b test]# openssl genrsa -aes256 -out server.key 2048
```

```
Generating RSA private key, 2048 bit long modulus (2 primes)
.....................+++++
.....................+++++
e is 65537 (0x010001)
Enter pass phrase for server.key: # Enter the password of the server private key, for example, Test@123.
Verifying - Enter pass phrase for server.key: # Enter the password of the server private key again, for example, Test@123.
```

Step 2 Generate the server certificate request file **server.req**.

```
[root@ecs-5a5b test]# openssl req -config openssl.cnf -new -key server.key -out server.req ## The entered information must be consistent with that used for creating the CA certificate.
Enter pass phrase for server.key: # Enter the root private key password, Test@123.
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:CN
State or Province Name (full name) [Some-State]:shanxi
Locality Name (eg, city) []:xian
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Abc
Organizational Unit Name (eg, section) []:hello
Common Name (e.g. server FQDN or YOUR name) []:ecs-5a5b
Email Address []:swea425@a.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:Test@123
An optional company name []:xian
```

Step 3 Generate the server certificate **server.crt**.

```
[root@ecs-5a5b test]# vi openssl.cnf
---------------------
basicConstraints=CA:FALSE    ##Multiple constraints need to be changed.
---------------------
[root@ecs-5a5b test]# vi demoCA/index.txt.attr
---------------------
unique_subject = no
---------------------
[root@ecs-5a5b test]# openssl ca -config openssl.cnf -in server.req -out server.crt -days 3650 -md sha256

Using configuration from openssl.cnf
Enter pass phrase for ./demoCA/private/cakey.pem: # Enter the password of the root private key, Test@123.
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 2 (0x2)
```

```
        Validity
            Not Before: Nov  6 02:50:03 2023 GMT
            Not After : Nov  3 02:50:03 2033 GMT
        Subject:
            countryName                 = CN
            stateOrProvinceName         = shanxi
            organizationName            = Abc
            organizationalUnitName      = hello
            commonName                  = ecs-5a5b
            emailAddress                = swea425@a.com
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            Netscape Comment:
                OpenSSL Generated Certificate
            X509v3 Subject Key Identifier:
                7C:1C:02:F0:3F:BF:AF:DC:A0:03:49:44:D7:F8:95:B5:FF:3D:DF:B2
            X509v3 Authority Key Identifier:
                keyid:69:D8:80:CC:6E:18:5F:97:50:ED:03:9C:2B:70:44:A1:4E:1D:62:72

Certificate is to be certified until Nov  3 02:50:03 2033 GMT (3650 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
```

Step 4 Remove password protection on the private key.

```
[root@ecs-5a5b test]# openssl rsa -in server.key -out server.key

Enter pass phrase for client.key: # Enter the root private key password, Test@123.
writing RSA key
-- If the password protection of the server private key is not removed, you need to use gs_guc to
encrypt the password.
gs_guc encrypt -M server -K Test@123 -D ./ ## Two private key password protection files
(server.key.cipher and server.key.rand) are generated after gs_guc encryption.
```

5.2.4 SSL Connection Configuration (Creating a Client Certificate)

Step 1 Generate a client private key.

```
[root@ecs-5a5b test]# openssl genrsa -aes256 -out client.key 2048

Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
Enter pass phrase for client.key: # Enter the root private key password, Test@123.
Verifying - Enter pass phrase for client.key: # Enter the root private key password, Test@123.
```

Step 2 Generate a client certificate request file.

```
[root@ecs-5a5b test]# openssl req -config openssl.cnf -new -key client.key -out client.req

Enter pass phrase for server.key: Enter the root private key password, Test@123.
writing RSA key
[root@ecs-5a5b test]# openssl genrsa -aes256 -out client.key 2048
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
Enter pass phrase for client.key:
Verifying - Enter pass phrase for client.key:
[root@ecs-5a5b test]# openssl req -config openssl.cnf -new -key client.key -out client.req
Enter pass phrase for client.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:CN
State or Province Name (full name) [Some-State]:shanxi
Locality Name (eg, city) []:xian
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Abc
Organizational Unit Name (eg, section) []:hello
Common Name (e.g. server FQDN or YOUR name) []:ecs-5a5b
Email Address []:swea425@a.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:Test@123
An optional company name []:xian
```

Step 3 Generate the client certificate file **client.crt**.

```
[root@ecs-5a5b test]# openssl ca -config openssl.cnf -in client.req -out client.crt -days 3650 -md
sha256

Using configuration from openssl.cnf
Enter pass phrase for ./demoCA/private/cakey.pem: # Enter the password of the root private key,
Test@123.
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 3 (0x3)
        Validity
            Not Before: Nov  6 03:03:03 2023 GMT
            Not After : Nov  3 03:03:03 2033 GMT
        Subject:
            countryName               = CN
            stateOrProvinceName       = shanxi
            organizationName          = Abc
            organizationalUnitName    = hello
            commonName                = ecs-5a5b
            emailAddress              = swea425@a.com
```

```
            X509v3 extensions:
                X509v3 Basic Constraints:
                    CA:FALSE
                Netscape Comment:
                    OpenSSL Generated Certificate
                X509v3 Subject Key Identifier:
                    40:00:F2:1C:CB:74:CE:4B:57:C1:D9:08:B7:DD:02:59:DD:E0:16:01
                X509v3 Authority Key Identifier:
                    keyid:69:D8:80:CC:6E:18:5F:97:50:ED:03:9C:2B:70:44:A1:4E:1D:62:72

Certificate is to be certified until Nov  3 03:03:03 2033 GMT (3650 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
```

Step 4 Remove password protection on the private key.

```
[[root@ecs-5a5b test]# openssl rsa -in client.key -out client.key

Enter pass phrase for client.key: # Enter the root private key password, Test@123.
writing RSA key
-- If password protection for a client private key is not removed, you need to use gs_guc to encrypt the password.
gs_guc encrypt -M client -K Test@123 -D ./ ## Two private key password protection files (client.key.cipher and client.key.rand) are generated after gs_guc encryption.
```

5.2.5 SSL Connection Configuration (Configuring the Client Certificate)

Step 1 Copy the certificate to the sslcert client and modify the permission.

```
[root@db1 ~]# cp -r ~/test/* /opt/software/openGauss/libcgroup/share/sslcert/om
[root@db1 ~]# chmod 0600 /opt/software/openGauss/libcgroup/share/sslcert/om/client.crt
[root@db1 ~]# chmod 0600 /opt/software/openGauss/libcgroup/share/sslcert/om/client.key
[root@db1 ~]# chmod 0600 /opt/software/openGauss/libcgroup/share/sslcert/om/demoCA/ca*
[omm@db1 ~]$ cd /opt/software/openGauss/libcgroup/share/sslcert/om
[root@db1 om]# cp demoCA/cacert.pem .
[root@db1 ~]# chown omm:dbgrp -R /opt/software/openGauss/libcgroup/share/sslcert/om
[root@db1 om]$ ll
total 88
-rw------- 1 omm dbgrp 4391 Aug 26 14:34 cacert.pem
-rw------- 1 omm dbgrp 1062 Aug 26 14:34 careq.pem
-rw-r--r-- 1 omm dbgrp 4394 Aug 26 14:33 client.crt -rw-r--r-- 1 omm dbgrp 1679 Aug 26 14:33 client.key
-rw-r--r-- 1 omm dbgrp 1218 Aug 26 14:33 client.key.pk8
-rw-r--r-- 1 omm dbgrp 1066 Aug 26 14:33 client.req
drwxr-xr-x 4 omm dbgrp 4096 Aug 26 14:33 demoCA
-rw------- 1 omm dbgrp 286 Aug 26 14:34 index.txt -rw------- 1 omm dbgrp 20 Aug 26 14:34 index.txt.attr
-rw------- 1 omm dbgrp 20 Aug 26 14:34 index.txt.attr.old
```

```
-rw------- 1 omm dbgrp 213 Aug 26 14:34 index.txt.old
-rw-r--r-- 1 omm dbgrp 10920 Aug 26 14:33 openssl.cnf -rw------- 1 omm dbgrp 3 Aug 26 14:34
serial
-rw------- 1 omm dbgrp 3 Aug 26 14:34 serial.old
-rw-r--r-- 1 omm dbgrp 4388 Aug 26 14:33 server.crt -rw-r--r-- 1 omm dbgrp 1675 Aug 26 14:33
server.key
-rw-r--r-- 1 omm dbgrp 1062 Aug 26 14:33 server.req
```

Step 2 Copy the CA certificate and server certificate to the **$PG_DATA** directory.

```
[root@db1 om]# cp server.* /opt/huawei/install/data/dn
[root@db1 om]# cp ca* /opt/huawei/install/data/dn
[root@db1 om]# chown omm:dbgrp -R /opt/huawei/install/data/dn/server.*
[root@db1 om]# chown omm:dbgrp -R /opt/huawei/install/data/dn/ca*
[root@db1 om]# chmod 0600 /opt/huawei/install/data/dn/server.*
[root@db1 om]# chmod 0600 /opt/huawei/install/data/dn/ca*
```

Step 3 Configure the client certificate and test the connection.

```
[root@ecs-98c8 om]# chown -R omm:dbgrp /opt
[root@ecs-98c8 om]# su - omm
# Log in to the database as user omm, create user jack, and test the remote connection as user jack.
Note that the initial user omm cannot remotely access the openGauss database.
openGauss=# create jack password 'Test@123';
# Exit the database and perform the following settings:
[omm@client ~]$ mkdir ~/.ssl
[omm@ecs-5a5b ~]$ cd /opt/software/openGauss/libcgroup/share/sslcert/om
[omm@client ~]$ mv client.* cacert.pem ~/.ssl/ ## Copy the client and CA certificates to the client
directory.
[omm@client ~]$ vi ~/.bash_profile
export SSLCERT_HOME=/home/omm/.ssl
export PGSSLCERT=$SSLCERT_HOME/client.crt
export PGSSLKEY=$SSLCERT_HOME/client.key
export PGSSLMODE="verify-ca" export PGSSLROOTCERT=$SSLCERT_HOME/cacert.pem
[omm@client ~]$ source ~/.bash_profile
##### Append the line "host all all 0.0.0.0/0 sha256" to the pg_hba.conf file.
[omm@ecs-98c8 ~]$ vi /opt/huawei/install/data/dn/pg_hba.conf
# IPv4 local connections:
host    all             all             127.0.0.1/32            trust
host    all    all    192.168.0.208/32    sha256
host  all  all    0.0.0.0/0  sha256
# IPv6 local connections:
[omm@ecs-98c8 ~]$ gs_om -t stop && gs_om -t start # Restart the database.
[omm@ecs-98c8 ~]$gsql -d postgres -p 26000 -U jack -W Test@123 -h 192.168.0.208
gsql ((openGauss 5.0.0 build 0bd0ce80) compiled at 2020-06-30 18:19:27 commit 0 last mr )
SSL connection (cipher: DHE-RSA-AES256-GCM-SHA384, bits: 256)
Type "help" for help.
tpcc=>
```

5.3 Permission Management

SELECT: allows **SELECT** on a specified table, view, or sequence. The SELECT permission on the corresponding column is also required for **UPDATE** or **DELETE**.

INSERT: allows **INSERT** on a specified table.

UPDATE: allows **UPDATE** on any column of a specified table. Generally, **UPDATE** also requires the SELECT permission to query which rows need to be updated. SELECT... FOR UPDATE, SELECT... FOR NO KEY UPDATE, SELECT... FOR

SHARE and **SELECT... FOR KEY SHARE** require the UPDATE permission in addition to the SELECT permission.

DELETE: allows **DELETE** on a row of a specified table. Generally, **DELETE** also requires the SELECT permission to query which rows need to be deleted.

TRUNCATE: allows **TRUNCATE** on a specified table.

REFERENCES: allows creation of a FOREIGN KEY constraint. This permission is required on both referencing and referenced tables.

CREATE:

- For a database, new schemas can be created in the database.
- For schemas, new objects can be created in schemas. To rename an existing object, you must own the object and have the CREATE permission on the schema of the object.
- For tablespaces, it allows tables to be created within the tablespace, and allows databases and schemas to be created that have the tablespace as their default tablespace.

CONNECT: allows you to access specified databases.

EXECUTE: allows the use of specified functions as well as the operators that are implemented by these functions.

USAGE:

- For procedural languages, USAGE allows the use of the language for the creation of functions in that language.
- For schemas, USAGE allows access to objects contained in the schema. Without this permission, it is still possible to see the object names.
- For sequences, USAGE allows the use of the nextval function.
- For data sources, USAGE specifies access permissions or is used as ALL PRIVILEGES.

ALTER: allows you to modify the attributes of a specified object, excluding the owner and schema of the object.

DROP: allows you to delete a specified object.

COMMENT: allows you to define or modify comments of a specified object.

INDEX: allows you to create indexes on specified tables, manage indexes on the tables, and perform REINDEX and CLUSTER operations on the tables.

VACUUM: allows you to perform ANALYZE and VACUUM operations on specified tables.

ALL PRIVILEGES: grants all available permissions to specified users or roles at a time. Only a system administrator has the GRANT ALL PRIVILEGES permission.

5.3.1 User Permission Management

Step 1 Grant system permissions to a user or role.

Create user **joe** and grant it the sysadmin permissions.

```
openGauss=# CREATE USER joe PASSWORD 'openGauss@123456';
CREATE ROLE
openGauss=# GRANT ALL PRIVILEGES TO joe;
ALTER ROLE
```

5.3.2 Granting Object Permissions to Users or Roles

Step 1 Create schema **test** and table **customer**.

```
openGauss=# CREATE SCHEMA test;
CREATE SCHEMA
openGauss=# CREATE TABLE customer
(
c_customer_sk integer,
c_customer_id char(5),
c_first_name char(6),
c_last_name char(8),
Amount integer
);
CREATE TABLE
```

Step 2 Revoke user **joe**'s sysadmin permission, and grant it the usage permission on the **test** schema and all permissions on the **customer** table.

After the permissions are successfully granted, user **joe** has all permissions on the **customer** table, including addition, deletion, modification, and query permissions.

```
openGauss=# REVOKE ALL PRIVILEGES FROM joe;
ALTER ROLE
openGauss=# GRANT USAGE ON SCHEMA test TO joe;
GRANT
openGauss=# GRANT ALL PRIVILEGES ON customer TO joe;
GRANT
```

Step 3 Grant user **joe** the query permission on the **c_customer_sk**, **c_customer_id**, and **c_first_name** columns and the update permission on the **c_last_name** column in the **customer** table.

```
openGauss=# GRANT select (c_customer_sk,c_customer_id,c_first_name),update (c_last_name) ON
customer TO joe;
GRANT
```

After the permissions are successfully granted, user **joe** immediately has the query permission on the **c_customer_sk**, **c_customer_id**, **c_first_name** columns in the **customer** table.

Step 4 Allow user **joe** to grant these permissions to other users.

```
openGauss=# GRANT select (c_customer_sk,c_customer_id,c_first_name) ON customer TO joe WITH
GRANT OPTION;
```

```
GRANT
```

Step 5 Grant user **joe** the **postgres** database connection permission and schema creation permission and allow user **joe** to grant these permissions to other users.

```
openGauss=# GRANT create,connect on database postgres TO joe WITH GRANT OPTION;
GRANT
```

Step 6 Create the **test_manager** role, grant the access and object creation permissions on the **test** schema to the **test_manager** role, but do not allow this role to grant these permissions to others.

```
openGauss=# CREATE ROLE test_manager PASSWORD 'openGauss@123456';
CREATE ROLE
openGauss=# GRANT USAGE,CREATE ON SCHEMA test TO test_manager;
GRANT
```

Step 7 Create the **tpcds_tbspc** tablespace and grant user **joe** full permissions on this tablespace, but do not allow user **joe** to grant these permissions to other users.

```
openGauss=# CREATE TABLESPACE tpcds_tbspc RELATIVE LOCATION 'tablespace/tablespace_1';
CREATE TABLESPACE
openGauss=# GRANT ALL ON TABLESPACE tpcds_tbspc TO joe;
GRANT
```

5.3.3 Granting the Permissions of One User or Role to Others

Step 1 Create the **manager** role, grant **joe**'s permissions to the **manager** role, and allow the **manager** role to grant these permissions to others.

```
openGauss=# CREATE ROLE manager PASSWORD 'Test321##';
CREATE ROLE
openGauss=# GRANT joe TO manager WITH ADMIN OPTION;
GRANT ROLE
```

Step 2 Create user **senior_manager** and grant the permissions of role **manager** to the user.

```
openGauss=# CREATE ROLE senior_manager PASSWORD 'Test321##';
CREATE ROLE
openGauss=# GRANT manager TO senior_manager;
GRANT ROLE
```

5.4 Row-level Security

The row-level security feature enables database access control to be accurate to each row of data tables. In this way, the same SQL query may return different results for different users.

You can create a row-level security policy for a data table. The policy defines an expression that takes effect only for specific database users and SQL operations. When a

database user accesses the data table, if an SQL statement meets the specified row-level security policy of the data table, the expressions that meet the specified condition will be combined by using AND or OR based on the attribute type (**PERMISSIVE** or **RESTRICTIVE**) and applied to the execution plan in the query optimization phase.

Row-level security is used to control the visibility of row-level table data. By predefining filters for data tables, the expressions that meet the specified condition can be applied to execution plans in the query optimization phase, which will affect the final execution result. Currently, the SQL statements that can be affected include SELECT, UPDATE, and DELETE.

Step 1 Create users **alice**, **bob**, and **peter**.

```
openGauss=# CREATE USER alice PASSWORD 'openGauss@123456';
openGauss=# CREATE USER bob PASSWORD 'openGauss@123456';
openGauss=# CREATE USER peter PASSWORD 'openGauss@123456';
```

Step 2 Create the **all_data** table that contains user information.

```
openGauss=# CREATE TABLE all_data(id int, role varchar(100), data varchar(100));
```

Step 3 Insert data into the data table.

```
openGauss=# INSERT INTO all_data VALUES(1, 'alice', 'alice data');
openGauss=# INSERT INTO all_data VALUES(2, 'bob', 'bob data');
openGauss=# INSERT INTO all_data VALUES(3, 'peter', 'peter data');
```

Step 4 Grant the read permission for the **all_data** table to users **alice**, **bob**, and **peter**.

```
openGauss=# GRANT SELECT ON all_data TO alice, bob, peter;
```

Step 5 Enable row-level security.

```
openGauss=# ALTER TABLE all_data ENABLE ROW LEVEL SECURITY;
```

Step 6 Create a row-level security policy to specify that the current user can view only their own data.

```
openGauss=# CREATE ROW LEVEL SECURITY POLICY all_data_rls ON all_data USING(role =
CURRENT_USER);
```

Step 7 View table details.

```
openGauss=# \d+ all_data
Table "public.all_data"
Column | Type | Modifiers | Storage | Stats target | Description
--------+------------------------+-----------+----------+--------------+------------
id | integer | | plain | |
role | character varying(100) | | extended | |
data | character varying(100) | | extended | |
Row Level Security Policies:
```

```
Remarks:
PG_STATISTIC and PG_STATISTIC_EXT store sensitive information about statistical objects, such as high-frequency MCVs.
If the query permission on the two system catalogs is granted to a common user after row-level security is created, the common user can still access the two system catalogs.
Two system catalogs are used to obtain the information in the statistical object.
POLICY "all_data_rls"
USING (((role)::name = "current_user"()))
Has OIDs: no
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, enable_rowsecurity=true
```

Step 8 Switch to user **alice** and run **SELECT * FROM public.all_data**.

```
openGauss=# \c postgres alice ###Switch to user alice.
Password for user alice:
You are now connected to database "postgres" as user "alice".
openGauss=> SELECT * FROM public.all_data;
id | role | data
----+-------+------------
1 | alice | alice data
(1 row)

openGauss=> EXPLAIN(COSTS OFF) SELECT * FROM public.all_data;
QUERY PLAN
------------------------------------------------------------------
Streaming (type: GATHER)
Node/s: All datanodes
-> Seq Scan on all_data
Filter: ((role)::name = 'alice'::name)
Notice: This query is influenced by row level security feature
(5 rows)
```

Step 9 Switch to user **bob** and run **SELECT * FROM public.all_data**.

```
openGauss=> \c postgres bob ###Switch to user bob.
Password for user bob:
You are now connected to database "postgres" as user "bob".
openGauss=> SELECT * FROM public.all_data;
id | role | data
----+-------+------------
2 | bob   | bob data
(1 row)
openGauss=> EXPLAIN(COSTS OFF) SELECT * FROM public.all_data;
QUERY PLAN
------------------------------------------------------------------
Streaming (type: GATHER)
Node/s: All datanodes
-> Seq Scan on all_data
Filter: ((role)::name = 'peter'::name)
Notice: This query is influenced by row level security feature
(5 rows)
```

5.5 Database Security Audit

The audit mechanism involves managing audit logs, tracing database management and service events, and conducting audits.

Audit log management includes creating, rotating, and clearing audit logs.

Audit log tracing involves recording events and using an API to query audit information.

5.5.1 Database Security Audit

Step 1 Enable database audit.

```
[omm@Run861 dn]$ gs_guc set -N all -I all -c "audit_enabled=on"
The gs_guc run with the following arguments: [gs_guc -N all -I all -c audit_enabled=on set ].
Begin to perform the total nodes: 3.
Popen count is 3, Popen success count is 3, Popen failure count is 0.
Begin to perform gs_guc for datanodes.
Command count is 3, Command success count is 3, Command failure count is 0.
Total instances: 3. Failed instances: 0.
ALL: Success to perform gs_guc!

[omm@Run861 dn]$ gs_guc reload -N all -I all -c "audit_system_object=12295"
The gs_guc run with the following arguments: [gs_guc -N all -I all -c audit_system_object=12295
reload ].
Begin to perform the total nodes: 3.
Popen count is 3, Popen success count is 3, Popen failure count is 0.
Begin to perform gs_guc for datanodes.
Command count is 3, Command success count is 3, Command failure count is 0.
Total instances: 3. Failed instances: 0.
ALL: Success to perform gs_guc!
```

Step 2 View audit-related parameters.

```
openGauss=# select name,setting,short_desc from pg_settings where name like '%audit%';

name                  |                  setting                  |  short_desc
----------------------------+-----------------------------------------------------------+----------------------------
------------------------------------ audit_copy_exec                 | 1
| audit copy execution.
 audit_data_format             | binary                                    | Sets the data format for audit
files.
 audit_database_process        | 1                                         | audit database start, stop,
recover and switchover.
 audit_directory               | /var/log/omm/omm/pg_audit/dn_6001 | Sets the destination directory
for audit files.
 audit_dml_state               | 0                                         | audit DML operation.
 audit_dml_state_select        | 0                                        | audit DML select operation.
 audit_enabled                 | on                                        | Starts a subprocess to capture
audit output into audit files.
 audit_file_remain_threshold | 1048576                                   | audit file remain threshold.
 audit_file_remain_time        | 90                                        | the days of the audit files can
be remained
 audit_function_exec           | 0                                         | audit function execution.
```

```
 audit_grant_revoke              | 1                                                    | audit grant and revoke
privilege.
 audit_login_logout              | 7                                                    | audit user login logout.
 audit_resource_policy           | on                                                   | the policy is used to determine
how to cleanup the audit files; True means to cleanup the audit files based on space limitation and
False me
ans to cleanup the audit files when the remained time is arriving.
 audit_rotation_interval         | 1440                                                 | Automatic audit file rotation will
occur after N minutes.
 audit_rotation_size             | 10240                                                | Automatic audit file rotation
will occur after N kilobytes.
 audit_set_parameter             | 0                                                    | audit set operation.
 audit_space_limit               | 1048576                                              | audit data space limit in MB
unit
 audit_system_function_exec      | 0                                                    | Sets whether audit system
function execution or not.
 audit_system_object             | 12295                                                | audit DDL operation on
system object.
 audit_thread_num                | 1                                                    | Sets the number of audit
threads.
 audit_user_locked               | 1                                                    | audit lock and unlock user.
 audit_user_violation            | 0                                                    | audit user violation.
 audit_xid_info                  | 0                                                    | whether record xid info in audit
log.
 full_audit_users                |                                                      | Sets the users under
comprehensive audit.
 no_audit_client                 |                                                      | Sets the appname@ip should
not be audited.
```

Step 3 Query audit information.

```
openGauss=# SELECT time,type,result,username,database,object_name,detail_info FROM
pg_query_audit('2023-09-30 09:00:00','2023-09-30 10:00:00');
             time              |       type        | result | username |      database       |     object_name
|                                  detail_info
-------------------------------+-------------------+--------+----------+---------------------+---------------------
-----------------------------------------------
 2023-09-30 09:00:05+08 | login_success | ok     | omm      | postgres            | postgres            |
login db(postgres) success,the current user is:omm, SSL=off
 2023-09-30 09:00:05+08 | user_logout   | ok     | omm      | postgres            | postgres            |
session timeout, logout db(postgres) success
 2023-09-30 09:00:05+08 | login_success | ok     | omm      | postgres            | postgres            |
login db(postgres) success,the current user is:omm, SSL=on
 2023-09-30 09:00:05+08 | user_logout   | ok     | omm      | postgres            | postgres            |
logout db(postgres) success
 2023-09-30 09:00:05+08 | login_success | ok     | omm      | mydb                | mydb
| login db(mydb) success,the current user is:omm, SSL=on
 2023-09-30 09:00:05+08 | user_logout   | ok     | omm      | mydb                | mydb
| logout db(mydb) success
 2023-09-30 09:00:05+08 | login_success | ok     | omm      | test                | test                |
login db(test) success,the current user is:omm, SSL=on
 2023-09-30 09:00:05+08 | user_logout   | ok     | omm      | test                | test                |
logout db(test) success
 2023-09-30 09:00:05+08 | login_success | ok     | omm      | oid_01              | oid_01              |
login db(oid_01) success,the current user is:omm, SSL=on
```

```
 2023-09-30 09:00:05+08 | user_logout     | ok      | omm          | oid_01              | oid_01
| logout db(oid_01) success
 2023-09-30 09:00:05+08 | login_success | ok      | omm          | newdata             | newdata
| login db(newdata) success,the current user is:omm, SSL=on
 2023-09-30 09:00:05+08 | user_logout     | ok      | omm          | newdata             | newdata
| logout db(newdata) success
 2023-09-30 09:00:05+08 | login_success | ok      | omm          | mm                  | mm
| login db(mm) success,the current user is:omm, SSL=on
 2023-09-30 09:00:05+08 | user_logout     | ok      | omm          | mm                  | mm
| logout db(mm) success
 2023-09-30 09:00:05+08 | login_success | ok      | omm          | kk                  | kk                  |
login db(kk) success,the current user is:omm, SSL=on
 2023-09-30 09:00:05+08 | user_logout     | ok      | omm          | kk                  | kk
| logout db(kk) success
 2023-09-30 09:00:05+08 | login_success | ok      | omm          | mysql2opengauss | mysql2opengauss
| login db(mysql2opengauss) success,the current user is:omm, SSL=on
 2023-09-30 09:00:05+08 | user_logout     | ok      | omm          | mysql2opengauss |
mysql2opengauss | logout db(mysql2opengauss) success
 2023-09-30 09:00:05+08 | login_success | ok      | omm          | postgres            | postgres            |
login db(postgres) success,the current user is:omm, SSL=on
 2023-09-30 09:00:05+08 | user_logout     | ok      | omm          | postgres            | postgres            |
logout db(postgres) success
 2023-09-30 09:00:06+08 | login_success | ok      | omm          | smart_data          | smart_data          |
login db(smart_data) success,the current user is:omm, SSL=on
 2023-09-30 09:00:06+08 | user_logout     | ok      | omm          | smart_data          | smart_data
| logout db(smart_data) success
 2023-09-30 09:00:06+08 | login_success | ok      | omm          | test_restore        | test_restore        |
login db(test_restore) success,the current user is:omm, SSL=on
```

Step 4 Delete audit logs.

```
openGauss=# SELECT pg_delete_audit('2023-10-30 01:00:00','2023-10-30 02:00:00');
```

5.5.2 Unified Database Audit

The unified audit mechanism enables customers to customize policies for generating audit logs.

The unified audit mechanism customizes audit behaviors based on resource labels and classifies supported audit behaviors into the access and privileges classes.

Currently, unified audit supports the following audit behaviors:

- DDL: ALL ALTER ANALYZE COMMENT CREATE DROP GRANT REVOKE SET SHOW
- DML: ALL COPY DEALLOCATE DELETE_P EXECUTE REINDEX INSERT REPARE SELECT TRUNCA

Step 1 Enable database audit.

```
[omm@Run861 ~]$ gs_guc reload -I all -N all -c "enable_security_policy=1"
The gs_guc run with the following arguments: [gs_guc -I all -N all -c enable_security_policy=1
reload ].
Begin to perform the total nodes: 3.
Popen count is 3, Popen success count is 3, Popen failure count is 0.
Begin to perform gs_guc for datanodes.
```

```
Command count is 3, Command success count is 3, Command failure count is 0.

Total instances: 3. Failed instances: 0.
ALL: Success to perform gs_guc!
```

Step 2 Create resource labels.

```
openGauss=# CREATE TABLE t6 (
ID bigint NOT NULL,
Name varchar (255) NOT NULL,
Age (int NOT NULL),
Address varchar (255) DEFAULT NULL
) ;

openGauss=# grant select,update,delete on t6 to alice;
GRANT

openGauss=#    CREATE RESOURCE LABEL adt_lb0 add TABLE(t6);
CREATE RESOURCE LABEL
```

Step 3 Audit user **alice**'s ALTER operation on the **adt_lb0** resource.

```
openGauss=# CREATE AUDIT POLICY adt1 PRIVILEGES ALTER ON LABEL(adt_lb0) FILTER ON
ROLES(alice);
CREATE AUDIT POLICY
```

Step 4 Audit user **alice** only using gsql as the client tool. Then, perform SELECT, INSERT, and DELETE operations on the **adt_lb0** resource.

```
openGauss=# CREATE AUDIT POLICY adt2 ACCESS SELECT ON LABEL(adt_lb0),INSERT ON
LABEL(adt_lb0), DELETE ON LABEL(adt_lb0) FILTER ON ROLES(alice),APP(gsql);
CREATE AUDIT POLICY
```

5.6 Encrypted Equality Query

The encrypted equality query solution utilizes deterministic encryption methods, which generates identical ciphertext for identical plaintext data to support equality calculations.

The command for creating a client master key (CMK) is:

CREATE CLIENT MASTER KEY client_master_key_name WITH (KEY_STORE = key_store_name, KEY_PATH = "key_path_value", ALGORITHM = algorithm_type)

- **client_master_key_name**: specifies the key name. In the same namespace, the value of this parameter must be unique. Value range: a string that complies with the identifier naming convention.
- **KEY_STORE**: specifies the key tool or component that manages CMKs. Currently, only **localkms** is supported.
- **KEY_PATH**: uniquely identifies a CMK in **KEY_STORE**. **KEY_STORE** manages multiple CMKs. Value range: Refer to that of **key_path_value**.

- **ALGORITHM**: specifies the type of the encryption algorithm used to encrypt column encryption keys (CEKs). Value range: **RSA_2048**, **RSA_3072**, and **SM2**.

The command for creating a CEK is:

CREATE COLUMN ENCRYPTION KEY column_encryption_key_name WITH VALUES(CLIENT_MASTER_KEY = client_master_key_name, ALGORITHM = algorithm_type, ENCRYPTED_VALUE = encrypted_value);

- **column_encryption_key_name**: specifies the key name. In the same namespace, the value of this parameter must be unique. Value range: a string that complies with the identifier naming convention.
- **CLIENT_MASTER_KEY**: specifies the CMK used to encrypt the CEK. The value is the CMK object name, which is created using the CREATE CLIENT MASTER KEY syntax.
- **ALGORITHM**: specifies the encryption algorithm to be used by the CEK. Value range: **AEAD_AES_256_CBC_HMAC_SHA256**, **AEAD_AES_128_CBC_HMAC_SHA256**, and **SM4_SM3**.
- **ENCRYPTED_VALUE** (optional): specifies the key password. The key password length ranges from 28 to 256 bits. The derived 28-bit key meets the AES128 security requirements. If the user needs to use AES256, the key password length must be 39 bits. If the user does not specify the key password length, a key of 256 characters long is automatically generated.
- Remarks: SM2, SM3, and SM4 are Chinese cryptographic algorithms. To avoid legal risks, these algorithms must be used together. If you specify the SM2 algorithm to encrypt CEKs when creating a CMK, you must specify the SM3 and SM4 algorithms (SM4_SM3) to encrypt data when creating CEKs.

Step 1 Create the *$GAUSSHOME*/etc/localkms/ directory.

```
[omm@ecs-b168 ~]$ mkdir -p $GAUSSHOME/etc/localkms/
[omm@ecs-b168 ~]$
```

This step prepares for creating the **KEY_PATH** path in the CMK.

Step 2 Log in to the **postgres** database as user **omm** and enable the encrypted equality query function.

```
[omm@ecs-b168 ~]$   gsql -d postgres -p 26000 -r -C
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=#
```

The **-C** parameter is the switch of the encrypted equality query function.

Step 3 Create a CMK and a CEK.

```
openGauss=# CREATE CLIENT MASTER KEY ImgCMK01 WITH (KEY_STORE = localkms, KEY_PATH =
"key_path_value1", ALGORITHM = RSA_2048);
CREATE CLIENT MASTER KEY
```

```
openGauss=# CREATE CLIENT MASTER KEY ImgCMK02 WITH (KEY_STORE = localkms, KEY_PATH =
"key_path_value2", ALGORITHM = RSA_2048);
CREATE CLIENT MASTER KEY
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK01 WITH VALUES (CLIENT_MASTER_KEY =
ImgCMK01, ALGORITHM  = AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK02 WITH VALUES (CLIENT_MASTER_KEY =
ImgCMK02, ALGORITHM  = AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
```

Step 4 Create an encrypted card information table and an encrypted card balance table.

```
openGauss=# CREATE TABLE creditcard_info01 (id_number int, name  text encrypted with
(column_encryption_key = ImgCEK02, encryption_type = DETERMINISTIC),credit_card varchar(19)
encrypted with (column_encryption_key = ImgCEK01, encryption_type = DETERMINISTIC));
CREATE TABLE

openGauss=# CREATE TABLE creditcard_bal01 (credit_card varchar(19) encrypted with
(column_encryption_key = ImgCEK01, encryption_type = DETERMINISTIC),card_bal int);
CREATE TABLE
```

The **encryption_type** parameter is described as follows:

encryption_type: selects the encryption mode, either **DETERMINISTIC** or **RANDOMIZED**.

- **DETERMINISTIC**: always generates the same encrypted value for any given plain text value. The use of deterministic encryption supports comparisons, groupings, joins, and queries on encrypted values, but also allows unauthorized users to guess information about encrypted values by checking patterns in encrypted columns. Two tables can be joined on deterministically encrypted columns only if both columns are encrypted using the same CEK. Deterministic encryption must sort character columns in binary order.
- **RANDOMIZED**: encrypts data in a less predictable manner. Randomized encryption is more secure and prevents queries, groupings, and joins on encrypted columns. Columns that use randomized encryption cannot be indexed.

Step 5 Query an encrypted table.

```
openGauss=# \d creditcard_info01
          Table "public.creditcard_info01"
    Column      |           Type            | Modifiers
-------------+-------------------+------------
 id_number   | integer                |
 name           | text                    |  encrypted
 credit_card | character varying |  encrypted
```

The value of **Modifiers** is **encrypted**, indicating that the column is encrypted.

Step 6 Insert data.

```
openGauss=# INSERT INTO creditcard_info01 VALUES (1,'joe','6217986500001288393');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info01 VALUES (2, 'joy','6219985678349800033');
INSERT 0 1
```

```
openGauss=# INSERT INTO creditcard_info01 VALUES (3, 'Adam','6219985678349804034');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info01 VALUES (4,'Bob','6217986500001243512');
INSERT 0 1

openGauss=# INSERT INTO creditcard_bal01 VALUES ('6217986500001288393',100);
INSERT 0 1
openGauss=# INSERT INTO creditcard_bal01 VALUES('6219985678349800033',700);
INSERT 0 1
openGauss=# INSERT INTO creditcard_bal01 VALUES('6219985678349804034',900);
INSERT 0 1
openGauss=# INSERT INTO creditcard_bal01 VALUES('6217986500001243512',400);
INSERT 0 1
```

Step 7 Query data.

```
openGauss=# select * from creditcard_info01 where name = 'joe';
 id_number | name |        credit_card
-----------+------+--------------------
           1 | joe  | 6217986500001288393
(1 row)
```

Use a non-encrypted client for query.

```
openGauss=# select id_number,name from creditcard_info01 where name = 'joe';
 id_number |                                                
```

```
openGauss=# select * from creditcard_info01 where id_number like'4%';
 id_number | name |        credit_card
-----------+------+--------------------
           4 | Bob    | 6217986500001243512
(1 row)
```

The preceding result indicates that fuzzy query is not supported for encrypted columns, but is supported for non-encrypted columns.

```
openGauss=# select * from creditcard_info01 where credit_card > '6219985678349800033';
ERROR(CLIENT): operator is not allowed on datatype of this column
openGauss=# select * from creditcard_info01 where id_number > 1;
 id_number | name |        credit_card
-----------+------+--------------------
           2 | joy    | 6219985678349800033
           3 | Adam | 6219985678349804034
           4 | Bob    | 6217986500001243512
(3 rows)
```

The preceding result indicates that range query is not supported for encrypted columns, but is supported for non-encrypted columns.

In summary, except equality query, encrypted queries, such as sorting, range query, and fuzzy query, are not supported.

Step 9 Perform a join query.

Query the username and balance of the **joy** user.

```
openGauss=# select a.name,b. card_bal from creditcard_info01 a inner join creditcard_bal01 b on a.
credit_card = b. credit_card and a.name = 'joy';
name | card_bal
------+----------
 joy   |         700
(1 row)
```

Query the total balance of the **joy** and **joe** users:

```
openGauss=# select sum(b. card_bal) from creditcard_info01 a inner join creditcard_bal01 b on a.
credit_card = b. credit_card and a.name in ('joy', 'joe');
sum
-----
 800
(1 row)
```

According to the preceding query result, two tables can be joined on encrypted columns for encrypted equality query.

Step 10 Create a randomized encrypted table and insert data into it.

Create a randomized encrypted table **creditcard_info02**. The table structure and inserted data are the same as those of **creditcard_info01**. The only difference is that the column encryption type is changed to **RANDOMIZED**.

```
openGauss=# CREATE TABLE creditcard_info02 (id_number int, name    text encrypted with
(column_encryption_key = ImgCEK02, encryption_type = RANDOMIZED),credit_card varchar(19)
encrypted with (column_encryption_key = ImgCEK01, encryption_type = RANDOMIZED));
CREATE TABLE
openGauss=# INSERT INTO creditcard_info02 VALUES (1,'joe','6217986500001288393');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info02 VALUES (2, 'joy','6219985678349800033');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info02 VALUES (3, 'Adam','6219985678349804034');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info02 VALUES (4,'Bob','6217986500001243512');
INSERT 0 1
```

Step 11 Query the randomized encrypted table.

```
openGauss=# select * from creditcard_info02;
 id_number | name |       credit_card
-----------+------+--------------------
         1 | joe    | 6217986500001288393
         2 | joy    | 6219985678349800033
         3 | Adam | 6219985678349804034
         4 | Bob    | 6217986500001243512
(4 rows)
openGauss=# select * from creditcard_info02 where id_number = 1;
 id_number | name |       credit_card
-----------+------+--------------------
         1 | joe    | 6217986500001288393
(1 row)
openGauss=# select * from creditcard_info02 where name = 'joe';
ERROR(CLIENT): operator is not allowed on datatype of this column
openGauss=# select * from creditcard_info02 order by credit_card;
ERROR(CLIENT): could not support order by operator for column encryption
openGauss=# select * from creditcard_info02 where name like'j%';
ERROR(CLIENT): operator is not allowed on datatype of this column
openGauss=# select a.name,b. card_bal from creditcard_info02 a inner join creditcard_bal01 b on a.
credit_card = b. credit_card and a.name = 'joy';
ERROR(CLIENT): equal operator is not allowed on datatype of this column
```

As shown in the preceding result, the encrypted column cannot be used as the WHERE or sorting condition, nor be used to join tables.

Step 12 Alter the encrypted table.

Add an encrypted column **age**.

```
openGauss=# ALTER TABLE creditcard_info01 ADD COLUMN age int ENCRYPTED WITH
(COLUMN_ENCRYPTION_KEY = ImgCEK01, ENCRYPTION_TYPE = DETERMINISTIC);
ALTER TABLE
openGauss=# \d creditcard_info01
          Table "public.creditcard_info01"
    Column      |          Type             | Modifiers
-------------+-------------------+------------
 id_number     | integer                |
 name            | text                     |   encrypted
```

```
 credit_card | character varying |    encrypted
 age         | integer           |    encrypted
```

Delete the encrypted column **age**.

```
openGauss=# ALTER TABLE creditcard_info01 DROP COLUMN age;
ALTER TABLE
openGauss=# \d creditcard_info01
          Table "public.creditcard_info01"
    Column      |          Type          | Modifiers
----------------+------------------------+-----------
 id_number      | integer                |
 name           | text                   |    encrypted
 credit_card | character varying |    encrypted
```

Step 13 Update the encrypted table.

```
openGauss=# update creditcard_info01 set credit_card = '80000000011111111' where name = 'joy';
UPDATE 1
openGauss=# select * from creditcard_info01    where name = 'joy';
 id_number | name |      credit_card
-----------+------+-------------------
         2 | joy  | 80000000011111111
(1 row)
```

5.7 Ledger Database

The ledger database records user operations on a tamper-proof table and verifies data consistency based on the operation records, ensuring that the user data cannot be maliciously tampered with.

5.7.1 Creating and Operating Tamper-proof User Tables

Tamper-proof tables do not support non-row-store tables, temporary tables, foreign tables, or unlog tables. Non-row-store tables do not have the tamper-proof attribute.

When a tamper-proof table is created, a system column named **hash** is automatically added. Therefore, the maximum number of columns in the tamper-proof table is 1599.

The attribute of a schema that contains a table cannot be changed to blockchain using the ALTER SCHEMA statement.

Step 1 Create tamper-proof schema **ledgernsp**.

```
openGauss=# CREATE SCHEMA ledgernsp WITH BLOCKCHAIN;
CREATE SCHEMA
```

Step 2 Create tamper-proof user table **ledgernsp.usertable**.

```
openGauss=# CREATE TABLE ledgernsp.usertable(id int, name text);
```

```
CREATE TABLE
```

Check the structure of the tamper-proof user table and the corresponding user history table.

```
openGauss=# \d+ ledgernsp.usertable;
                          Table "ledgernsp.usertable"
 Column |   Type    | Modifiers | Storage  | Stats target | Description
--------+-----------+-----------+----------+--------------+-------------
 id     | integer |           | plain    |              |
 name   | text    |           | extended |              |
 hash   | hash16  |           | plain    |              |
Has OIDs: no
Options: orientation=row, compression=no
History table name: ledgernsp_usertable_hist

openGauss=# \d+ blockchain.ledgernsp_usertable_hist;
                 Table "blockchain.ledgernsp_usertable_hist"
   Column   |   Type   | Modifiers | Storage | Stats target | Description
-----------+----------+-----------+---------+--------------+-------------
 rec_num   | bigint |           | plain   |              |
 hash_ins | hash16 |           | plain   |              |
 hash_del | hash16 |           | plain   |              |
 pre_hash | hash32 |           | plain   |              |
Indexes:
    "gs_hist_19063_index" PRIMARY KEY, btree (rec_num int4_ops) TABLESPACE pg_default
Has OIDs: no
Options: internal_mask=263
```

Step 3 Execute INSERT, UPDATE, and DELETE statements on the tamper-proof user table.

```
openGauss=# INSERT INTO ledgernsp.usertable VALUES(1, 'alex'), (2, 'bob'), (3, 'peter');
INSERT 0 3
openGauss=# SELECT *, hash FROM ledgernsp.usertable ORDER BY id;
 id | name  |          hash
----+-------+------------------
  1 | alex  | 1f2e543c580cb8c5
  2 | bob   | 8fcd74a8a6a4b484
  3 | peter | f51b4b1b12d0354b
(3 rows)

openGauss=# UPDATE ledgernsp.usertable SET name = 'bob2' WHERE id = 2;
UPDATE 1
openGauss=# SELECT *, hash FROM ledgernsp.usertable ORDER BY id;
 id | name  |          hash
----+-------+------------------
  1 | alex  | 1f2e543c580cb8c5
  2 | bob2  | 437761affbb7c605
  3 | peter | f51b4b1b12d0354b
(3 rows)

openGauss=# DELETE FROM ledgernsp.usertable WHERE id = 3;
DELETE 1
openGauss=# SELECT *, hash FROM ledgernsp.usertable ORDER BY id;
```

```
 id | name |          hash
----+------+------------------
  1 | alex | 1f2e543c580cb8c5
  2 | bob2 | 437761affbb7c605
(2 rows)
```

5.7.2 Viewing the Operation History of a Ledger Database

Only users with the AUDITADMIN attribute can view historical operation records in the ledger.

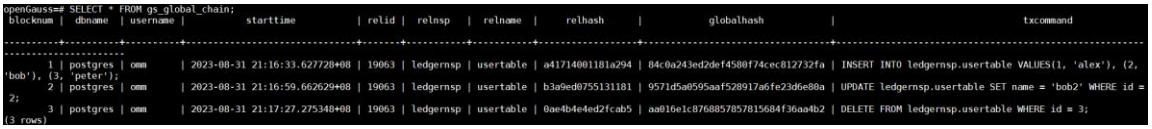
Generally, the name of a user history table is in the format of **blockchain.<*schemaname*>_<*tablename*>_hist**.

If the schema name or table name of the tamper-proof user table is too long, the length of the table name generated using the preceding format may exceed the upper limit. In this case, the **blockchain.<*schema_oid*>_<*table_oid*>_hist** format is used to name the table.

Step 1 Query the global blockchain table, **gs_global_chain**.

```
openGauss=# SELECT * FROM gs_global_chain;
```

Result:



The query result indicates that user **omm** has consecutively executed three DML commands, including INSERT, UPDATE, and DELETE.

Step 2 Query the user history table in blockchain mode.

```
openGauss=# SELECT * FROM blockchain.ledgernsp_usertable_hist;
```

Result:

```
openGauss=# SELECT * FROM blockchain.ledgernsp_usertable_hist;
 rec_num |     hash_ins     |     hash_del     |             pre_hash
---------+------------------+------------------+----------------------------------
       0 | 1f2e543c580cb8c5 |                  | e45acf22fe042b2373d148f52903d29a
       1 | 8fcd74a8a6a4b484 |                  | af08f23d38ecfec2ad9c6f1c4685a837
       2 | f51b4b1b12d0354b |                  | 69e2885fb802fbb2b191211623115f9d
       3 | 437761affbb7c605 | 8fcd74a8a6a4b484 | fd61cb772033da297d10c4e658e898d7
       4 |                  | f51b4b1b12d0354b | 6475a497b7a272a92bab012d7f3d615b
(5 rows)
```

The query result shows that user **omm** inserts three rows of data to the **ledgernsp.usertable** table, updates one row of data, deletes one row of data, and leaves two rows of data, and the hash values are **1f2e543c580cb8c5** and **437761affbb7c605**.

Step 3 Query user table data and hash verification columns.

```
openGauss=# SELECT *, hash FROM ledgernsp.usertable;
```

```
 id | name |          hash
----+------+------------------
  1 | alex | 1f2e543c580cb8c5
  2 | bob2 | 437761affbb7c605
(2 rows)
```

The query result indicates that the remaining two records in the user table are the same as those in the previous step.

5.7.3 Checking Data Consistency of a Ledger Database

The ledger database provides two verification APIs: ledger_hist_check(text, text) and ledger_gchain_check(text, text). When a common user calls a verification API, only the tables that the user has the permission to access can be verified.

- The pg_catalog.ledger_hist_check API is used to verify the tamper-proof user table and user history table. The API call returns **t** if the verification is passed, and **f** if it is not.
- The pg_catalog.ledger_gchain_check API is used to verify the consistency between the tamper-proof user table, user history table, and global blockchain table. The API call returns **t** if the verification is passed, and **f** if it is not.

Step 1 Check whether the tamper-proof user table **ledgernsp.usertable** is consistent with the corresponding user history table.

```
openGauss=# SELECT pg_catalog.ledger_hist_check('ledgernsp', 'usertable');
 ledger_hist_check
-------------------
 t
(1 row)
```

The query result shows that the results recorded in the tamper-proof user table and user history table are consistent.

Step 2 Check whether the records in the tamper-proof **ledgernsp.usertable** table are the same as those in the corresponding user history table and global blockchain table.

```
openGauss=# SELECT pg_catalog.ledger_gchain_check('ledgernsp', 'usertable');
 ledger_gchain_check
---------------------
 t
(1 row)
```

The query result shows that the records of **ledgernsp.usertable** in the preceding three tables are consistent and no tampering occurs.

5.7.4 Archiving and Repairing a Ledger Database

There are two APIs for verifying the ledger database archive result: ledger_hist_archive and ledger_gchain_archive.

Only audit administrators have the permission to call the ledger database APIs.

- pg_catalog.ledger_hist_archive is user to verify the archive of the user history table. The API call returns **t** if the archive is successful, and **f** if it is not. SELECT pg_catalog.ledger_hist_archive(schema_name text,table_name text);
- pg_catalog.ledger_gchain_archive is user to verify the archive of the global blockchain table. The API call returns **t** if the archive is successful, and **f** if it is not. SELECT pg_catalog.ledger_gchain_archive();

Step 1 Archive a user history table.

```
openGauss=# SELECT pg_catalog.ledger_hist_archive('ledgernsp', 'usertable');
 ledger_hist_archive
---------------------
 t
(1 row)
```

Step 2 Check the archived user history table.

```
openGauss=# SELECT * FROM blockchain.ledgernsp_usertable_hist;
 rec_num |       hash_ins       |       hash_del       |                pre_hash
---------+----------------------+----------------------+----------------------------------
       4 | e78e75b00d396899 | 84e8bfc3b974e9cf | 6475a497b7a272a92bab012d7f3d615b
(1 row)
```

The command output indicates that the user history table of the current node is exported successfully.

Step 3 Export the global blockchain table.

```
openGauss=# SELECT pg_catalog.ledger_gchain_archive();
 ledger_gchain_archive
-----------------------
 t
(1 row)
```

Step 4 The global history table will be archived to *n* (number of user tables) data records by user table:

```
openGauss=# SELECT * FROM gs_global_chain;
```

```
openGauss=# SELECT * FROM gs_global_chain;
 blocknum |  dbname  | username |          starttime          | relid |  relnsp   |  relname  |     relhash      |            globalhash            | txcommand
----------+----------+----------+-----------------------------+-------+-----------+-----------+------------------+----------------------------------+-----------
        2 | postgres | omm      | 2023-08-31 21:16:33.627728+08 | 19063 | ledgernsp | usertable | 62a5b5ec53c47eca | 82fa05fc07e6e793cac12ec95215b68e | Archived.
(1 row)
```

The command output indicates that the global blockchain table of the current node is successfully exported.

Step 5 Repair the historical table.

```
openGauss=# SELECT pg_catalog.ledger_hist_repair('ledgernsp', 'usertable');
 ledger_hist_repair
--------------------
 0000000000000000
(1 row)
```

Step 6 Repair the global blockchain table.

```
openGauss=# SELECT pg_catalog.ledger_gchain_repair('ledgernsp', 'usertable');
 ledger_gchain_repair
----------------------
 62a5b5ec53c47eca
(1 row)
```

The API for repairing the user history table is pg_catalog.ledger_hist_repair. If the repair is successful, the API call returns the hash increment of the user history table during the repair.

The API for archiving the global blockchain table is pg_catalog.ledger_gchain_repair. If the repair is successful, the API call returns the hash increment of the global blockchain table during the repair.

6 Comprehensive Lab

6.1 Introduction

6.1.1 About This Lab

The lab is designed using the financial data model of the HCIA comprehensive lab and includes important theoretical knowledge from the HCIP course. With this account, you can:

- Use information about Bank C to create a finance database and tables, and import instance data.
- Create a fund manager user who is not allowed to access the finance product and insurance tables. Additionally, the user cannot view any records related to finance products and insurance in the property table.
- Use SQL statements to gather statistics on the total return of all clients in finance products, insurance, and funds of the bank, and sort the clients in descending order of total return.
- Write a Java program to execute the previous SQL statements, print the returned result, and simulate concurrent database connections using multi-thread technology as the background load for subsequent steps.
- Use gsql to access information about user tables and indexes, active sessions, and SQL statements being executed in system catalogs and system views.
- Obtain a list of all check items for the network, device, OS, and database that can be checked by gs_check. Execute all database check items and fix any issues found during the check, such as unanalyzed data tables.
- Use gs_checkperf to check key performance indicators of the openGauss cluster.
- Use gs_collector to gather OS information, log information, and configuration files, and view the contents of each log.
- Pay close attention to resource parameters and set the shared buffer size of the database based on the memory size of the OS.
- After frequent data deletion and updates, maintain tables and indexes using the **VACUUM**, **REINDEX**, and **ANALYZE** commands.
- Generate a WDR based on the latest two snapshots and interpret the report.
- Add a column to the client table and set it as an encrypted column. Query data on the client to assign a value to the encrypted column and verify that the data in the column has been automatically encrypted on the server.

- Transform the property table to a tamper-proof table using the ledger database. Increase the transaction capacity for a specific client's transaction. Retrieve the complete revision history of the specific data by querying the system view.

6.1.2 Objectives

By participating in this application practice, you will gain a deeper understanding of the key knowledge and be able to apply it to actual production projects.

6.1.3 Model Overview

Assuming that Bank C in City A uses openGauss for managing and operating bank data, this lab classifies database objects into clients, bank cards, finance products, insurance, and funds. For these objects, the lab assumes that the finance database of Bank C supports the following service logic: a client can apply for a bank card, and the client can purchase different bank products such as finance products, funds, and insurance. The database consists of six tables:

- client (c_id, c_name, c_mail, c_id_card, c_phone, and c_password)
- bank_card (b_number, b_type, and b_c_id)
- finances_product (p_name, p_id, p_description, p_amount, and p_year)
- insurance (i_name, i_id, i_amount, i_person, i_year, and i_project)
- fund (f_name, f_id, f_type, f_amount, risk_level, and f_manager)
- property(pro_id, pro_c_id, pro_pif_id, pro_type, pro_status, pro_quantity, pro_income, pro_purchase_time)

The **pro_pif_id** column in the **property** table references the **id** columns in the **finances_product** table, **insurance** table, and **fund** table. To avoid conflicts between the **id** columns of the three products, add a **pro_type** column to differentiate the three products: **1** for a finance product, **2** for an insurance product, and **3** for a fund product.

6.2 Procedure

6.2.1 Creating Tables and Importing Data

Use information about Bank C to create a finance database and tables, and import instance data.

Step 1 Use gsql to log in to openGauss as user **omm**.

```
[omm@ecs-5a5b ~]$ gsql -d postgres -p 26000 -r
```

Step 2 Create database **finance**.

```
openGauss=# CREATE DATABASE finance ENCODING 'UTF8' template = template0;
```

Step 3 Create a database user.

```
openGauss=#CREATE USER bank_app IDENTIFIED BY 'Gauss#3demo';
openGauss=#GRANT ALL privileges to bank_app;
```

Step 4 Log in to the **finance** database as the **bank_app** user.

```
[omm@ecs-5a5b ~]$ gsql -d finance -U bank_app -W Gauss#3demo -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

finance=>
```

Step 5 Create tables.

```
finance=> CREATE TABLE client
(
        c_id INT PRIMARY KEY,
        c_name VARCHAR(100) NOT NULL,
        c_mail CHAR(30) UNIQUE,
        c_id_card CHAR(20) UNIQUE NOT NULL,
        c_phone CHAR(20) UNIQUE NOT NULL,
        c_password CHAR(20) NOT NULL
);
finance=> COMMENT ON TABLE client IS 'Client table';

finance=> CREATE TABLE bank_card
(
        b_number CHAR(30) PRIMARY KEY,
        b_type CHAR(20),
        b_c_id INT NOT NULL
);
finance=> COMMENT ON TABLE bank_card IS 'Bank card table';

finance=> CREATE TABLE finances_product
(
        p_name VARCHAR(100) NOT NULL,
        p_id INT PRIMARY KEY,
        p_description VARCHAR(4000),
        p_amount INT,
        p_year INT
);
finance=> COMMENT ON TABLE finances_product IS 'Finance product table';

finance=> CREATE TABLE insurance
(
        i_name VARCHAR(100) NOT NULL,
        i_id INT PRIMARY KEY,
        i_amount INT,
        i_person CHAR(20),
        i_year INT,
        i_project VARCHAR(200)
);
finance=> COMMENT ON TABLE insurance IS 'Insurance information table';
```

```
finance=> CREATE TABLE fund
(
        f_name VARCHAR(100) NOT NULL,
        f_id INT PRIMARY KEY,
        f_type CHAR(20),
        f_amount INT,
        risk_level CHAR(20) NOT NULL,
        f_manager INT NOT NULL
);
finance=> COMMENT ON TABLE fund IS 'Fund information table';

finance=> CREATE TABLE property
(
        pro_id INT PRIMARY KEY,
pro_c_id INT NOT NULL,
        pro_pif_id INT NOT NULL,
        pro_type INT NOT NULL,
        pro_status CHAR(20),
        pro_quantity INT,
        pro_income INT,
        pro_purchase_time DATE
);
finance=> COMMENT ON TABLE property IS 'Asset information table';
```

Step 6 Initialize table data.

```
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (1,'Zuo
Xiaoting','zuoxiaoting@huawei.com','340211199301010001','18815650001','gaussdb_001')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (2,'Yu
Chenggang','yuchenggang@huawei.com','340211199301010002','18815650002','gaussdb_002')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (3,'Zhu
Changgang','zhuchanggang@huawei.com','340211199301010003','18815650003','gaussdb_003')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (4,'Ren
Gaofeng','rengaofeng@huawei.com','340211199301010004','18815650004','gaussdb_004')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (5,'An
Yanfang','anyanfang@huawei.com','340211199301010005','18815650005','gaussdb_005')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (6,'Teng
Changli','tengchangli@huawei.com','340211199301010006','18815650006','gaussdb_006')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (7,'Fu
Xiaofang','fuxiaofang@huawei.com','340211199301010007','18815650007','gaussdb_007')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (8,'Bian
Lanjuan','bianlanjuan@huawei.com','340211199301010008','18815650008','gaussdb_008')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (9,'Shao
Xiaoting','shaoxiaoting@huawei.com','340211199301010009','18815650009','gaussdb_009')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (10,'Zhang
Xiaofeng','zhangxiaofeng@huawei.com','340211199301010010','18815650010','gaussdb_010')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (11,'Kang
Xiaopeng','kangxiaopeng@huawei.com','340211199301010011','18815650011','gaussdb_011')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (12,'Feng
Changqiang','fengchangqiang@huawei.com','340211199301010012','18815650012','gaussdb_012')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (13,'Ying
Xiaopeng','yingxiaopeng@huawei.com','340211199301010013','18815650013','gaussdb_013')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (14,'Zhao
Xiaopeng','zhaoxiaopeng@huawei.com','340211199301010014','18815650014','gaussdb_014')
```

```
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (15,'Su
Xiaogang','suxiaogang@huawei.com','340211199301010015','18815650015','gaussdb_015')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (16,'Yin
Gaofeng','yingaofeng@huawei.com','340211199301010016','18815650016','gaussdb_016')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (17,'Meng
Xiaoxuan','mengxiaoxuan@huawei.com','340211199301010017','18815650017','gaussdb_017')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (18,'Tang
Gaowei','tanggaowei@huawei.com','340211199301010018','18815650018','gaussdb_018')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (19,'Hong
Gaopeng','gaohongpeng@huawei.com','340211199301010019','18815650019','gaussdb_019')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (20,'Song
Chengfeng','songchengfeng@huawei.com','340211199301010020','18815650020','gaussdb_020')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (21,'Qiu
Changqiang','qiuchangqiang@huawei.com','340211199301010021','18815650021','gaussdb_021')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (22,'Wei
Gaopeng','weigaopeng@huawei.com','340211199301010022','18815650022','gaussdb_022')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (23,'Pan
Chengxuan','panchengxuan@huawei.com','340211199301010023','18815650023','gaussdb_023')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (24,'Ji
Xiaomei','jixiaomei@huawei.com','340211199301010024','18815650024','gaussdb_024')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (25,'Li
Chunting','lichunting@huawei.com','340211199301010025','18815650025','gaussdb_025')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (26,'Ni
Lanfang','nilanfang@huawei.com','340211199301010026','18815650026','gaussdb_026')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (27,'Jiang
Xiaofang','jiangxiaofang@huawei.com','340211199301010027','18815650027','gaussdb_027')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (28,'Li
Lanfang','lilanfang@huawei.com','340211199301010028','18815650028','gaussdb_028')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (29,'Lou
Xiaoting','louxiaoting@huawei.com','340211199301010029','18815650029','gaussdb_029')
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (30,'Fu
Xiaoting','fuxiaoting@huawei.com','340211199301010030','18815650030','gaussdb_030')

INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000001','credit card',1);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000002','credit card',3);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000003','credit card',5);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000004','credit card',7);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000005','credit card',9);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000006','credit card',10);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000007','credit card',12);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000008','credit card',14);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000009','credit card',16);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000010','credit card',18);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000011','debit card',19);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000012','debit card',21);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000013','debit card',7);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000014','debit card',23);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000015','debit card',24);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000016','debit card',3);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000017','debit card',26);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000018','debit card',27);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000019','debit card',12);
INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000020','debit card',29);
```

```
INSERT INTO finances_product(p_name, p_id, p_description, p_amount, p_year) VALUES
('Bonds',1,'Bank finance products that mainly invest in government bonds, financial bonds, central
bank bills, and enterprise bonds ',50000,6);
INSERT INTO finances_product(p_name, p_id, p_description, p_amount, p_year) VALUES ('credit
assets',2,'The assets generally refers to a situation where a bank entrusts a trust company to raise
funds by issuing finance products. The trust company then creates a trust plan and purchases finance
products, which are sold to the bank or third-party credit assets. The bank acts as the trustor and the
trust company acts as the trustee in this arrangement. ',50000,6);
INSERT INTO finances_product(p_name, p_id, p_description, p_amount, p_year) VALUES
('Stock',3,'Finance product linked to stocks, mainly Hong Kong stocks',50000,6);
INSERT INTO finances_product(p_name, p_id, p_description, p_amount, p_year) VALUES
('Commodity',4,'Finance products linked to commodity futures, mainly finance products such as gold,
oil, agricultural products ',50000,6);

INSERT INTO insurance(i_name, i_id, i_amount, i_person, i_year, i_project) VALUES ('Health
insurance',1,2000,'Elderly',30,'Ping An Insurance');
INSERT INTO insurance(i_name, i_id, i_amount, i_person, i_year, i_project) VALUES ('Life
insurance',2,3000,'Elderly',30,'Ping An Insurance');
INSERT INTO insurance(i_name, i_id, i_amount, i_person, i_year, i_project) VALUES ('Accident
insurance',3,5000,'All people',30,'Ping An Insurance');
INSERT INTO insurance(i_name, i_id, i_amount, i_person, i_year, i_project) VALUES ('Medical
insurance',4,2000,'All people',30,'Ping An Insurance');
INSERT INTO insurance(i_name, i_id, i_amount, i_person, i_year, i_project) VALUES ('Property loss
insurance',5,1500,'Middle-aged',30,'Ping An Insurance');

INSERT INTO fund(f_name, f_id, f_type, f_amount, risk_level, f_manager) VALUES
('Stock',1,'Stock',10000,'High',1);
INSERT INTO fund(f_name, f_id, f_type, f_amount, risk_level, f_manager) VALUES
('Investment',2,'Bond',10000,'Medium',2);
INSERT INTO fund(f_name, f_id, f_type, f_amount, risk_level, f_manager) VALUES ('Government
bond',3,'Currency',10000,'Low',3);
INSERT INTO fund(f_name, f_id, f_type, f_amount, risk_level, f_manager) VALUES ('CSI300
index',4,'Exponential',10000,'Medium',4);

INSERT INTO
property(pro_id,pro_c_id,pro_pif_id,pro_type,pro_status,pro_quantity,pro_income,pro_purchase_time)
VALUES (1,5,1,1,'Available',4,8000,'2018-07-01');
INSERT INTO
finance.property(pro_id,pro_c_id,pro_pif_id,pro_type,pro_status,pro_quantity,pro_income,pro_purchase
_time) VALUES (2,10,2,2,'Available',4,8000,'2018-07-01');
INSERT INTO
finance.property(pro_id,pro_c_id,pro_pif_id,pro_type,pro_status,pro_quantity,pro_income,pro_purchase
_time) VALUES (3,15,3,3,'Available',4,8000,'2018-07-01');
INSERT INTO
finance.property(pro_id,pro_c_id,pro_pif_id,pro_type,pro_status,pro_quantity,pro_income,pro_purchase
_time) VALUES (4,20,4,1,Frozen,4,8000,'2018-07-01');
```

6.2.2 Database Security Management

Create a fund manager user who is not allowed to access the finance product and insurance tables. Additionally, the user cannot view any records related to finance products and insurance in the **property** table.

Step 1 Use gsql to log in to openGauss as user **bank_app**.

```
[omm@ecs-5a5b ~]$ gsql -d finance -U bank_app -W Gauss#3demo -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

finance=>
```

Step 2 Create database user **fund_app**.

```
finance=> CREATE USER fund_app IDENTIFIED BY 'Gauss#3demo';
```

Step 3 Grant user **fund_app** only the query permission on the four tables.

```
finance=> GRANT select ON table bank_card to fund_app;
GRANT
finance=> GRANT select ON table client to fund_app;
GRANT
finance=> GRANT select ON table fund to fund_app;
GRANT
finance=> GRANT select ON table property to fund_app;
GRANT
```

Step 4 Use gsql to log in to openGauss as user **fund_app**.

```
[omm@ecs-5a5b ~]$ gsql -d finance -U fund_app -W Gauss#3demo -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

finance=>
```

Step 5 Query the **property** table as user **fund_app**.

```
finance=> select * from property;
 pro_id | pro_c_id | pro_pif_id | pro_type |       pro_status       | pro_quantity | pro_income |
pro_purchase_time
--------+----------+------------+----------+------------------------+--------------+------------+---------------------
       1 |        5 |          1 |        1 | Available              |            4 |       8000 | 2018-07-
01 00:00:00
       2 |       10 |          2 |        2 | Available              |            4 |       8000 | 2018-07-
01 00:00:00
       3 |       15 |          3 |        3 | Available              |            4 |       8000 | 2018-07-
01 00:00:00
       4 |       20 |          4 |        1 | Frozen                 |            4 |       8000 | 2018-
07-01 00:00:00
(4 rows)
```

Step 6 Use gsql to log in to openGauss as user **bank_app**.

```
[omm@ecs-5a5b ~]$ gsql -d finance -U bank_app -W Gauss#3demo -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.
```

```
finance=>
```

Step 7 Use row-level security to restrict **fund_app** to access only fund-related records in the **property** table, that is, **pro_type=3** records.

```
finance=>ALTER TABLE property ENABLE ROW LEVEL SECURITY;
finance=>CREATE ROW LEVEL SECURITY POLICY property_rls ON property
USING(current_user !='fund_app' or current_user ='fund_app' and pro_type =3);
```

Step 8 Use gsql to log in to openGauss as user **fund_app**.

```
[omm@ecs-5a5b ~]$ gsql -d finance -U fund_app -W Gauss#3demo -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

finance=>
```

Step 9 Query the **property** table as user **fund_app**.

```
finance=> select * from property;
 pro_id | pro_c_id | pro_pif_id | pro_type |       pro_status       | pro_quantity | pro_income |
pro_purchase_time
--------+----------+------------+----------+------------------------+--------------+------------+---------------------
       3 |       15 |          3 |        3 | Available              |            4 |       8000 | 2018-07-
01 00:00:00
(1 row)
```

6.2.3 Advanced SQL Syntax

Use SQL statements to gather statistics on the total return of all clients in finance products, insurance, and funds of the bank, and sort the clients in descending order of total return.

Step 1 Log in to the **finance** database as the **bank_app** user.

```
[omm@ecs-5a5b ~]$ gsql -d finance -U bank_app -W Gauss#3demo -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

finance=>
```

Step 2 Implement complex queries using right join and aggregate functions of SQL:

```
select c_id_card, c_name,sum(pro_income) total_income
from
client c,
bank_card b,
property p,
(select p_name name, p_id pro_pif_id, 1 pro_type from finances_product union all
```

```
select i_name name, i_id pro_pif_id, 2 pro_type from insurance union all
select f_name name, f_id pro_pif_id, 3 pro_type from fund) d
where
c.c_id = p.pro_c_id
and c.c_id = b.b_c_id(+)
and p.pro_pif_id=d.pro_pif_id
and p.pro_type=d.pro_type
group by c_id_card, c_name
order by 2 desc;

        c_id_card          | c_name | total_income
---------------------+--------+--------------
340211199301010015 | Su Xiaogang | 8000
340211199301010010 | Zhang Xiaofeng | 8000
340211199301010020 | Song Chengfeng | 8000
340211199301010005 | An Yanfang | 8000
(4 rows)
```

Step 3 Run the **EXPLAIN ANALYZE** command to view the execution plan of the SQL statement.

```
finance=> explain analyze select c_id_card, c_name,sum(pro_income) total_income
from
client c,
bank_card b,
property p,
(select p_name name, p_id pro_pif_id, 1 pro_type from finances_product union all
select i_name name, i_id pro_pif_id, 2 pro_type from insurance union all
select f_name name, f_id pro_pif_id, 3 pro_type from fund) d
where
c.c_id = p.pro_c_id
and c.c_id = b.b_c_id(+)
and p.pro_pif_id=d.pro_pif_id
and p.pro_type=d.pro_type
group by c_id_card, c_name
order by 2 desc;
                                                                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------
-------------------------------------
 GroupAggregate  (cost=78.23..78.37 rows=7 width=314) (actual time=0.290..0.292 rows=4 loops=1)
   Group By Key: c.c_name, c.c_id_card
   ->  Sort  (cost=78.23..78.25 rows=7 width=306) (actual time=0.284..0.284 rows=4 loops=1)
         Sort Key: c.c_name DESC, c.c_id_card
         Sort Method: quicksort  Memory: 25kB
         ->  Hash Right Join  (cost=63.56..78.13 rows=7 width=306) (actual time=0.163..0.262
rows=4 loops=1)
               Hash Cond: (b.b_c_id = c.c_id)
               ->  Seq Scan on bank_card b  (cost=0.00..13.38 rows=338 width=4) (actual
time=0.003..0.004 rows=20 loops=1)
               ->  Hash  (cost=63.51..63.51 rows=4 width=310) (actual time=0.113..0.113 rows=4
loops=1)
                     Buckets: 32768  Batches: 1  Memory Usage: 257kB
                     ->  Nested Loop  (cost=40.08..63.51 rows=4 width=310) (actual
time=0.074..0.110 rows=4 loops=1)
```

```
                                    ->  Hash Join  (cost=40.08..61.31 rows=6 width=8) (actual
time=0.064..0.069 rows=4 loops=1)
                                          Hash Cond: ((p.pro_pif_id = finances_product.p_id) AND
(p.pro_type = (1)))
                                          ->  Seq Scan on property p  (cost=0.00..15.59 rows=559
width=16) (actual time=0.006..0.006 rows=4 loops=1)
                                          ->  Hash  (cost=34.03..34.03 rows=403 width=8) (actual
time=0.021..0.021 rows=13 loops=1)
                                                Buckets: 32768  Batches: 1  Memory Usage: 257kB
                                                ->  Append  (cost=0.00..34.03 rows=403 width=8) (actual
time=0.004..0.012 rows=13 loops=1)
                                                      ->  Seq Scan on finances_product
(cost=0.00..11.05 rows=105 width=8) (actual time=0.004..0.006 rows=4 loops=1)
                                                      ->  Seq Scan on insurance  (cost=0.00..11.07
rows=107 width=8) (actual time=0.002..0.002 rows=5 loops=1)
                                                      ->  Seq Scan on fund  (cost=0.00..11.91 rows=191
width=8) (actual time=0.002..0.003 rows=4 loops=1)
                                    ->  Index Scan using client_pkey on client c  (cost=0.00..0.36 rows=1
width=306) (actual time=0.013..0.016 rows=4 loops=4)
                                          Index Cond: (c_id = p.pro_c_id)
 Total runtime: 0.441 ms
(23 rows)
```

6.2.4 Application Development

Write a Java program to execute the previous SQL statements, print the returned result, and simulate concurrent database connections using multi-thread technology as the background load for subsequent steps.

Create the **/home/omm/jdbc** directory.

```
[omm@ecs-5a5b ~]$ mkdir -p /home/omm/jdbc
[omm@ecs-5a5b ~]$ cd /home/omm/jdbc
```

Step 1 Use JDBC to write the following Java program and save it as **DBTest.java** in the **/home/omm/jdbc** directory.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class DBTest {

  // Create a database connection.
  public static Connection GetConnection(String username, String passwd) {
    String driver = "org.opengauss.Driver";
    String sourceURL = "jdbc:opengauss://localhost:26000/finance";
    Connection conn = null;
    try {
      // Load the database driver.
      Class.forName(driver).newInstance();
    } catch (Exception e) {
      e.printStackTrace();
```

```
            return null;
        }

        try {
            // Create a database connection.
            conn = DriverManager.getConnection(sourceURL, username, passwd);
            System.out.println("Connection succeed!");
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }

        return conn;
    };

    //Run the prepared statement to query data.
    public static void ExecPreparedSQL(Connection conn) {
        PreparedStatement pstmt = null;
        try {
            pstmt = conn
                    .prepareStatement("select c_id_card, c_name,sum(pro_income) total_income from client c,
bank_card b, property p, (select p_name name, p_id pro_pif_id, 1 pro_type from finances_product
union all select i_name name, i_id pro_pif_id, 2 pro_type from insurance union all select f_name
name, f_id pro_pif_id, 3 pro_type from fund) d where c.c_id = p.pro_c_id and c.c_id = b.b_c_id(+) and
p.pro_pif_id=d.pro_pif_id and p.pro_type=d.pro_type group by c_id_card, c_name order by 2 desc");
            ResultSet rs = pstmt.executeQuery();
            while (rs.next())
            {
                System.out.println(rs.getString(1) + "," + rs.getString(2) + "," + rs.getInt(3));
            }
            rs.close();
            pstmt.close();
        } catch (SQLException e) {
            if (pstmt != null) {
                try {
                    pstmt.close();
                } catch (SQLException e1) {
                    e1.printStackTrace();
                }
            }
            e.printStackTrace();
        }
    }

    /**
     * Main process, which calls static methods step by step.
     * @param args
    */
    public static void main(String[] args) {
        // Create a database connection.
        Connection conn = GetConnection("bank_app", "Gauss#3demo");

        //Run the prepared statement to query data.
        ExecPreparedSQL(conn);
```

```
        // Disconnect from the database.
        try {
            conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Step 2 Use javac to write the **DBTest.java** program.

```
javac DBTest.java
```

The **DBTest.class** file is generated in the current directory.

Step 3 Download the JDBC driver **openGauss-5.0.0-JDBC.tar.gz** from the openGauss website and compress it in the current directory.

```
[omm@ecs-840f jdbc]$ wget https://opengauss.obs.cn-south-
1.myhuaweicloud.com/5.0.0/arm/openGauss-5.0.0-JDBC.tar.gz
[omm@ecs-840f jdbc]$ tar zxvf openGauss-5.0.0-JDBC.tar.gz
postgresql.jar
opengauss-jdbc-5.0.0.jar
README_cn.md
README_en.md
```

The **opengauss-jdbc-5.0.0.jar** file is generated.

Step 4 Use Java to execute **DBTest.class**.

```
[omm@ecs-840f lab]$ java -cp opengauss-jdbc-5.0.0.jar:. DBTest
Oct 17, 2023 8:12:58 AM org.opengauss.core.v3.ConnectionFactoryImpl openConnectionImpl
INFO: [4747cad8-6489-475a-afa3-7563e87d706f] Try to connect. IP: localhost:26000
Oct 17, 2023 8:12:58 AM org.opengauss.core.v3.ConnectionFactoryImpl openConnectionImpl
INFO: [127.0.0.1:51874/ocalhost/127.0.0.1:26000] Connection is established. ID: 4747cad8-6489-475a-
afa3-7563e87d706f
Oct 17, 2023 8:12:58 AM org.opengauss.core.v3.ConnectionFactoryImpl openConnectionImpl
INFO: Connect complete. ID: 4747cad8-6489-475a-afa3-7563e87d706f
Connection succeed!
340211199301010015, Su Xiaogang, 8000
340211199301010010, Zhang Xiaofeng, 8000
340211199301010020, Song Chengfeng, 8000
340211199301010005, An Yanfang, 8000
```

Step 5 To add the multi-thread function to the Java program, write a Java program and save it as **DBSession.java**. Create 100 threads for the program. Then, have each thread connect to the database and execute a query statement every 100 ms.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

```
import java.sql.Statement;
import java.sql.CallableStatement;
import java.sql.Types;
import java.util.ArrayList;
import java.util.logging.LogManager;

public class DBSession    implements Runnable{

   // Create a database connection.
   public static Connection GetConnection(String username, String passwd) {
      String driver = "org.opengauss.Driver";
      String sourceURL = "jdbc:opengauss://localhost:26000/finance";
      Connection conn = null;
      try {
         // Load the database driver.
         Class.forName(driver).newInstance();
      } catch (Exception e) {
         e.printStackTrace();
         return null;
      }

      try {
         // Create a database connection.
         conn = DriverManager.getConnection(sourceURL, username, passwd);
         //System.out.println("Connection succeed!");
      } catch (Exception e) {
         e.printStackTrace();
         return null;
      }

      return conn;
   };

   //Run the prepared statement to query data.
   public static void ExecPreparedSQL(Connection conn) {
      PreparedStatement pstmt = null;
      try {
         pstmt = conn
               .prepareStatement("select c_id_card, c_name,sum(pro_income) total_income from client c,
bank_card b, property p, (select p_name name, p_id pro_pif_id, 1 pro_type from finances_product
union all select i_name name, i_id pro_pif_id, 2 pro_type from insurance union all select f_name
name, f_id pro_pif_id, 3 pro_type from fund) d where c.c_id = p.pro_c_id and c.c_id = b.b_c_id(+) and
p.pro_pif_id=d.pro_pif_id and p.pro_type=d.pro_type group by c_id_card, c_name order by 2 desc");
         ResultSet rs = pstmt.executeQuery();
         while (rs.next())
         {
            //System.out.println(rs.getString(1) + "," + rs.getString(2) + "," + rs.getInt(3));
         }
         System.out.print(".");
         rs.close();
         pstmt.close();
      } catch (SQLException e) {
         if (pstmt != null) {
            try {
               pstmt.close();
```

```
            } catch (SQLException e1) {
                e1.printStackTrace();
            }
        }
        e.printStackTrace();
    }
  }

  @Override
  public void run() {

    while (true) {
      // Create a database connection.
      Connection conn = GetConnection("bank_app", "Gauss#3demo");

      //Run the prepared statement to query data.
      ExecPreparedSQL(conn);

      // Disconnect from the database.
      try {
        conn.close();
      } catch (SQLException e) {
        e.printStackTrace();
      }
      try {
        Thread.sleep(100);
      } catch (InterruptedException e) {
        e.printStackTrace();
      }
    }
  }

  public static void main(String[] args) {
    LogManager.getLogManager().reset();
    int k = 0;
    ArrayList<Thread> list = new ArrayList<Thread>();
  while (k < 100) {
   DBSession slave = new DBSession();
   Thread thread = new Thread(slave);
   thread.start();
   list.add(thread);
   k++;
  }
  }

}
```

Step 6 Use javac to write **DBSession.java**.

```
javac DBSession.java
```

Step 7 Run **DBSession.class**

```
[omm@ecs-840f lab]$ java -cp opengauss-jdbc-5.0.0.jar:. DBSession
```

```
.....................................................................
```

Each point represents a thread.

Step 8 Press **Ctrl+C** to end the session. Then, run the following command to execute the preceding command in the background.

```
nohup java -cp opengauss-jdbc-5.0.0.jar:. DBSession &
```

6.2.5 Database Object Management

Use gsql to access information about user tables and indexes, active sessions, and SQL statements being executed in system catalogs and system views.

Step 1 Log in to the database as user **omm**.

```
[omm@ecs-5a5b ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=#
```

Step 2 Switch to the **finance** database.

```
openGauss=# \c finance
Non-SSL connection (SSL connection is recommended when requiring high-security)
You are now connected to database "finance" as user "omm".
finance=#
```

Step 3 View all tables.

```
finance=# \d
                                                List of relations
 Schema |          Name          | Type  |  Owner   |                                 Storage
--------+------------------------+-------+----------+-------------------------------------------------------------------
 public | bank_card              | table | bank_app | {orientation=row,compression=no}
 public | client                 | table | bank_app | {orientation=row,compression=no}
 public | finances_product | table | bank_app | {orientation=row,compression=no}
 public | fund                   | table | bank_app | {orientation=row,compression=no}
 public | insurance              | table | bank_app | {orientation=row,compression=no}
 public | property               | table | bank_app |
{orientation=row,compression=no,enable_rowsecurity=true}
(6 rows)
```

Step 4 View the detailed definition of the **property** table.

```
finance=# \d property
                        Table "public.property"
       Column         |                Type                | Modifiers
----------------------+------------------------------------+-----------
 pro_id               | integer                            | not null
 pro_c_id             | integer                            | not null
```

```
 pro_pif_id          | integer                          | not null
 pro_type            | integer                          | not null
 pro_status          | character(20)                    |
 pro_quantity        | integer                          |
 pro_income          | integer                          |
 pro_purchase_time | timestamp(0) without time zone |
Indexes:
    "property_pkey" PRIMARY KEY, btree (pro_id) TABLESPACE pg_default
Row Level Security Policies:
    POLICY "property_rls2" FOR ALL
      TO public
      USING ((("current_user"() <> 'fund_app'::name) OR (("current_user"() = 'fund_app'::name)
AND (pro_type = 3))))
```

Step 5 View all indexes.

```
finance=# \di
                                List of relations
 Schema |            Name             | Type  |  Owner   |         Table          | Storage
--------+-----------------------------+-------+----------+------------------------+---------
 public | bank_card_pkey              | index | bank_app | bank_card              |
 public | client_c_id_card_key        | index | bank_app | client                 |
 public | client_c_mail_key           | index | bank_app | client                 |
 public | client_c_phone_key          | index | bank_app | client                 |
 public | client_pkey                 | index | bank_app | client                 |
 public | finances_product_pkey | index | bank_app | finances_product |
 public | fund_pkey                   | index | bank_app | fund                   |
 public | insurance_pkey              | index | bank_app | insurance              |
 public | property_pkey               | index | bank_app | property               |
(9 rows)
```

Step 6 View the detailed definition of a specific index.

```
finance=# \d bank_card_pkey
       Index "public.bank_card_pkey"
  Column  |       Type        | Definition
----------+-------------------+------------
 b_number | character(30) | b_number
primary key, btree, for table "public.bank_card"
```

Step 7 Query the sessions and SQL statements running in the current database.

```
openGauss=# select a.tid,a.thread_name,a.query_id,a.wait_status,a.wait_event,left(b.query,50)
from dbe_perf.thread_wait_status a left join dbe_perf.session_stat_activity b
on a.tid=b.pid and a.query_id =b.query_id
where a.wait_status <> 'none';
        tid         |        thread_name         | query_id | wait_status | wait_event |
left
-----------------+------------------------+----------+-------------+------------+-----------------------------------
----------------
 281450480441712 | PostgreSQL JDBC Driver |        0 | wait cmd    | wait cmd   | select count(*)
from pg_settings where name = 'sup
 281450502527344 | PostgreSQL JDBC Driver |        0 | wait cmd    | wait cmd   | select version()
```

```
 281450637138288 | PostgreSQL JDBC Driver |          0 | wait cmd     | wait cmd     | select count(*)
from pg_settings where name = 'sup
 281450545650032 | PostgreSQL JDBC Driver |          0 | wait cmd     | wait cmd     | set
connection_info = '{"driver_name":"JDBC","driv
 281450721352048 | PostgreSQL JDBC Driver |          0 | wait cmd     | wait cmd     |
 281450687666544 | PostgreSQL JDBC Driver |          0 | wait cmd     | wait cmd     | set
connection_info = '{"driver_name":"JDBC","driv
 281450653981040 | PostgreSQL JDBC Driver |          0 | wait cmd     | wait cmd     | select
datcompatibility from pg_database where dat
(7 rows)
```

6.2.6 Health Check

Obtain all check items for the network, device, OS, and database that can be checked by gs_check. Execute all database check items and fix any issues found during the check, such as unanalyzed data tables.

Step 1 Obtain the gs_check directory.

```
[omm@ecs-840f lab]$ dirname `which gs_check`
/opt/huawei/install/om/script
```

Step 2 Replace **/opt/huawei/install/om/script** in the following command with the gs_check directory returned in the preceding command to obtain all check items supported by gs_check.

```
[omm@ecs-840f lab]$ find /opt/huawei/install/om/script/gspylib/inspection/items/ |grep '.py$' | awk -
F/ '{print $(NF-1), substr($NF,1,length($NF)-3)}'
find: '/opt/huawei/install/om/script/gspylib/inspection/items/network/__pycache__': Permission denied
find: '/opt/huawei/install/om/script/gspylib/inspection/items/device/__pycache__': Permission denied
find: '/opt/huawei/install/om/script/gspylib/inspection/items/os/__pycache__': Permission denied
find: '/opt/huawei/install/om/script/gspylib/inspection/items/cluster/__pycache__': Permission denied
network CheckBond
network CheckUsedPort
network CheckMTU
network CheckRouting
network CheckNICModel
network CheckNetWorkDrop
network CheckPing
network CheckRXTX
network CheckMultiQueue
network CheckNetSpeed
database CheckPgxcgroup
database CheckTDDate
database CheckSysTabSize
database CheckCurConnCount
database CheckSysadminUser
database CheckXid
database CheckPMKData
database CheckDropColumn
database CheckTableSpace
database CheckGUCConsistent
database CheckLockState
```

```
database CheckGUCValue
database CheckReturnType
database CheckSysTable
database CheckPgxcRedistb
database CheckIdleSession
database CheckMaxDatanode
database CheckDBConnection
database CheckDiskFailure
database CheckLockNum
database CheckArchiveParameter
database CheckCreateView
database CheckPgPreparedXacts
database CheckHashIndex
database CheckNodeGroupName
database CheckCursorNum
database CheckDNSkew
database CheckNextvalInDefault
database CheckUnAnalyzeTable
database CheckTableSkew
other CheckLogDiskUsage
other CheckTmpDiskUsage
other CheckDataDiskUsage
other CheckInstallDiskUsage
device CheckSwapMemory
device CheckInodeUsage
device CheckLogicalBlock
device CheckIOConfigure
device CheckSpaceUsage
device CheckMaxAsyIOrequests
device CheckDiskFormat
device CheckIOrequestqueue
device CheckBlockdev
device CheckSlowDisk
device CheckDiskConfig
os CheckKeyProAdj
os CheckMaxProcMemory
os CheckNTPD
os CheckHyperThread
os CheckOmmUserExist
os CheckProcessLeft
os CheckProcMem
os CheckPortConflict
os CheckBootItems
os CheckMaxHandle
os CheckSysParams
os CheckFilehandle
os CheckSshdConfig
os CheckDirLeft
os CheckSysPortRange
os CheckCrondService
os CheckKernelVer
os CheckSshdService
os CheckEncoding
os CheckMemInfo
os CheckTHP
```

```
os CheckTimeZone
os CheckCPU
os CheckCrontabLeft
os CheckOSVer
os CheckEtcHosts
os CheckStack
os CheckCpuCount
os CheckFirewall
os CheckDropCache
cluster CheckMpprcFile
cluster CheckLargeFile
cluster CheckProcessStatus
cluster CheckUpVer
cluster CheckCatchup
cluster CheckSpecialFile
cluster CheckDBParams
cluster CheckEnvProfile
cluster CheckDirPermissions
cluster CheckIntegrity
cluster CheckPortRange
cluster CheckProStartTime
cluster CheckDilateSysTab
cluster CheckDebugSwitch
cluster CheckGaussVer
cluster CheckReadonlyMode
cluster CheckClusterState
cluster CheckCollector
```

Step 3 Create the **/home/omm/lab** directory and save all the following database check items to the **/home/omm/lab /check_db_list** file.

```
CheckPgxcgroup
CheckTDDate
CheckSysTabSize
CheckCurConnCount
CheckSysadminUser
CheckXid
CheckPMKData
CheckDropColumn
CheckTableSpace
CheckGUCConsistent
CheckLockState
CheckGUCValue
CheckReturnType
CheckSysTable
CheckPgxcRedistb
CheckIdleSession
CheckMaxDatanode
CheckDBConnection
CheckDiskFailure
CheckLockNum
CheckArchiveParameter
CheckCreateView
CheckPgPreparedXacts
CheckHashIndex
```

```
CheckNodeGroupName
CheckCursorNum
CheckDNSkew
CheckNextvalInDefault
CheckUnAnalyzeTable
CheckTableSkew
```

Step 4 Check all database check items.

```
[omm@ecs-840f lab]$ for i in `cat check_db_list`; do gs_check -L -i $i; done
2023-10-19 21:58:37 [NAM] CheckPgxcgroup
2023-10-19 21:58:37 [STD] Check the number of records whose in_redistribution is Y in the
pgxc_group table. If the number is 0, this check is passed.
2023-10-19 21:58:37 [RST] OK
The cluster has been redistributed.
2023-10-19 21:58:37 [RAW]
select count(group_name) from pgxc_group where in_redistribution='y' OR in_redistribution='t';

2023-10-19 21:58:40 [NAM] CheckTDDate
2023-10-19 21:58:40 [STD] Checks whether the ORC table in a Teradata database contains columns
of the DATE type. If yes, this check fails.
2023-10-19 21:58:40 [RST] OK
The database with TD mode does not exist.
2023-10-19 21:58:40 [RAW]
select datname from pg_database where datcompatibility = 'TD';

2023-10-19 21:58:42 [NAM] CheckSysTabSize
2023-10-19 21:58:42 [STD] Abnormal is thrown on capacity insufficiency.
2023-10-19 21:58:42 [RST] OK
FileSystem: /dev/vda2                SystemTableSize: 63.0390625M          DiskAvailSize: 30144M
Status: OK;

2023-10-19 21:58:42 [RAW]

2023-10-19 21:58:46 [NAM] CheckCurConnCount
2023-10-19 21:58:46 [STD] Check the number of database connections. If the number of connections
is less than 90% of the maximum allowed, this check is passed.
2023-10-19 21:58:46 [RST] OK
0.00%
2023-10-19 21:58:46 [RAW]
show max_connections;SELECT count(*) FROM pg_stat_activity;

2023-10-19 21:58:49 [NAM] CheckSysadminUser
2023-10-19 21:58:49 [STD] Check whether the sysadmin user exists except the cluster owner. If yes,
this check fails.
2023-10-19 21:58:49 [RST] NG
There are sysadmin users except omm:
bank_app
2023-10-19 21:58:49 [RAW]
select rolname from pg_roles where rolsystemadmin=True and rolname !='omm';

2023-10-19 21:58:52 [NAM] CheckXid
2023-10-19 21:58:52 [STD] Warning is thrown if the xid value is greater than 1 billion. Abnormal is
thrown if the xid value is greater than 1.8 billion.
2023-10-19 21:58:52 [RST] OK
```

```
The xid value is 57275
2023-10-19 21:58:52 [RAW]
select txid_current();

2023-10-19 21:58:55 [NAM] CheckPMKData
2023-10-19 21:58:55 [STD] Check whether the PMK schema of the database contains abnormal data. If no, this check is passed.
2023-10-19 21:58:55 [RST] OK
No exception data in PMK.
2023-10-19 21:58:55 [RAW]
select proname,pronamespace from pg_proc where pronamespace not in (select oid from pg_namespace);

2023-10-19 21:58:58 [NAM] CheckDropColumn
2023-10-19 21:58:58 [STD] Check whether the impact caused by the alter table drop column command has been eliminated. If no, this check fails.
2023-10-19 21:58:58 [RST] OK
No alter table drop column operation
2023-10-19 21:58:58 [RAW]


2023-10-19 21:59:01 [NAM] CheckTableSpace
2023-10-19 21:59:01 [STD] Check whether the tablespace path and cluster path are nested or the tablespace paths are nested. If no, this check is passed.
2023-10-19 21:59:01 [RST] OK
All table space path is normal.
2023-10-19 21:59:01 [RAW]

2023-10-19 21:59:04 [NAM] CheckGUCConsistent
2023-10-19 21:59:04 [STD]
2023-10-19 21:59:04 [RST] OK
{"DN_6001": {"acce_min_datasize_per_thread": ...}}
2023-10-19 21:59:04 [RAW]
{'DN_6001': {'acce_min_datasize_per_thread': ...'}}

2023-10-19 21:59:07 [NAM] CheckLockState
2023-10-19 21:59:07 [STD] Check whether the cluster is locked. If no, this check is passed.
2023-10-19 21:59:07 [RST] OK
1
2023-10-19 21:59:07 [RAW]
select count(1) from pg_stat_activity where instr( query, 'pgxc_lock_for_backup()') > 0 or instr( query, 'pg_advisory_lock(65535,65535)') > 0;

2023-10-19 21:59:11 [NAM] CheckGUCValue
2023-10-19 21:59:11 [STD] Check whether the result of (max_connections + max_prepared_transactions) *
max_locks_per_transaction is greater than or equal to 1000000. If yes, this check fails.
2023-10-19 21:59:11 [RST] OK
max_locks_per_transaction[256] * (max_connections[5000] + max_prepared_transactions[800]) = 1484800
2023-10-19 21:59:11 [RAW]
show max_connections;show max_prepared_transactions;show max_locks_per_transaction;

2023-10-19 21:59:14 [NAM] CheckReturnType
```

```
2023-10-19 21:59:14 [STD] Check whether the user-defined function contains an invalid return type. If no, this check is passed.
2023-10-19 21:59:14 [RST] OK
User-defined functions do not contain illegal return types.
2023-10-19 21:59:14 [RAW]
select proname from pg_proc, pg_type where pg_proc.prorettype=pg_type.oid and pg_type.typisdefined=false;

2023-10-19 21:59:17 [NAM] CheckSysTable
2023-10-19 21:59:17 [STD] Query the system catalog. If the query is successful, this check is passed.
2023-10-19 21:59:17 [RST] OK
Instance table              size              row       width row*width
DN_6001   pg_attribute      1572864           7676      13    99788
DN_6001   pg_class          466944            812       9     7308
DN_6001   pg_constraint     49152             8         14    112
DN_6001   pg_partition      8192              0         13    0
DN_6001   pgxc_class        8192              0         11    0
DN_6001   pg_index          90112             272       9     2448
DN_6001   pg_stats          0                 545       9     4905

2023-10-19 21:59:17 [RAW]
Instance table              size              row       width row*width
DN_6001   pg_attribute      1572864           7676      13    99788
DN_6001   pg_class          466944            812       9     7308
DN_6001   pg_constraint     49152             8         14    112
DN_6001   pg_partition      8192              0         13    0
DN_6001   pgxc_class        8192              0         11    0
DN_6001   pg_index          90112             272       9     2448
DN_6001   pg_stats          0                 545       9     4905


2023-10-19 21:59:21 [NAM] CheckPgxcRedistb
2023-10-19 21:59:21 [STD] If the target is not found, this check is passed.
2023-10-19 21:59:21 [RST] OK
Residue Table pgxc_redistb and residue schema data_redis do not exist in the cluster.
2023-10-19 21:59:21 [RAW]
select * from pg_tables where tablename='pgxc_redistb';select * from pg_namespace where nspname='data_redis';

2023-10-19 21:59:24 [NAM] CheckIdleSession
2023-10-19 21:59:24 [STD] Check if there are idle sessions. If no, this check is passed.
2023-10-19 21:59:24 [RST] OK
No idle process.
2023-10-19 21:59:24 [RAW]
select pid, query_id, application_name, query_start, state, query from pg_stat_activity where state <> 'idle' and  application_name not in ('JobScheduler', 'WorkloadManager', 'WLMArbiter', 'workload', 'WorkloadMonitor', 'Snapshot', 'PercentileJob') and query_id <> 0 and query not like '%pg_stat_activity%';

2023-10-19 21:59:27 [NAM] CheckMaxDatanode
2023-10-19 21:59:27 [STD] Check if the maximum number of DNs is less than the number of nodes configured in the XML file multiplied by the number of DNs (= 90 x 5 by default). If yes, Warning is thrown; if no, this check is passed.
2023-10-19 21:59:27 [RST] OK
dataNum: 450, comm_max_datanode:
```

```
2023-10-19 21:59:27 [RAW]
SELECT setting FROM pg_settings WHERE name='comm_max_datanode';

2023-10-19 21:59:31 [NAM] CheckDBConnection
2023-10-19 21:59:31 [STD] Check whether the database can be connected. If yes, this check is passed.
2023-10-19 21:59:31 [RST] OK
The database connection is normal.
2023-10-19 21:59:31 [RAW]

source '/home/omm/.bashrc' && gsql -m -d postgres -p 26000 -c 'select pg_sleep(1);'

2023-10-19 21:59:35 [NAM] CheckDiskFailure
2023-10-19 21:59:35 [STD] Perform full query on all tables in all databases in the cluster. If a fault occurs, NG is thrown. If all tables are successfully queried, OK is thrown.
2023-10-19 21:59:35 [RST] OK
No data is distributed on the fault disk
2023-10-19 21:59:35 [RAW]

2023-10-19 21:59:38 [NAM] CheckLockNum
2023-10-19 21:59:38 [STD] Check the number of database locks. If the query is successful, this check is passed.
2023-10-19 21:59:38 [RST] OK
2
2023-10-19 21:59:38 [RAW]
select   count(*) from pg_locks

2023-10-19 21:59:41 [NAM] CheckArchiveParameter
2023-10-19 21:59:41 [STD] Check the database archive parameter. If the parameter is not enabled, or is enabled and is in the CN directory, this check is passed. If the parameter is enabled but is not in the CN directory, this check fails.
2023-10-19 21:59:41 [RST] OK
off
2023-10-19 21:59:41 [RAW]
show archive_mode;

2023-10-19 21:59:44 [NAM] CheckCreateView
2023-10-19 21:59:44 [STD] A renamed field is found, and the check fails.
2023-10-19 21:59:44 [RST] OK
No view needs to be fixed.
2023-10-19 21:59:44 [RAW]

2023-10-19 21:59:47 [NAM] CheckPgPreparedXacts
2023-10-19 21:59:47 [STD] Check the pg_prepared_xacts parameter. If no two-phase transaction exists, this check is passed.
2023-10-19 21:59:47 [RST] OK

2023-10-19 21:59:47 [RAW]
SELECT * FROM pg_catalog.pgxc_prepared_xacts;

2023-10-19 21:59:51 [NAM] CheckHashIndex
2023-10-19 21:59:51 [STD] Check whether there are hash indexes. If yes, this check fails.
2023-10-19 21:59:51 [RST] OK
There is no hash index in all databases.
2023-10-19 21:59:51 [RAW]
```

```
2023-10-19 21:59:54 [NAM] CheckNodeGroupName
2023-10-19 21:59:54 [STD] Check whether the node group name contains non-SQL_ASCII characters. If yes, this check fails.
2023-10-19 21:59:54 [RST] OK
The node group name with SQL_ASCII characters in all databases.
2023-10-19 21:59:54 [RAW]

2023-10-19 21:59:57 [NAM] CheckCursorNum
2023-10-19 21:59:57 [STD] Check the number of cursors in the database. If the query is successful, this check is passed.
2023-10-19 21:59:57 [RST] OK
0
2023-10-19 21:59:57 [RAW]
select   count(*) from pg_cursors;

2023-10-19 21:59:59 [NAM] CheckDNSkew
2023-10-19 21:59:59 [STD]
2023-10-19 21:59:59 [RST] OK
Data distributed well in local host
2023-10-19 21:59:59 [RAW]
{"6001": "72"}

2023-10-19 22:00:03 [NAM] CheckNextvalInDefault
2023-10-19 22:00:03 [STD] Check whether the DEFAULT expression contains nextval(sequence). If yes, this check fails.
2023-10-19 22:00:03 [RST] OK
no default expression contains nextval(sequence)
2023-10-19 22:00:03 [RAW]

2023-10-19 22:00:07 [NAM] CheckUnAnalyzeTable
2023-10-19 22:00:07 [STD] If the database has an unanalyzed table, this check fails.
2023-10-19 22:00:07 [RST] NG
Tables unanalyzed:
template1:
dbe_pldeveloper.gs_source
finance:
public.insurance
public.bank_card
public.client
public.finances_product
public.property
public.fund
dbe_pldeveloper.gs_source
postgres:
bank_app.customer_t1

2023-10-19 22:00:07 [RAW]

2023-10-19 22:00:10 [NAM] CheckTableSkew
2023-10-19 22:00:10 [STD] Check whether there is table-level data skew. If yes, this check fails.
2023-10-19 22:00:10 [RST] OK
Data is well distributed
2023-10-19 22:00:10 [RAW]
```

Step 5 Performs ANALYZE operation on the **finance** database's tables returned by CheckUnAnalyzeTable, and log in to the **finance** database as user **omm**.

```
[omm@ecs-840f lab]$ gsql -d finance -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.
```

Step 6 Perform **ANALYZE** operation.

```
finance=# analyze insurance;
ANALYZE
finance=# analyze bank_card;
ANALYZE
finance=# analyze client;
ANALYZE
finance=# analyze finances_product;
ANALYZE
finance=# analyze property;
ANALYZE
finance=# analyze fund;
ANALYZE
finance=# analyze dbe_pldeveloper.gs_source;
ANALYZE
```

Step 7 Run **gs_check** on CheckUnAnalyzeTable again. The **finance** table is not shown in the returned result, indicating that the **finance** table has been analyzed.

```
[omm@ecs-840f lab]$ gs_check -L -i CheckUnAnalyzeTable

2023-10-19 22:42:42 [NAM] CheckUnAnalyzeTable
2023-10-19 22:42:42 [STD] If the database has an unanalyzed table, this check fails.
2023-10-19 22:42:42 [RST] NG
Tables unanalyzed:
template1:
dbe_pldeveloper.gs_source
postgres:
bank_app.customer_t1

2023-10-19 22:42:42 [RAW]
```

6.2.7 Performance Check

Use gs_checkperf to check key performance indicators of the openGauss cluster.

Step 1 Run the **gs_checkperf** command as user **omm** to view brief information about the current cluster.

```
[omm@ecs-840f ~]$ gs_checkperf
Cluster statistics information:
     Host CPU busy time ratio                     :     4.63          %
     MPPDB CPU time % in busy time                :     76.18         %
     Shared Buffer Hit ratio                      :     100.00        %
```

```
    In-memory sort ratio                : 0
    Physical Reads                      : 3332
    Physical Writes                     : 63279
    DB size                             : 73 MB
    Total Physical writes               : 63279
    Active SQL count                    : 4
    Session count                       : 8l
```

Step 2 Run the **gs_checkperf** command as user **omm** to view detailed information about the current cluster.

```
[omm@ecs-840f lab]$ gs_checkperf -i PMK --detail

Cluster statistics information:
Host CPU usage rate:
    Host total CPU time                 : 8499462120.000 Jiffies
    Host CPU busy time                  : 403670590.000 Jiffies
    Host CPU iowait time                : 142980.000 Jiffies
    Host CPU busy time ratio            : 4.75 %
    Host CPU iowait time ratio          : 0.00 %
MPPDB CPU usage rate:
    MPPDB CPU time % in busy time       : 76.15 %
    MPPDB CPU time % in total time      : 3.62 %
Shared buffer hit rate:
    Shared Buffer Reads                 : 8695
    Shared Buffer Hits                  : 12322002129
    Shared Buffer Hit ratio             : 100.00 %
In memory sort rate:
    In-memory sort count                : 0
    In-disk sort count                  : 0
    In-memory sort ratio                : 0
I/O usage:
    Number of files                     : 220
    Physical Reads                      : 3328
    Physical Writes                     : 61439
    Read Time                           : 249260 ms
    Write Time                          : 179203 ms
Disk usage:
    DB size                             : 71 MB
    Total Physical writes               : 61439
    Average Physical write              : 342845.82
    Maximum Physical write              : 61439
Activity statistics:
    Active SQL count                    : 4
    Session count                       : 8
Node statistics information:
dn_6001:
    MPPDB CPU Time                      : 307377430 Jiffies
    Host CPU Busy Time                  : 403670590 Jiffies
    Host CPU Total Time                 : 8499462120 Jiffies
    MPPDB CPU Time % in Busy Time       : 76.15 %
    MPPDB CPU Time % in Total Time      : 3.62 %
    Physical memory                     : 32886882304 Bytes
    DB Memory usage                     : 9808445440 Bytes
    Shared buffer size                  : 1073741824 Bytes
```

```
        Shared buffer hit ratio                 :      100.00        %
        Sorts in memory                         :      0
        Sorts in disk                           :      0
        In-memory sort ratio                    :      0
        Number of files                         :      220
        Physical Reads                          :      3328
        Physical Writes                         :      61439
        Read Time                               :      249260
        Write Time                              :      179203
Session statistics information(Top 10):
Session CPU statistics:
1 dn_6001-postgres-omm:
        Session CPU time                        :      3
        Database CPU time                       :      307377480
        Session CPU time %                      :      0.00          %
2 dn_6001-postgres-omm:
        Session CPU time                        :      0
        Database CPU time                       :      307377480
        Session CPU time %                      :      0.00          %
3 dn_6001-postgres-omm:
        Session CPU time                        :      0
        Database CPU time                       :      307377480
        Session CPU time %                      :      0.00          %
4 dn_6001-postgres-omm:
        Session CPU time                        :      0
        Database CPU time                       :      307377480
        Session CPU time %                      :      0.00          %

Session Memory statistics:
1 dn_6001-postgres-omm:
        Buffer Reads                            :      302
        Shared Buffer Hit ratio                 :      100.00
        In Memory sorts                         :      0
        In Disk sorts                           :      0
        In Memory sorts ratio                   :      0
        Total Memory Size                       :      6774376
        Used Memory Size                        :      5566024
2 dn_6001-postgres-omm:
        Buffer Reads                            :      301
        Shared Buffer Hit ratio                 :      99.67
        In Memory sorts                         :      0
        In Disk sorts                           :      0
        In Memory sorts ratio                   :      0
        Total Memory Size                       :      6714248
        Used Memory Size                        :      5479976
3 dn_6001-postgres-omm:
        Buffer Reads                            :      1746918
        Shared Buffer Hit ratio                 :      99.85
        In Memory sorts                         :      0
        In Disk sorts                           :      0
        In Memory sorts ratio                   :      0
        Total Memory Size                       :      6078488
        Used Memory Size                        :      4583200
4 dn_6001-postgres-omm:
        Buffer Reads                            :      1113
```

```
    Shared Buffer Hit ratio                 :    100.00
    In Memory sorts                         :    1
    In Disk sorts                           :    0
    In Memory sorts ratio                   :    100.00
    Total Memory Size                       :    12239936
    Used Memory Size                        :    10024248

Session IO statistics:
1 dn_6001-postgres-omm:
    Physical Reads                          :    2603
    Read Time                               :    20330
2 dn_6001-postgres-omm:
    Physical Reads                          :    1
    Read Time                               :    542
3 dn_6001-postgres-omm:
    Physical Reads                          :    0
    Read Time                               :    0
4 dn_6001-postgres-omm:
    Physical Reads                          :    0
    Read Time                               :    0
```

6.2.8 Log Collection

Use gs_collector to gather OS information, log information, and configuration files, and view the contents of each log.

Step 1 Run the **gs_collector** command and change the timestamp to a proper value.

```
[omm@ecs-840f lab]$ gs_collector --begin-time='20231101 07:00' --end-time='20231107 16:10'
Successfully parsed the configuration file.
create Dir.
Successfully create dir.
do system check interval 0 : count 1
Collecting OS information.
Successfully collected OS information.
do database check interval 0 : count 1
Collecting catalog statistics.
Successfully collected catalog statistics.
do log check interval 0 : count 1
Collecting Log files.
Successfully collected Log files.
do Config check 0:1
Collecting Config files.
Successfully collected Config files.
Collecting files.
Successfully collected files.
All results are stored in /var/log/omm/omm/collector_20231020_072319.tar.gz.
```

Step 2 Decompress the package.

```
[omm@ecs-840f lab]$ tar zxvf /var/log/omm/omm/collector_20231020_072319.tar.gz
collector_20231020_072319/
collector_20231020_072319/Summary.log
collector_20231020_072319/ecs-840f.tar.gz
```

```
collector_20231020_072319/Detail.log
```

Step 3 Go to the decompression directory.

```
[omm@ecs-840f lab]$ cd collector_20231020_072319/
```

Step 4 Decompress the compressed file named after the current node.

```
[omm@ecs-840f collector_20231020_072319]$ tar zxvf ecs-840f.tar.gz
ecs-840f/
ecs-840f/gstackfiles/
ecs-840f/systemfiles/
ecs-840f/systemfiles/OS_information_20231020_072319705145.txt
ecs-840f/systemfiles/database_system_info_20231020_072323730748.txt
ecs-840f/catalogfiles/
ecs-840f/catalogfiles/gs_clean_20231020_072326105481.txt
ecs-840f/catalogfiles/dn_6001_pg_stat_activity_20231020_072325804445.csv
ecs-840f/catalogfiles/dn_6001_pg_locks_20231020_072325650139.csv
ecs-840f/catalogfiles/dn_6001_pg_thread_wait_status_20231020_072325954258.csv
ecs-840f/planSimulatorfiles/
ecs-840f/xlogfiles/
ecs-840f/coreDumpfiles/
ecs-840f/configfiles/
ecs-840f/configfiles/config_20231020_072327542052/
ecs-840f/configfiles/config_20231020_072327542052/dn_6001/
ecs-840f/configfiles/config_20231020_072327542052/dn_6001/pg_replslot/
ecs-840f/configfiles/config_20231020_072327542052/dn_6001/postgresql.conf
ecs-840f/configfiles/config_20231020_072327542052/dn_6001/pg_hba.conf
ecs-840f/configfiles/config_20231020_072327542052/dn_6001/pg_ident.conf
ecs-840f/configfiles/config_20231020_072327542052/dn_6001/gaussdb.state
ecs-840f/logfiles/
ecs-840f/logfiles/log_20231020_072326806736.tar.gz
```

Step 5 View the collected OS status information.

```
[omm@ecs-840f collector_20231020_072319]$ vi ecs-840f/systemfiles/OS_information_*.txt
```

Step 6 View the collected database status information.

```
[omm@ecs-840f collector_20231020_072319]$ vi ecs-840f/systemfiles/database_system_info_*.txt
```

Step 7 View the collected information about the pg_stat_activity, pg_locks and pg_thread_wait_status views.

```
[omm@ecs-840f collector_20231020_072319]$ vi ecs-840f/catalogfiles/*_pg_*
```

Step 8 View the content of the collected configuration files **postgresql.conf**, **pg_ident.conf**, and **pg_hba.conf**.

```
[omm@ecs-840f collector_20231020_072319]$ vi ecs-840f/configfiles/config_*/dn_6001/*.conf
```

6.2.9 Parameter Settings

Pay close attention to resource parameters and set the shared buffer size of the database based on the memory size of the OS.

Step 1 Check the total memory and free memory of the OS.

```
[omm@ecs-840f lab]$ free -h
              total        used        free      shared  buff/cache   available
Mem:           30Gi       1.0Gi        25Gi       1.0Gi       4.0Gi        24Gi
Swap:            0B          0B          0B
```

Step 2 Use the gsql client to connect to the **postgres** database as an administrator.

```
gsql -d postgres -p 26000 -r
```

Step 3 Check the shared buffer size of the current database.

```
openGauss=# show shared_buffers;
 shared_buffers
----------------
 1GB
(1 row)
```

Step 4 Adjust the shared buffer parameter.

```
openGauss=# alter system set shared_buffers='10GB';
NOTICE:  please restart the database for the POSTMASTER level parameter to take effect.
ALTER SYSTEM SET
```

Step 5 Stop the database.

```
[omm@ecs-e1b3 ~]$ gs_om -t stop;
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
```

Step 6 Start the database.

```
[omm@ecs-840f collector_20231020_072319]$ gs_om -t start;
Starting cluster.
=========================================
[SUCCESS] ecs-840f
=========================================
Successfully started.
```

Step 7 Check that the parameter has been adjusted successfully.

```
[omm@ecs-840f lab]$ gsql -d postgres -p 26000 -r -c 'show shared_buffers'
 shared_buffers
```

```
----------------
 10GB
(1 row)
```

6.2.10 Background Maintenance

After frequent data deletion and updates, maintain tables and indexes using the **VACUUM**, **REINDEX**, and **ANALYZE** commands.

Step 1 Log in to the **finance** database.

```
gsql -d finance -p 26000 -r
```

Step 2 Create the test table **many_clients**. Before creating this table, ensure that **enable_default_ustore_table** has been set to **off**. Otherwise, run the following command to change the setting and restart the database: gs_guc set -N all -I all -c "enable_default_ustore_table=off"

```
create table many_clients
as
select c.*, generate_series gid
from
client c,
generate_series(1, 10000)
;
create index mc_gid on many_clients(gid);
```

Step 3 Analyze the **many_clients** table.

```
finance=# analyze verbose many_clients;
INFO:  analyzing "public.many_clients"(dn_6001 pid=1668253)
INFO:  ANALYZE INFO : "many_clients": scanned 5173 of 5173 pages, containing 300000 live rows
and 0 dead rows; 30000 rows in sample, 300000 estimated total rows(dn_6001 pid=1668253)
ANALYZE
```

Step 4 Check the size of the **many_clients** table.

```
finance=# \dt+ many_clients;
                                    List of relations
 Schema |      Name      | Type  | Owner | Size  |                  Storage                   | Description
--------+----------------+-------+-------+-------+--------------------------------------------+-------------
 public | many_clients | table | omm     | 40 MB | {orientation=row,compression=no} |
(1 row)
```

Step 5 Check the size of the **mg_gid** index.

```
finance=# \di+  mc_gid
                                      List of relations
 Schema |   Name   | Type  | Owner |      Table      |   Size   | Storage | Description
--------+----------+-------+-------+-----------------+----------+---------+-------------
 public | mc_gid | index | omm     | many_clients | 6616 kB |             |
(1 row)
```

Step 6 Delete 99% of the data from the table.

```
finance=# delete from many_clients where gid>1;
DELETE 299970
```

Step 7 Run **ANALYZE** on the **many_clients** table.

```
finance=# analyze verbose many_clients;
INFO:  analyzing "public.many_clients"(dn_6001 pid=1668253)
INFO:  ANALYZE INFO : "many_clients": scanned 5173 of 5173 pages, containing 30 live rows and
299970 dead rows; 30 rows in sample, 30 estimated total rows(dn_6001 pid=1668253)
ANALYZE
```

Step 8 Check the size of the **many_clients** table.

```
finance=# \dt+ many_clients;
                                        List of relations
 Schema |       Name        | Type  | Owner | Size  |                    Storage                     | Description
--------+-------------------+-------+-------+-------+------------------------------------------------+-------------
 public | many_clients | table | omm     | 40 MB | {orientation=row,compression=no} |
(1 row)
```

Step 9 Check the size of the **mg_gid** index.

```
finance=# \di+   mc_gid
                                          List of relations
 Schema |   Name   | Type  | Owner |      Table       |   Size    | Storage | Description
--------+----------+-------+-------+------------------+-----------+---------+-------------
 public | mc_gid | index | omm     | many_clients | 6616 kB |             |
(1 row)
```

Step 10 Run the **VACUUM** statement to reclaim the disk space of the **many_clients** table.

```
finance=# vacuum full many_clients;
VACUUM
```

Step 11 Run **ANALYZE** on the **many_clients** table.

```
finance=#   analyze verbose many_clients;
INFO:  analyzing "public.many_clients"(dn_6001 pid=1668253)
INFO:  ANALYZE INFO : "many_clients": scanned 1 of 1 pages, containing 30 live rows and 0 dead
rows; 30 rows in sample, 30 estimated total rows(dn_6001 pid=1668253)
ANALYZE
```

Step 12 Check the size of the **many_clients** table.

```
finance=# \dt+ many_clients;
                                          List of relations
 Schema |       Name        | Type  | Owner |      Size      |                    Storage                     |
Description
--------+-------------------+-------+-------+----------------+------------------------------------------------+-------------
 public | many_clients | table | omm     | 8192 bytes | {orientation=row,compression=no} |
```

```
(1 row)
```

Step 13 Use **REINDEX** to re-create the **mc_gid** index.

```
finance=# REINDEX INDEX mc_gid;
REINDEX
```

Step 14 Check the index size.

```
finance=# \di+  mc_gid
                                    List of relations
 Schema |  Name  | Type  | Owner |     Table      | Size  | Storage | Description
--------+--------+-------+-------+----------------+-------+---------+-------------
 public | mc_gid | index | omm   | many_clients | 16 kB |         |
(1 row)
```

6.2.11 WDR

Generate a WDR based on the latest two snapshots and interpret the report.

Step 1 Log in to the **postgres** database as user **omm**.

```
gsql -d postgres -p 26000 -r
```

Step 2 View the value of **enable_wdr_snapshot**.

```
openGauss=# show enable_wdr_snapshot;
 enable_wdr_snapshot
---------------------
 off
(1 row)
```

Step 3 If **enable_wdr_snapshot** is **off**, run the following command to set it to **on**:

```
openGauss=# alter system set enable_wdr_snapshot=on;
ALTER SYSTEM SET
```

Step 4 Query available snapshots.

```
openGauss=# select * from snapshot.snapshot;
 snapshot_id |               start_ts               |                end_ts
-------------+---------------------------------------+---------------------------------------
           1 | 2023-10-21 07:59:36.85467+08 | 2023-10-21 07:59:38.064506+08
(1 row)
```

Step 5 If there is only one snapshot, create one more snapshot.

```
openGauss=# select create_wdr_snapshot();
              create_wdr_snapshot
-----------------------------------------
 WDR snapshot request has been submitted
```

```
(1 row)
```

Step 6 Check that a new snapshot has been generated.

```
openGauss=# select * from snapshot.snapshot;
 snapshot_id |              start_ts                |                end_ts
-------------+---------------------------------+-------------------------------
           1 | 2023-10-21 07:59:36.85467+08    | 2023-10-21 07:59:38.064506+08
           2 | 2023-10-21 08:13:33.540987+08 | 2023-10-21 08:13:34.717914+08
(2 rows)
```

Step 7 Use the latest two snapshots to generate a WDR.

```
openGauss=#    \a \t \o wdr.html
Output format is unaligned.
Showing only tuples.
openGauss=# select generate_wdr_report(1, 2, 'all', 'node', pgxc_node_str()::cstring);
openGauss=# \o
```

Step 8 Exit gsql, download **wdr.html** to the local PC, open it using a browser, and view top 5 waiting events and top SQL records.

6.2.12 Encrypted Query

Add a column to the **client** table and set it as an encrypted column. Query data on the client to assign a value to the encrypted column and verify that the data in the column has been automatically encrypted on the server.

Step 1 Create the KMS directory for creating an encrypted database as user **omm**.

```
[omm@ecs-840f lab]$ mkdir -p $GAUSSHOME/etc/localkms/
```

Step 2 Log in to the **finance** database as user **omm** and enable the encryption function using **-C**.

```
gsql -p 26000 finance -r -C
```

Step 3 Create a CMK.

```
finance=# CREATE CLIENT MASTER KEY finCMK WITH (KEY_STORE = localkms, KEY_PATH =
"finance",
ALGORITHM = RSA_2048);
CREATE CLIENT MASTER KEY
```

Step 4 Create a CEK.

```
finance=# CREATE COLUMN ENCRYPTION KEY finCEK WITH VALUES (CLIENT_MASTER_KEY =
finCMK,
ALGORITHM   = AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
```

Step 5 Add the encrypted column **c_name_e** to the **client** table.

```
finance=# ALTER TABLE client ADD COLUMN c_name_e VARCHAR(100) ENCRYPTED WITH
  (COLUMN_ENCRYPTION_KEY = finCEK, ENCRYPTION_TYPE = DETERMINISTIC);
ALTER TABLE
```

Step 6 Query the name of the user whose **c_id** is **1**.

```
finance=> select c_name from client where c_id=1;
  c_name
--------
  Zuo Xiaoting
(1 row)
```

Step 7 Run the **update** statement to update the queried value to the **c_name_e** column.

```
finance=# update client set c_name_e='Zuo Xiaoting' where c_id=1;
UPDATE 1
```

Step 8 Run the query statement.

```
finance=# select c_name_e from client where c_id=1;
  c_name_e
----------
  Zuo Xiaoting
(1 row)
```

Step 9 Exit gsql, log in again, and do not enable the encryption function (**-C**).

```
gsql -p 26000 finance -r
```

Step 10 Check whether the value of **c_name_e** is encrypted.

```
finance=# select c_name_e from client where c_id=1;
                                                
```

Step 2 Query the ledger database schema.

```
finance=# CREATE SCHEMA ledgernsp WITH BLOCKCHAIN;
CREATE SCHEMA
```

Step 3 Create table **property** in the schema.

```
finance=# CREATE TABLE ledgernsp.property
finance-# (
finance(#             pro_id INT PRIMARY KEY,
finance(#             pro_c_id INT NOT NULL,
finance(#             pro_pif_id INT NOT NULL,
          pro_quantity INT,
finance(#             pro_type INT NOT NULL,
          pro_purchase_time DATE
finance(#             pro_status CHAR(20),
finance(#             pro_quantity INT,
finance(#             pro_income INT,
finance(#             pro_purchase_time DATE
finance(# );
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "property_pkey" for table
"property"
CREATE TABLE
```

Step 4 Query data from the original **property** table and insert it into the new **property** table.

```
finance=# insert into ledgernsp.property select * from property;
INSERT 0 4
```

Step 5 Query data in the **property** table.

```
finance=# select *,hash from ledgernsp.property;
 pro_id | pro_c_id | pro_pif_id | pro_type |       pro_status       | pro_quantity | pro_income |
pro_purchase_time   |          hash
--------+----------+------------+----------+------------------------+--------------+------------+---------------------
+------------------
       1 |        5 |          1 |        1 | Available              |            4 |       8000 | 2018-07-
01 00:00:00 | f3de57e69253919e
       2 |       10 |          2 |        2 | Available              |            4 |       8000 | 2018-07-
01 00:00:00 | c81d20819d5917d6
       3 |       15 |          3 |        3 | Available              |            4 |       8000 | 2018-07-
01 00:00:00 | a57b0e921843d811
       4 |       20 |          4 |        1 | Frozen                 |            4 |       8000 | 2018-
07-01 00:00:00 | 225f6c904e149e9d
(4 rows)
```

Step 6 Find the transaction whose **pro_c_id** is **10**, and change its **pro_quantity** value from **4** to **5**.

```
finance=# update ledgernsp.property set pro_quantity = 5 where pro_c_id=10;
UPDATE 1
```

Step 7 Query the content of the **property** table.

```
finance=# select *,hash from ledgernsp.property;
 pro_id | pro_c_id | pro_pif_id | pro_type |       pro_status       | pro_quantity | pro_income |
pro_purchase_time   |          hash
--------+----------+------------+----------+------------------------+--------------+------------+---------------------
+------------------
        1 |        5 |          1 |        1 | Available              |            4 |       8000 | 2018-07-
01 00:00:00 | f3de57e69253919e
        3 |       15 |          3 |        3 | Available              |            4 |       8000 | 2018-07-
01 00:00:00 | a57b0e921843d811
        4 |       20 |          4 |        1 | Frozen                 |            4 |       8000 | 2018-
07-01 00:00:00 | 225f6c904e149e9d
        2 |       10 |          2 |        2 | Available              |            5 |       8000 | 2018-07-
01 00:00:00 | 5aa59018a53ddc7c
(4 rows)
```

Step 8 Query the historical content of the **property** table.

```
finance=# select * from blockchain.ledgernsp_property_hist;
 rec_num |       hash_ins       |       hash_del       |                 pre_hash
---------+----------------------+----------------------+----------------------------------
       0 | f3de57e69253919e |                      | b2f0373e538907da708489dd3eab1caf
       1 | c81d20819d5917d6 |                      | 80c9a338e81b73334b123fb76b76ec33
       2 | a57b0e921843d811 |                      | 9d1b724a131fbbf87f3b0fb343f138c1
       3 | 225f6c904e149e9d |                      | 5f391dbe6dd01584b5e9a37a5dcbb0f9
       4 | 5aa59018a53ddc7c | c81d20819d5917d6 | 5c4c248e73b2cb97d6d4113b9d2b0356
(5 rows)
```

Step 9 Query the content of table **gs_global_chain**. The preceding two DML operations are displayed.

```
finance=# SELECT * FROM gs_global_chain;
 blocknum | dbname  | username |              starttime               | relid |  relnsp   | relname  |
relhash          |                globalhash                |
                          txcommand
----------+---------+----------+--------------------------------------+-------+-----------+----------+------------------
+----------------------------------+----
------------------------------------------------------------
          1 | finance | omm      | 2023-10-21 10:12:27.916749+08 | 17540 | ledgernsp | property |
83d5f38a96052022 | fb01ebb45d887101353477022b02750e | insert into ledgernsp.property select *
from property;
          2 | finance | omm      | 2023-10-21 10:16:15.733258+08 | 17540 | ledgernsp | property |
92886f9707e4c4a6 | 63be71a0b61971364c12aa5d958b9f7a | update ledgernsp.property set
pro_quantity = 5 where pro_c_id=10;
(2 rows)
```

Step 10 Find the transaction whose **pro_c_id** is **10**, and change its **pro_quantity** value from **5** to **8**.

```
gsql -d finance -p 26000 -r
```

Step 11 Query the **property** table and check the hash value corresponding to the transaction whose **pro_c_id** is **10**.

```
finance=# select *,hash from    ledgernsp.property where pro_c_id=10;
 pro_id | pro_c_id | pro_pif_id | pro_type |        pro_status        | pro_quantity | pro_income |
pro_purchase_time   |           hash
--------+----------+------------+----------+--------------------+--------------+------------+--------------------
+------------------
         2 |          10 |              2 |           2 | Available         |                8 |          8000 | 2018-07-
01 00:00:00 | 7b27d899cca5b211
(1 row)
```

Step 12 Combine **blockchain.ledgernsp_property_hist** and the **gs_global_chain** view to view the complete revision history of this transaction.

```
finance=> WITH RECURSIVE res AS (
      SELECT xmin, rec_num, hash_ins::varchar,hash_del::varchar
      FROM blockchain.ledgernsp_property_hist WHERE hash_ins::varchar = '7b27d899cca5b211'
      UNION
      SELECT m.xmin, m.rec_num, m.hash_ins::varchar || ' > ' || r.hash_ins::varchar, m.hash_del::varchar
      FROM res r INNER JOIN blockchain.ledgernsp_property_hist m
      ON m.hash_ins::varchar = r.hash_del::varchar
)
select res.*,gc.starttime,gc.txcommand from res inner join gs_global_chain gc on res.xmin = gc.xmin;
 xmin   | rec_num |                                hash_ins                                 |        hash_del        |
starttime                 |                                txcommand

-------+---------+-------------------------------------------------------------------+------------------------+----------------------
---------+-------------------------------------------
-
 72081 |          5 | 7b27d899cca5b211                                                  | 5aa59018a53ddc7c
| 2023-10-21 11:52:00.423596+08 | update ledgernsp.property set pro_quantity=
 72057 |          4 | 5aa59018a53ddc7c > 7b27d899cca5b211                               |
c81d20819d5917d6 | 2023-10-21 10:16:15.733258+08 | update ledgernsp.property set pro_quantity
 72056 |          1 | c81d20819d5917d6 > 5aa59018a53ddc7c > 7b27d899cca5b211 |
| 2023-10-21 10:12:27.916749+08 | insert into ledgernsp.property select * fro
(3 rows)
```